UNDERGRADUATE THESIS

POTATO LEAF DISEASE CLASSIFICATION USING TRANSFER LEARNING BASED MOBILENETV3SMALL ARCHITECTURE

MUHAMMAD AHSANUR RAFI
NPM 21081010305

THESIS ADVISOR
Dr. Budi Nugroho, S.Kom., M.Kom.
Chrystia Aji Putra, S.Kom., M.T.

MINISTRY OH HIGHER EDUCATION, SCIENCE, AND TECHNOLOGY
UNIVERSITAS PEMBANGUNAN NASIONAL VETERAN JAWA TIMUR
FACULTY OF COMPUTER SCIENCE
INFORMATICS STUDY PROGRAM
SURABAYA
2026

UNDERGRADUATE THESIS

POTATO LEAF DISEASE CLASSIFICATION USING TRANSFER LEARNING BASED MOBILENETV3SMALL ARCHITECTURE

MUHAMMAD AHSANUR RAFI
NPM 21081010305

THESIS ADVISOR
Dr. Budi Nugroho, S.Kom., M.Kom.
Chrystia Aji Putra, S.Kom., M.T.

MINISTRY OH HIGHER EDUCATION, SCIENCE, AND TECHNOLOGY
UNIVERSITAS PEMBANGUNAN NASIONAL VETERAN JAWA TIMUR
FACULTY OF COMPUTER SCIENCE
INFORMATICS STUDY PROGRAM
SURABAYA
2026

This page is intentionally left blank

APPROVAL SHEET

POTATO LEAF DISEASE CLASSIFICATION USING TRANSFER LEARNING-BASED MOBILENETV3-SMALL ARCHITECTURE

By:
MUHAMMAD AHSANUR RAFI
NPM. 21081010305

Has been defended before, and accepted by, the Board of Assessors of the Thesis Examination of the Informatics Study Program, Faculty of Computer Science, Universitas Pembangunan Nasional Veteran Jawa Timur, on 17 Juni 2026:

Approved,

Dr. Budi Nugroho, S.Kom. M.Kom.
NIP. 19800907 202121 1 005 ..... **(Advisor I)**

Chrystia Aji Putra, S.Kom, M.T
NIP. 19861008 202121 1 001 ..... **(Advisor II)**

Dr. Rizky Parlika, S.Kom, M.Kom.
NIP. 19840518 2021211 003 ..... **(Head Assessor)**

Achmad Junaidi, S.Kom, M.Kom.
NIP. 3 7811 04 0199 1 ..... **(Assessor I)**

Acknowledge by,

Dean of the Faculty of Computer Science

Prof. Dr. Ir. Novirina Hendrasarie, MT.
NIP. 19681126 199403 2 001

This page is intentionally left blank

APPROVAL SHEET

Potato Leaf Disease Classification Using Transfer Learning Based MobileNetV3Small Architecture

By:
MUHAMMAD AHSANUR RAFI
NPM. 21081010305

Approved by,

Informatics Study Program Coordinator
Faculty of Computer Science

Dr. Intan Yuniar Purbasari, S.Kom., M.Kom.
NIP. 19800602 202521 2 029

Dipindai dengan CamScanner

This page is intentionally left blank

STATEMENT OF ORIGINALITY

I am the undersigned:

Student Name	: Muhammad Ahsanur Rafi
NPM	: 21081010305
Degree Program	: Bachelor Degree
Study Program	: Informatics
Faculty	: Faculty of Computer Science

Hereby declares that this undergraduate thesis contains no part of any other scientific work that has been submitted to obtain an academic degree at any higher education institution. Furthermore, it does not contain any work or opinions previously written or published by others, except for those which are explicitly cited in this thesis and listed completely in references.

And I declare that this scientific document is free from elements of plagiarism. If in the future indications of plagiarism are found in this Thesis, I am willing to accept sanctions in accordance with the applicable laws and regulations. Thus, I made this statement without any coercion from anyone and to be used as it should.

Surabaya, July 22, 2026
Declarant,



Muhammad Ahsanur Rafi
NPM.21081010305

Dipindai dengan CamScanner

This page is intentionally left blank

ABSTRACT

Student Name / NPM :	Muhammad Ahsanur Rafi / 21081010305
Thesis Title :	Potato Leaf Disease Classification Using Transfer Learning Based MobileNetV3Small Architecture
Supervisors :	1. Dr. Budi Nugroho, S.Kom. M.Kom. 2. Chrystia Aji Putra, S.Kom, M.T

Leaf disease infections (*early* and *late blight*) in potatoes have the potential to reduce harvest quality. To overcome the limitations of manual visual identification, this research develops a *deep learning* classification model using the efficient MobileNetV3-Small architecture based on *transfer learning*. Preprocessing leverages a naturally balanced dataset, pixel standardization to [-1.0, 1.0] to match the pre-trained ImageNet weights, and dynamic augmentation (*Gaussian blur, noise*, lighting variation) to simulate real-world environmental disturbances. The *hyperparameter* configuration was optimized using *Bayesian Optimization*. Training was executed in two phases: feature extraction (Adam *optimizer*) and *fine-tuning* of the final 40 layers (SGD Nesterov). The test results show that this architecture is efficient, with an internal accuracy reaching 98.15%. However, the cross-dataset (external) evaluation recorded a drop in accuracy to 84.67%, confirming the occurrence of *domain shift*. Although robust in detecting infected specimens, the model exhibited a hypersensitivity bias toward healthy leaves (*Healthy*), in which foreign elements of the real environment were misinterpreted as pathology. This research concludes that this lightweight architecture has potential for field diagnostics, but the robustness of its inference absolutely requires enrichment of the physical dataset's diversity during training in order to suppress *false positives* caused by external visual variation.

Keywords: *Deep Learning, MobileNetV3-Small, Transfer Learning, Domain Shift, Hyperparameter Optimization.*

This page is intentionally left blank

ACKNOWLEDGEMENTS

The author is grateful to Allah SWT for the grace, guidance, and gifts that have been conferred upon them, enabling them to successfully complete the research report for the thesis titled "Analysis of the Influence of Influencers on Bitcoin Price Prediction Using LSTM." The author would like to extend his appreciation to Mr. Dr. Rizky Parlika, S.Kom., M.Kom., as Supervisor I and Mr. Achmad Junaidi, S.Kom., M.Kom., as Supervisor II, for their assistance, advice, and motivation. Furthermore, the author received substantial support from a variety of sources during the development of this thesis report. In recognition of this, the author would like to extend their gratitude to:

1. Prof. Dr. Ir. Akhmad Fauzi, M.MT., IPU., ASEAN.Eng. as the Rector of the National Development University “Veteran” East Java.
2. Prof. Dr. Ir. Novirina Hendrasarie, MT as Dean of the Faculty of Computer Science, National Development University "Veteran" East Java.
3. Dr. Intan Yuniar Purbasari, S.Kom. MSc. as the Coordinator of the Informatics Study Program, Faculty of Computer Science, National Development University "Veteran" East Java.
4. Mr/Mrs Lecturer of Informatics Study Program, National Development University "Veteran" East Java for their guidance during the author's education.
5. Parents, family, and closest friends who have provided support, prayers, and motivation during the completion of this thesis,

T he author acknowledges that this thesis has numerous deficiencies. Constructive criticism and suggestions from all parties are therefore greatly valued in the pursuit of this thesis's perfection. Ultimately, I trust that this report will be advantageous to all parties, including the author, despite its constraints.

Surabaya, July 22, 2026

Writer

This page is intentionally left blank

LIST OF CONTENTS

APPROVAL SHEET ii
APPROVAL SHEET iv
STATEMENT OF ORIGINALITY vi
ABSTRACT viii
ACKNOWLEDGEMENTS x
LIST OF CONTENTS xii
LIST OF FIGURES xv
LIST OF TABLES xvii
CHAPTER I INTRODUCTION 1
1.1 Background 1
1.2 Problem Formulation 3
1.3 Research Objectives 4
1.4 Research Benefits 4
1.5 Scope of the Problem 5
CHAPTER II LITERATURE REVIEW 7
2.1 Previous Research 7
2.2 Potato 9
2.2.1 *Early blight* 10
2.2.2 *Late blight* 11
2.2.3 Healthy 12
2.3 *Deep Learning* and *Convolutional Neural Network (CNN)* 13
2.4 *MobileNetV3-Small* Architecture 14
2.4.1 Convolutional Layer 16
2.4.2 Pooling layer 17
2.4.3 Activation Function 18
2.4.4 Depthwise and Pointwise Convolution 18
2.4.5 Squeeze-and-Excitation (SE) Block 20
2.4.6 Inverted Residual Block 20
2.4.7 Fully Connected Layer (Dense Layer) 20
2.4.8 Bottleneck 21
2.5 Transfer Learning 22
2.6 Optimization Algorithms 23

2.7 *Loss Function: Categorical Crossentropy* 24
2.8 Callback Functions: *Early Stopping* and *ReduceLROnPlateau* 25
2.9 Performance Evaluation Metrics 26
2.10 Data Augmentation 28
2.10.1 Rotation Range 29
2.10.2 Horizontal Flip 29
2.10.3 Zoom Range 30
2.11 Data Normalization and Preprocessing 31
CHAPTER III METHODOLOGY 34
3.1 Literature Study 35
3.2 Dataset Collection 35
3.3 Preprocessing and Data Splitting 37
3.3.1 Dataset Splitting (*Data Splitting*) 38
3.3.2 Data Augmentation 39
3.3.3 Image Dimension Adjustment and Normalization 46
3.4 Design of the *MobileNetV3Small* Model Architecture 47
3.5Two-Phase Model Training Scenario 52
3.5.1. First Phase: Feature Extraction (*Feature Extraction* / *Freeze*) 53
3.5.2. Second Phase: Fine-Tuning (*Fine-Tuning/ Unfreeze*) 55
3.5.3. Loss Function and Optimization Algorithm 58
3.5.4. Training Control Mechanism (*Callbacks*) 59
3.6 Testing and Hyperparameter Scenario 60
3.7 Testing and Performance Evaluation 62
3.7.1. *Classification Report* 63
3.7.2. *Confusion Matrix* 64
CHAPTER IV RESULTS AND DISCUSSION 67
4.1 System Development Environment 67
4.1.1 Hardware 68
4.1.2 Software 69
4.2 Dataset Preparation and Exploration 71
4.3 Data Preprocessing and Augmentation (*Preprocessing*) 72
4.4 MobileNetV3-Small Architecture Design 75
4.5 Implementation of Two-Phase Model Training (*Transfer Learning*) 79
4.5.1. Phase 1: *Feature Extraction* (Feature Extraction) 79

4.5.2. Phase 2: *Fine-Tuning* (Fine Adjustment) 80
4.6 Testing Scenario and Hyperparameter Optimization 81
4.6.1 Data Proportion 81
4.6.2 Optimizer 86
4.6.3 Model 91
4.6.4. Hyperparameter Optimization (Bayesian Optimization) 96
4.7 Analysis of Training Stability and the Effect of *Callbacks* 98
4.8 Testing and Performance Evaluation of the Final Model 101
4.8.1 Internal Testing 102
4.8.2 External Testing 104
CHAPTER V CONCLUSION AND SUGGESTIONS 109
5.1 Conclusion 109
5.2 Suggestions 110
BIBLIOGRAPHY 111
APPENDICES 116
1. Program Code 4.2 116
2. Program Code 4.3 117
3. Program Code 4.6 118
4. Program Code 4.8 119
5. Certificate of Turnitin Check Results 120

LIST OF FIGURES

Figure 2. 1 Early blight Disease on a Potato Leaf 10
Figure 2. 2 Late blight Disease on a Potato Leaf 11
Figure 2. 3 Healty Potato Leaf 12
Figure 2. 4 CNN Architecture 14
Figure 2. 5 Flow of the MobileNetV3-Small Architecture 15
Figure 2. 6 The Convolution Operation Process in the Convolutional Layer 16
Figure 2. 7 The Pooling Operation Process 17
Figure 2. 8 Depthwise and Pointwise Convolution 18
Figure 2. 9 Fully Connected Layer 21
Figure 2. 10 Comparison of Training Schemes 22
Figure 3. 1 Research Flow 34
Figure 3. 2 Healthy Potato Leaf, *Late blight* and *Early blight* Diseases 36
Figure 3. 3 Visualization of the Number of Images per Class 37
Figure 3. 4 Data Preprocessing Flowchart 38
Figure 3. 5 Data Augmentation Process Flow 40
Figure 3. 6 The Augmentation Techniques Used 40
Figure 3. 7 Horizontal Flip Result 42
Figure 3. 8 Zoom Range Result 42
Figure 3. 9 Rotation Range Result 43
Figure 3. 10 Gaussian Blur Result 44
Figure 3. 11 Gaussian Noise Result 45
Figure 3. 12 Brightness Variation Result 46
Figure 3. 13 Detail of the MobileNetV3Small Architecture 49
Figure 3. 14 Initial Layer of MobileNetV3Small 50
Figure 3. 15 The Bottleneck Block Process 51
Figure 3. 16 Final Architecture of MobileNetV3Small 52
Figure 3. 17 Block Diagram of the Layer Freezing Status 54
Figure 3. 18 Block Diagram of the Status of Partially Unfrozen Layers 57
Figure 4. 1 Confusion Matrix 80:10:10 82
Figure 4. 2 Classification 80:10:10 82

Figure 4. 3 Confusion Matrix 70:15:15 83
Figure 4. 4 Classification Report 70:15:15 84
Figure 4. 5 Confusion Matrix 60:20:20 84
Figure 4. 6 Classification Report 60:20:20 85
Figure 4. 7 Confusion Matrix Adam-SGD 86
Figure 4. 8 Classification Report Adam-SGD 87
Figure 4. 9 Confusion Matrix Adam-Adam 87
Figure 4. 10 Classification Report Adam-Adam 88
Figure 4. 11 Confusion Matrix SGD Nesterov-SGD Nesterov 89
Figure 4. 12 Classification Report SGD Nesterov-SGD Nesterov 90
Figure 4. 13 Confusion Matrix MobileNetV3Small 91
Figure 4. 14 Classification Report MobileNetV3Small 92
Figure 4. 15 Confusion Matrix MobileNetV3Large 92
Figure 4. 16 Classification Report MobileNetV3Large 93
Figure 4. 17 Confusion Matrix MobileNetV2 94
Figure 4. 18 Classification Report MobileNetV2 95
Figure 4. 19 Accuracy and Loss Movement Curve 101
Figure 4. 20 Confusion Matrix on the Internal Testing 104
Figure 4. 21 Confusion Matrix on the External Testing 106

LIST OF TABLES

Table 3. 1 Dataset Distribution Allocation 39
Table 3. 2 Hyperparameter Configuration and Parameter Status of the First Phase 54
Table 3. 3 Hyperparameter Configuration and Parameter Status of the Second Phase 58
Table 3. 4 Testing Scenario 61
Table 3. 5 Hyperparameter Search Space (Search Space) 62
Table 3. 6 Confusion Matrix Framework 65
Table 4. 1 Hardware Specifications 68
Table 4. 2 Software Specifications 69
Table 4. 3 Distribution of the Number of Potato Leaf Image Samples Per Class 72
Table 4. 4 Data Proportion Test Results 85
Table 4. 5 Optimizer Test Results 90
Table 4. 6 Model Test Results 95
Table 4. 7 Summary of the Top-3 Bayesian Optimization Results 96
Table 4. 8 Classification Report of the Internal Testing Results 102
Table 4. 9 Classification Report of the External Testing Results 105

This page is intentionally left blank

CHAPTER I
INTRODUCTION

This chapter discusses the research background, which covers the importance of classifying potato leaf diseases as well as the influence of technology in supporting the early detection and handling of these diseases. In addition, this research presents the problem formulation, objectives, benefits, and scope of the study, which are elaborated in detail. The purpose of this chapter is to convey a general overview within the research context and the importance of the chosen topic, so that it can provide a strong framework for further discussion in the subsequent chapters.

1.1 Background

The potato (*Solanum tuberosum*) is one of the tuber crops that serves as an important source of carbohydrates besides rice, wheat, and corn. This plant is widely used as an alternative food to replace rice, especially for people with diabetes, because its glucose content is relatively lower and therefore safer to consume [1].

Potato production in East Java was recorded to have increased from 278,804 tons in 2023, rising to 301,875 tons in 2024. This increase in production volume shows that potatoes still hold great potential as a leading food commodity [2]. However, this production increase needs to be balanced with efforts to maintain the stability of harvest yields so that they do not fluctuate in the following years. One important factor in maintaining or even increasing potato productivity is preserving the quality of the plant itself, whether in terms of seed, cultivation techniques, or pest and disease control. Leaf quality, as the main organ of photosynthesis, becomes a crucial aspect that must be maintained, because damage to the leaves due to disease can reduce the efficiency of photosynthesis and directly affect the resulting tuber yield. Therefore, efforts toward prevention and early detection of potato leaf diseases become a strategic step in supporting the sustainability of production increases as well as maintaining the resilience of the potato commodity at the farmer level.

Diseases of potato leaves generally originate from fungi carried through seed tubers. The growth process of this fungus is strongly influenced by environmental conditions, especially temperature, so that at certain temperatures the fungus can develop more quickly and aggressively. Besides attacking parts of the plant, the fungus can also produce a toxic compound called solanine, which

not only harms the quality of the harvest but can also be dangerous if the tubers are still consumed. On an agricultural scale, the presence of this fungal disease becomes one of the serious threats to potato productivity. In general, there are two main types of disease caused by fungi in potato plants, namely *early blight* and *late blight*, each of which has different symptom characteristics but both have a significant impact on production yields [3].

Efforts to prevent, identify, and handle diseases in potato plants require an improved understanding to distinguish between healthy plants and infected plants. With good knowledge, control measures can be carried out more quickly and accurately, so that the risk of loss can be significantly reduced [4].

Diseases in potato plants can usually be recognized through differences in color, shape, and texture of the leaves. These visual factors become early signs that greatly assist the identification process. Therefore, the development of technology-based methods that can detect these changes automatically becomes an urgent need to support modern agricultural systems. Based on these problems, a method is needed that is capable of identifying the type of disease in potato plants in a computerized manner through digital image processing. This method is expected to be able to improve time efficiency and diagnostic accuracy, while also minimizing dependence on manual examination that requires special expertise [5].

Along with technological developments, digital image processing is increasingly being used to support the resolution of various problems in the field of *image processing*. This technology not only functions to process images, but can also perform pattern analysis, segmentation, and classification, all of which can be applied to problems in the agricultural sector. The application of digital image processing has been proven to provide positive impacts on human life, especially in supporting the food sector. One example is the application of digital image classification to obtain optimal accuracy values in detecting plant diseases, so that harvest yields can be preserved and food quality remains guaranteed [6].

Potato products of good quality are very important so that consumption levels remain stable and market competitiveness can increase. Therefore, an image-based plant disease classification method is needed that is not only accurate but also computationally efficient, so that it has the potential to be implemented on low-resource devices (*mobile devices*). Within the realm of *deep learning*, the *Convolutional Neural Network* (CNN) has proven to be the most reliable architecture

in extracting image features to detect plant diseases, surpassing the capabilities of conventional image processing methods [7].

Among the various CNN architectures, the MobileNet family, particularly MobileNetV3-Small, is specifically designed to minimize computational load and memory usage without drastically sacrificing classification performance. Research shows that MobileNetV3 is able to provide the best balance between inference speed and accuracy, making it an ideal model for *real-time* agricultural applications in the field [8]. This advantage is obtained through the use of *depthwise separable convolution* modules and the *squeeze-and-excitation* mechanism, which are highly efficient in selecting visual features [9].

Despite having advantages in terms of efficiency, lightweight (*lightweight*) architectures are often faced with the challenge of limited accuracy when trained from scratch (*from scratch*) using a relatively small dataset, such as specific potato leaf images. To overcome this obstacle, the *Transfer Learning* approach has proven to be highly effective. By leveraging the weights of a model that has been trained on a massive-scale dataset (such as ImageNet), a lightweight architecture can quickly adapt to recognize new plant disease patterns [10].

Furthermore, the success of *Transfer Learning* on the MobileNetV3 architecture strongly depends on the optimization strategy applied, including the suitability of the image preprocessing function (*preprocessing*) and the *fine-tuning* stage. Careful hyperparameter adjustment on the *classifier* layers and the *base layer* can prevent the phenomenon of forgetting the initial weights (*catastrophic forgetting*), so that it can push the model's accuracy close to its maximum level [10].

Based on this potential, this research aims to implement and optimize the performance of MobileNetV3-Small using the *Transfer Learning* technique and a dynamic *fine-tuning* strategy in classifying *Early Blight*, *Late Blight*, and healthy leaves in potato plants.

1.2 Problem Formulation

The problem formulation is arranged to clarify the focus of the research and to serve as a guide in the analysis process. Based on the background that has been described, the problem formulations in this research are:

1. How is the implementation process of the MobileNetV3-Small model in classifying potato leaf images?
2. How is the pixel distribution of input images standardized using the official *preprocessing* function so that it matches the characteristics of the *pre-trained* weights?

3. How does the two-phase optimization strategy (*Feature Extraction* using Adam and *Fine-Tuning* using SGD with Nesterov momentum) affect the overall stability and performance of the model based on training, validation, and test data metrics?

1.3 Research Objectives

Based on the problem formulation above, the objectives of this research are as follows:

1. To implement the MobileNetV3-Small architecture using the Transfer Learning technique in classifying potato leaf disease images.
2. To implement the process of standardizing the pixel distribution of input images using the official *preprocessing* function so that it matches the characteristics of the *pre-trained* weights.
3. To evaluate the effect of the two-phase optimization strategy (*Feature Extraction* using the Adam algorithm and *Fine-Tuning* using SGD with Nesterov momentum) on the stability and performance of the model based on training, validation, and test data metrics.

1.4 Research Benefits

The research benefits are described to show the contributions that can be made, both theoretically and practically. The benefits of this research include:

1. Academically.

   Providing additional knowledge and scientific contribution regarding the application of *deep learning*, particularly the lightweight *convolutional neural network* (CNN) architecture MobileNetV3-Small, in the field of computer engineering for plant disease image classification. In addition, this research can serve as reference material for further studies that focus on optimizing model architecture through engineering of the *fine-tuning* stages.

2. Practically

   Providing a concrete illustration of how the accuracy of selecting the preprocessing function and the optimization algorithm scenario significantly affects model accuracy. The results of this research are also expected to serve as a technical reference in the development of efficient digital-image-based plant disease detection systems to support the agricultural sector.

1.5 Scope of the Problem

The scope of the problem is defined so that the research boundaries become clearer and focused on the intended aspects. The scope of the problem in this research is as follows:

1. Classification is limited to three classes, namely Healthy, Early Blight, and Late Blight.
2. The scope of this research extends only to the development and evaluation of the model, and does not include the deployment stage.

This page is intentionally left blank

CHAPTER II
LITERATURE REVIEW

This chapter describes a number of previous studies relevant to the topic of plant disease detection and classification, particularly for the potato commodity, as references and a scientific foundation. The literature review covers various approaches and results of *deep learning* model implementations, with particular emphasis on the use of the lightweight and efficient *Convolutional Neural Network* (CNN) architecture, namely MobileNet. The focus of the review is also directed at preprocessing strategies and model training optimization techniques. In addition, this chapter describes the performance evaluation metrics used to validate classification reliability, thereby forming a comprehensive and measurable analytical framework for this research.

2.1 Previous Research

Before conducting further analysis, it is necessary to understand a number of previous studies that are directly related to the topic of plant disease detection. This literature review is focused on studies that use the same database, namely the Potato Leaf Disease Dataset (PLD), in order to see the extent of the development of methods and the accuracy achievements of current deep learning architectures. This review becomes the basis for finding room for improvement in building a classification model that balances high accuracy and computational efficiency.

Research by [11] developed a multi-level *deep learning* model for recognizing potato leaf diseases. This study proposed a complex architecture that combines the PDDCNN model for the classification stage and the YOLOv5 algorithm for the object detection stage on leaf images. Through an extensive training scenario, this combined model succeeded in reaching a very high accuracy level, namely 99.75%. Although this accuracy achievement is almost perfect, combining two large architectures at once causes a very heavy computational load, so that its application is less efficient when implemented on devices with limited resources or *mobile* devices.

Research by [12] focused on evaluating the effectiveness of data augmentation techniques in detecting plant leaf diseases using the EfficientNetV2B0 architecture. The main focus of this study was to analyze how various image augmentation techniques (such as rotation, translation, and *scaling*) affect the model's robustness against a limited dataset. The experimental results show that with the right augmentation engineering, the EfficientNetV2B0 architecture is able to reach accuracy of up to 99.75%. This study proves that image preprocessing and augmentation are

crucial steps that can significantly optimize feature extraction without always having to overhaul the basic structure of the artificial neural network.

Research by [13] proposed a special network architecture designed specifically for potato leaf disease classification, named PLDC-Net (*Potato Leaf Disease Classification Network*). This model was built by modifying the convolution layers to make them more sensitive to the specific texture patterns of *Early Blight* and *Late Blight* diseases. After undergoing a training and validation process using the PLD dataset, this *custom* model succeeded in recording an accuracy of 98.39%. These results show that the development of a domain-specific architecture is able to provide promising performance, although its development requires iteration time and *hyperparameter* tuning from scratch.

Research by [14] explored the use of the EfficientNet-B2 architecture as a *deep learning* approach to detect potato plant diseases at an early stage (*early treatment*). This approach leverages the principle of *Transfer Learning* to reduce training time on agricultural image datasets. Based on the results of the potato leaf disease classification test, the EfficientNet-B2 architecture proved fairly solid with an accuracy achievement of 97.33%. This study confirms that the use of a pre-trained mid-level architecture is highly relevant for disease detection needs, although there is still room for further efficiency in terms of the model's parameter size.

Research by [15] offered an industrial-grade solution for the plant disease image detection task. This study used the YOLO V5-CACT algorithm, which has been modified to be able to process *real-time* detection in the field at high speed. Evaluation of the model against the potato leaf disease dataset shows that this architecture is able to achieve an accuracy of 95.60%. The conclusion of this study highlights the *trade-off* between inference speed and accuracy, in which a system focused on industrial operational speed often experiences a slight decrease in absolute classification accuracy compared to a heavier model.

Based on the review of the five studies above, a clear research gap can be seen. Models that are able to reach accuracy above 99% depend heavily on a heavy architecture or a combination of multiple models, whereas faster models often lag in terms of accuracy. Therefore, this research is designed to fill that gap by using a very lightweight *deep learning* model, namely MobileNetV3-Small. By adopting a precise *preprocessing* function and controlling the layered *fine-tuning* engineering, this research aims to prove that a lightweight architecture is able to reach a high and

stable level of accuracy for potato leaf disease classification without requiring a massive computational load.

2.2 Potato

The potato (*Solanum tuberosum L.*) is one of the important food crops widely consumed throughout the world because it contains an energy source in the form of carbohydrates and is rich in micronutrients, such as vitamin C, potassium, and fiber. In addition, potatoes contain bioactive compounds, including polyphenols and carotenoids, that are beneficial for health, making them a functional food with potential for the prevention of degenerative diseases. Their wide use, both as a staple food and as a raw material for the processed food industry, gives potatoes an important role in supporting global food security and contributing to increasing the economic value of the agricultural sector in various countries. As a commodity with high adaptability, potatoes are able to grow at various elevations and soil conditions, making them a primary choice in food security programs in developing countries. Their ability to be processed into various processed products, such as potato flour, chips, and industrial raw materials, further strengthens their position as a strategic crop that not only supports direct consumption but also sustains the food industry chain broadly [16].

Besides being a main source of energy, potatoes are also known to contain various bioactive compounds, such as polyphenols, carotenoids, and vitamin C, which play an important role as natural antioxidants to help reduce the risk of degenerative diseases, particularly cardiovascular disease and cancer. The phytochemical content in potatoes varies across each variety, particularly in the pigment of the skin and flesh of the tuber. Purple and red varieties tend to have higher levels of polyphenols and anthocyanins compared to white- or yellow-skinned varieties. In addition, potatoes also play an important role in food diversification because they have a relatively short harvest cycle, lower water requirements compared to cereal crops, and can be cultivated under various agroecological conditions. This makes potatoes one of the strategic commodities for supporting national food security in many countries [17].

Two main diseases that frequently attack potato plants and cause significant damage to the leaves are *early blight* and *late blight*. *Early blight* disease is caused by the fungus *Alternaria solani*, which generally attacks plants in the mid to late growth phase, marked by the appearance of round, blackish-brown spots on old leaves surrounded by concentric rings. Meanwhile, *late blight* disease is caused by the pathogen *Phytophthora infestans*, which attacks the leaves, stems, and tubers,

with initial symptoms in the form of grayish-green spots that rapidly enlarge into blackish-brown lesions until they cause the leaves to rot. These two diseases can reduce potato harvest yields by up to 70% if not controlled properly, because their spread is very rapid, especially under conditions of high humidity and low temperature [18].

2.2.1 *Early blight*

Early blight is one of the important diseases of potato plants caused by the fungus *Alternaria solani* and usually attacks the lower leaves first before spreading to the entire plant canopy. Its symptoms are marked by the appearance of circular blackish-brown spots with a distinctive concentric ring pattern on old leaves, which can slowly cause early drying and dropping of the leaves. Environmental conditions with warm temperatures and high humidity strongly support the development of this pathogen, thereby accelerating the reduction of the photosynthetic area and directly affecting the reduction of tuber yields. A study shows that *early blight* infection can reduce potato harvest yields by more than 30% in susceptible varieties if control is not carried out properly [19]. An illustration of *early blight* disease on a potato leaf can be seen in Figure 2.1.



Figure 2. 1 Early blight Disease on a Potato Leaf

Recent research also shows that the severity of *early blight* in potatoes is influenced not only by weather factors but also by land conditions and the plant's nutritional status. Land with high sand content tends to have greater disease severity because the light soil structure accelerates water loss and causes stress in the plant, thereby increasing its susceptibility to infection. Conversely, high leaf potassium content has been shown to be negatively correlated with disease severity, in which plants with low potassium levels experience more severe infection. Field trials conducted show

that the application of potassium fertilizer is able to significantly reduce the severity of *early blight*, especially in varieties that were previously classified as susceptible. This study also affirms that agricultural management practices such as crop rotation and the selection of disease-resistant varieties play an important role in reducing the spread of *early blight* in the field. Therefore, an integrated management strategy that includes aspects of nutrition, variety selection, and soil management is considered more effective than a single fungicide-based approach [20].

2.2.2 *Late blight*

Late blight is one of the most destructive diseases of potato plants, caused by the pathogen *Phytophthora infestans*. This pathogen is able to attack the leaves, stems, and tubers, thereby having a serious impact on the productivity and quality of the harvest. Initial symptoms usually appear as greenish or grayish spots on the surface of moist leaves, which then develop into dark brown to black necrosis, accompanied by a yellowish halo around them. The infection can spread very quickly, especially under environmental conditions of high humidity and relatively low temperature; even the entire plant canopy can become infected in a matter of days. Recent studies affirm that delays in early detection can cause harvest losses of more than 70% in susceptible varieties, so that effective detection and control become highly crucial. With technological advances, approaches based on remote sensing and spectral analysis, particularly in the red and *red-edge* regions, have been shown to be able to detect *late blight* attacks both at the leaf and canopy levels, thereby greatly assisting efforts at integrated monitoring and control strategies in the field [21]. An illustration of *late blight* disease on a potato leaf can be seen in Figure 2.2.



Figure 2. 2 Late blight Disease on a Potato Leaf

The severity of late blight disease in potatoes is strongly influenced by the timing of the onset of infection and the climatic conditions at that time, in which early infection produces far greater yield losses compared to when it occurs in the late growth phase. In addition, temporal mapping based on weather data and the use of multispectral imagery show that combined time-series models, such as MSTS-RF, in recent research are able to predict the level of late blight attack with high accuracy on a broad scale, utilizing vegetation indices and radar features as input variables [22].

2.2.3 Healthy

Potato leaves (*Solanum tuberosum L.*) categorized in the *healthy* class represent the optimal condition of the plant in which physiological processes, particularly photosynthesis, proceed maximally without the interference of pathogens. Morphologically, the main visual symptom of a healthy leaf is marked by fresh green pigmentation evenly distributed across the entire surface of the leaf blade (lamina), which indicates a stable level of chlorophyll production. In this condition, the leaf surface appears clean of any form of visual anomaly such as chlorosis (yellowing of the tissue), lesions, or brown or black necrotic spots. In addition, the anatomical structure of the leaf veins (*veins*) appears regular, and the leaf edges (*margin*) show no signs of curling (*curling*) or early drying. Maintaining a healthy leaf canopy is very crucial in potato cultivation, because the integrity of the photosynthetic area is directly correlated with high biomass accumulation and tuber yield productivity at harvest time [23]. An illustration of a healthy potato leaf can be seen in Figure 2.3.



Figure 2. 3 Healty Potato Leaf

In the context of developing computer vision and artificial intelligence systems, the healthy leaf class plays a very important role as a *baseline* or control class for comparison against the disease classes. The absence of concentric ring patterns as in *early blight* and irregular blistered areas as in *late blight* gives the healthy leaf a far more homogeneous distribution of pixel values and geometric texture. The visual consistency and absence of disease artifacts in this class help the convolutional neural network architecture (such as MobileNet) to learn clean background features, so that the model avoids bias and is able to suppress the *false positive* rate. Accurate discriminative identification between intact leaf tissue and tissue undergoing necrosis is the main foundation for the model to be reliable in precision agriculture monitoring scenarios in the field [24].

2.3 *Deep Learning* and *Convolutional Neural Network (CNN)*

Deep Learning is one of the branches of development of *Machine Learning* that is built on the architecture of Artificial Neural Networks (*Artificial Neural Networks*). The main advantage of *deep learning* compared to conventional *machine learning* methods lies in its ability to perform feature extraction automatically (automation of *feature engineering*). In classical methods, feature extraction such as leaf color or texture must be defined manually by the researcher using certain mathematical formulas. However, in *deep learning*, the algorithm is able to learn very complex feature representations hierarchically directly from a set of raw data provided, thereby producing far more accurate prediction or classification results on unstructured data such as images, text, and sound [25].

In the field of digital image processing, particularly for plant disease diagnosis, the *Convolutional Neural Network* (CNN) has become the most standard and reliable *deep learning* architecture. CNN is specifically designed to process two-dimensional data such as images while maintaining the spatial relationships between pixels. The feature extraction process in CNN is carried out through convolutional layers (*convolutional layers*) that use filters (*kernels*) to detect various visual patterns, ranging from simple patterns such as lines and the edges of leaf lesions, to complex patterns such as fungal infection spots. After features are extracted, the *pooling* layer is used to reduce the dimensionality of the data without losing important information, thereby lightening the computational load. This hierarchical capability is what makes the implementation of CNN proven to have very high potential and efficiency in detecting and classifying various plant diseases based on leaf images in *real-time* [25].

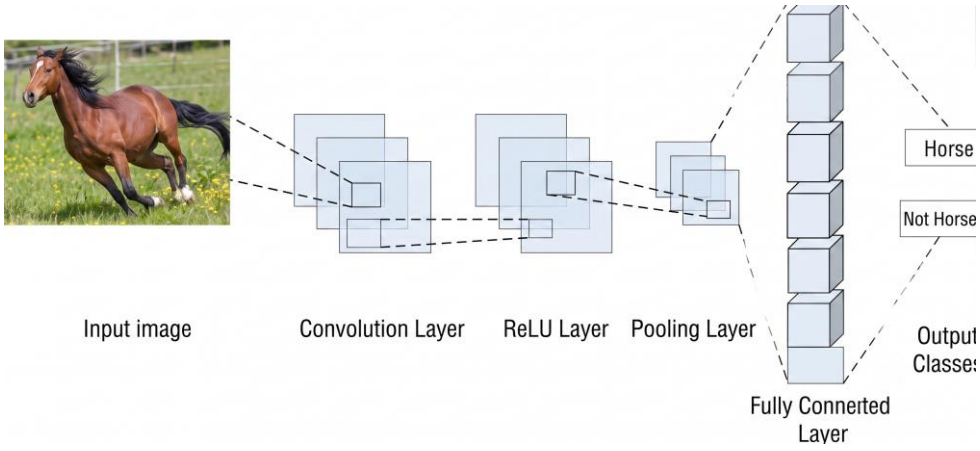


Figure 2. 4 CNN Architecture

Figure 2.4 displays the standard workflow of a *Convolutional Neural Network* (CNN) model in performing image classification. The process begins when an input image is fed into the network and processed by the *Convolutional Layer* to extract basic visual features such as lines, edges, and textures. The extraction results are then passed to the *ReLU Layer* to provide non-linearity properties, then enter the *Pooling Layer* which functions to reduce the spatial dimensions of the feature map so that the computational load becomes lighter without discarding important information. In the final stage, the feature matrix is flattened and passed to the *Fully Connected Layer*, a dense neural network that calculates the probabilities to determine the final classification in the *Output Classes*. This extraction and classification mechanism is what will later be adapted to recognize infection spot patterns on potato leaf images [26].

2.4 *MobileNetV3-Small* Architecture

MobileNetV3-Small is a variant of *MobileNetV3* designed specifically for devices with limited resources, emphasizing computational efficiency without significantly sacrificing performance. This architecture leverages *depthwise separable convolution* to reduce computational complexity, and integrates the *Squeeze-and-Excitation (SE)* module which functions to emphasize important features while suppressing less relevant ones. In addition, the *hard-swish* activation function is used to replace traditional non-linear activation because of its more computationally economical nature while still being able to maintain the quality of feature representation. With this combination, *MobileNetV3-Small* has been proven able to provide a balance between accuracy and efficiency, making it very ideal for image classification applications on devices with low computing power or *real-time mobile*-based systems [27].

MobileNetV3-Small has been used in corn leaf disease detection research with a lighter yet still effective approach. This model is designed to be efficient on devices with computational limitations, such as *smartphones* and *IoT*-based field systems. Research results show that the *improved MobileNetV3-Small* is able to reach high accuracy in the multi-class classification of corn leaf diseases, while maintaining a compact model size and fast inference time. This advantage affirms that *MobileNetV3-Small* is not only relevant for laboratory scenarios, but also highly potential for real-world applications in the field of precision agriculture that requires both detection speed and resource efficiency [28].

MobileNetV3-Small is an architecture capable of bridging the need between high accuracy and computational efficiency. The combination of lightweight design, small model size, and fast inference time makes it very suitable for application in plant disease detection systems based on mobile devices or *IoT*. Thus, *MobileNetV3-Small* is not only relevant in the context of academic research, but also has high practical value in supporting the application of precision agriculture technology in the field more effectively and sustainably.

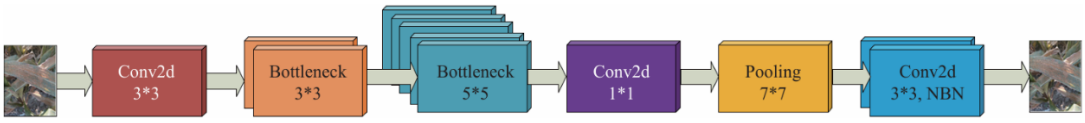


Figure 2. 5 Flow of the MobileNetV3-Small Architecture

Figure 2.5 shows that the *MobileNetV3-Small* architecture is designed with a systematic processing flow, from the input image to producing an output in the form of the classification result. The input image is first processed through a 3×3 convolution layer to extract basic features. Next, the obtained features are expanded through a series of *bottleneck blocks* and *inverted residual* units that make the network more efficient in capturing important information from the image. At this stage, the *Squeeze-and-Excitation (SE)* module is utilized to strengthen relevant features while dampening less important ones, whereas the *hard-swish* activation function is used so that the computation process is lighter without reducing the quality of the results. After that, the compacted features pass through the convolution and *pooling* layers to simplify the representation before being directed to the final layer in order to distinguish object classes. With this design, *MobileNetV3-Small* is able to maintain a balance between computational efficiency and image description quality, so that it remains effective for use on devices with limited resources [28].

2.4.1 Convolutional Layer

The *convolutional layer* functions as the main layer in the *Convolutional Neural Network* (*CNN*), tasked with extracting visual features from the image, such as edges, textures, shapes, and color differences. This layer works by using a small *kernel* or *filter* that moves across the input image and performs the convolution operation, producing a *feature map* that highlights important elements such as contours and disease spot patterns. In the early stages, the *convolutional layer* generally detects simple features such as edges or corners, whereas in deeper layers it is able to identify more complex patterns, including disease spot shapes and leaf textures. This layered process provides a strong foundation for the model to understand the image hierarchically and supports accuracy in plant disease classification [29].

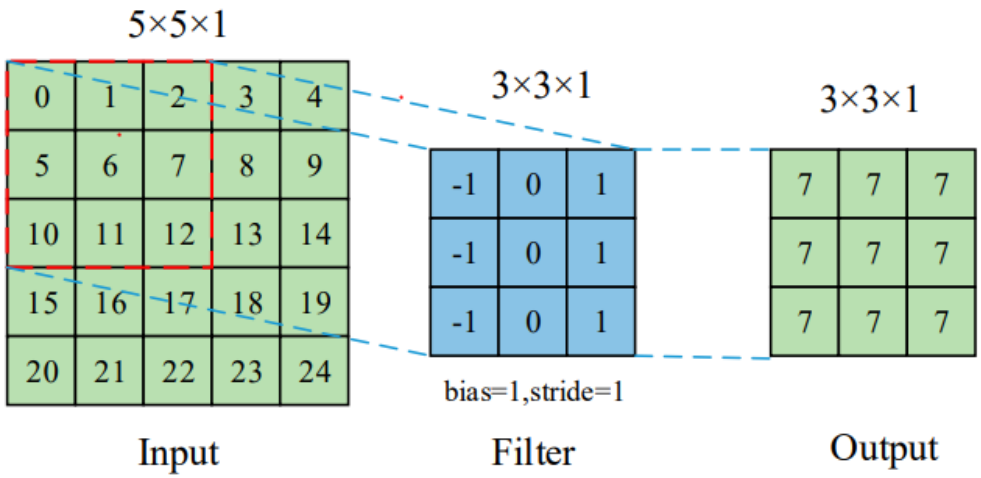


Figure 2. 6 The Convolution Operation Process in the Convolutional Layer

Figure 2.6 shows that the convolution operation in a *Convolutional Neural Network* (*CNN*) is the main process used to extract features from the input image. At this stage, a *filter* or *kernel* of a certain size is slid across the input image according to the *stride* value. Each time the *filter* covers part of the image area, the pixel values in that area are multiplied by the *kernel* weights, then the results are summed and added with a *bias* to produce one output value. This process is carried out repeatedly until the entire image is traversed, so that a *feature map* is formed that represents the response to a certain pattern, such as edges, corners, or textures. In this way, the convolution layer is able to capture simple local patterns that are then learned gradually in the next layer to recognize more complex features in the input image [30]. The following is the calculation equation of the Convolution Layer:

$$\mathbf{Y(i,j)} = \sum_{\{m=0\}}^{\{M-1\}} \sum_{\{n=0\}}^{\{N-1\}} \mathbf{X\,(i+m, j+n) \cdot K(m,n)} \qquad (2.1)$$

Description:

- Y*(i,j)* is the resulting pixel value (output/feature map) at coordinate position *(i,j)*.
- X*(i+m,j+n)* is the pixel value of the input image at coordinate *(i+m,j+n)*.
- K*(m,n)* is the value of the kernel/filter element at coordinate *(m,n)*.
- M,N are the kernel size.
- The symbol ∑ indicates the summation (accumulation) process.

2.4.2 Pooling layer

The pooling layer functions to reduce the spatial dimensions of the *feature map* produced by the *convolutional layer*, while still retaining important features and reducing computational complexity. This *pooling* operation helps the model become more robust to small variations such as translation, rotation, or scale changes of objects in the image. The two most commonly used types of *pooling* are *max pooling*, which selects the highest value from each small window in the *feature map*, and *average pooling*, which computes the average value of that window. By using *pooling* alternately with the *convolutional layer*, the network can be more focused on prominent features, while reducing the number of parameters and computational load. Thus, training and inference become more efficient in the plant leaf disease classification task [30].

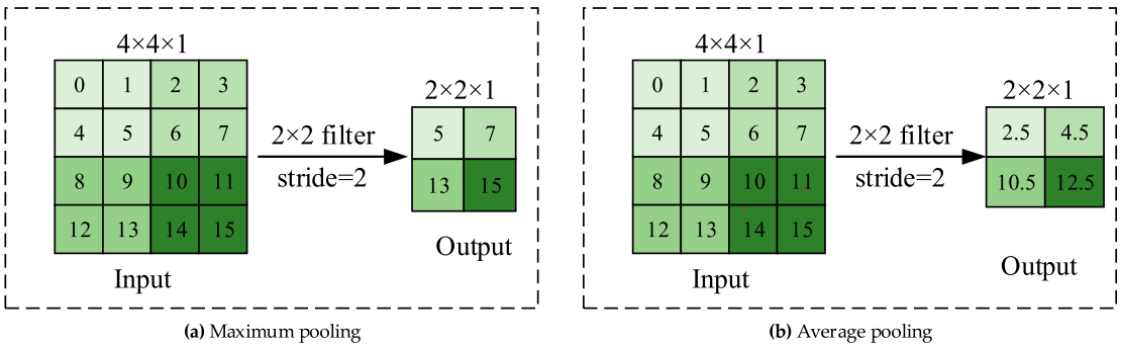


Figure 2. 7 The Pooling Operation Process

Figure 2.7 shows that the *pooling* process works by dividing the *feature map* into small areas, then taking a certain value from each of these areas, for example the maximum value in *max pooling* or the average value in *average pooling*. In this way, *pooling* helps maintain invariance to small translations in the input image, while also speeding up the model training process because the number of parameters is significantly reduced. In addition, *pooling* is also able to suppress

local *noise* that appears in the leaf image, so that the resulting features are more stable for the next classification stage [30].

2.4.3 Activation Function

The *h-swish activation function* was introduced as a more computationally economical variation of the *swish* function, and is used in the *Squeeze-and-Excitation* (SE) module of the *MobileNetV3* architecture. Mathematically, it can be expressed as follows:

$$\mathbf{h-swish(x)} = \mathbf{x} \cdot \frac{\mathbf{6ReLU6(x+3)}}{\mathbf{6}} \qquad (2.2)$$

This function replaces the *sigmoid* component in *swish* which is relatively more computationally expensive, so that *h-swish* is able to approximate the non-linear characteristics of *swish* but with a lower computational load. The use of *h-swish* in the network ensures training stability, accelerates convergence, and improves inference efficiency on devices with limited resources, such as mobile or *wearable* devices [27].

2.4.4 Depthwise and Pointwise Convolution

Depthwise separable convolution consists of two main complementary stages. First, *depthwise convolution*, in which each filter is applied independently to each input channel to capture local spatial information. Second, *pointwise convolution*, in the form of a 1×1 convolution that functions to combine the outputs of all these channels into a combined feature representation. This approach significantly reduces the number of parameters and computational load compared to standard convolution, because it separates the spatial filtering process from the combination of features across channels. The use of *separable convolution* in the *MobileNetV3 backbone* has been proven to improve feature extraction efficiency in object detection and *occlusion* of small objects without significant loss of performance [31].

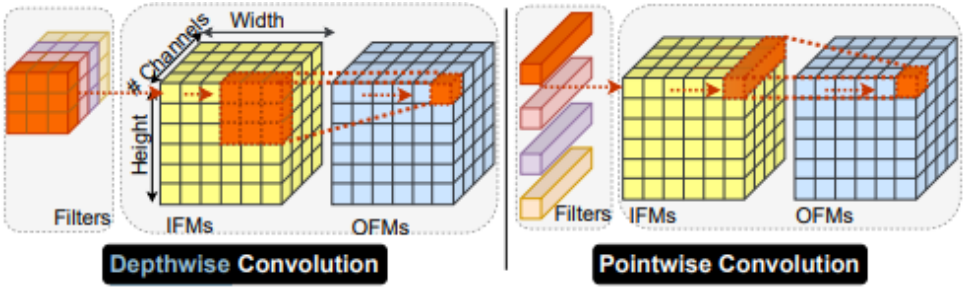


Figure 2. 8 Depthwise and Pointwise Convolution

Figure 2.8 shows that *depthwise convolution* processes each *input channel* separately using a spatial filter, in which each filter only moves on its own channel (*Input Feature Maps/IFMs*)

without combining across channels. This process produces *Output Feature Maps* (OFMs) for each channel. Conversely, *pointwise convolution* uses a 1×1 filter that is applied to all channels of the *depthwise* output, thereby allowing cross-channel combination. This 1×1 filter combines the three dimensions (*height, width, channels*) to produce a more representative final output. The *fusing* process between *depthwise* and *pointwise convolution* has been proven to save inference time on the GPU, because it reduces memory calls and the workload between *kernels*, while still maintaining feature representations relevant for classification [32]. The equations for the output size (*output size*) of the *depthwise convolution* and *pointwise convolution* operations can be written as follows:

output size *Depthwise*

$$\mathbf{Hout} = \frac{\mathbf{Hin} - \mathbf{Kh} + \mathbf{2P}}{\mathbf{S}} + \mathbf{1}$$

$$\mathbf{Wout} = \frac{\mathbf{Win} - \mathbf{Kw} + \mathbf{2P}}{\mathbf{S}} + \mathbf{1}$$

$$\mathbf{Cout} = \mathbf{Cin}$$

output size *Pointwise*

$$\mathbf{Hout} = \mathbf{Hin}, \mathbf{Wout} = \mathbf{Win}, \mathbf{Cout} = \mathbf{Nf}$$

(2.4)

Description:

- Hin,Win = height and width of the input
- Kh,Kw = kernel/filter size (height × width)
- P = padding
- S = stride
- Cin = number of input channels
- Cout = number of output channels
- Nf = number of 1×1 filters used
- Hout = height (number of rows) of the resulting convolution feature map.
- Wout = width (number of columns) of the resulting convolution feature map.

2.4.5 Squeeze-and-Excitation (SE) Block

The Squeeze-and-Excitation (SE) block is a *channel attention* module that allows the network to adaptively recalibrate the weights between feature channels, so that relevant features are strengthened while less important features are suppressed. This module consists of two stages: the *squeeze* stage, in which the feature map is compressed using *global average pooling* to summarize spatial information into one value per channel, and the *excite* stage which uses two *fully connected* layers to compute the channel attention weights. These weights are then multiplied by the original feature map so that channels with important features receive a greater response. In architectures such as *MobileNetV3-Small*, the SE block has been proven to improve feature representation without adding much computational load, and is very useful on plant disease datasets where the differences between channels can be significant, for example variations in the color and texture of spots [33].

2.4.6 Inverted Residual Block

The Inverted Residual Block is a development of the classic residual block with the principle of channel dimension transformation. The process begins from a small channel size, which is then enlarged through a 1×1 convolution, followed by spatial feature extraction using *depthwise convolution*, and finally projected back to a smaller channel dimension through a 1×1 convolution. Thus, the input and output of the block can be summed through a *skip connection*. This structure also uses *linear bottlenecks*, namely eliminating the activation function on the last projection layer so that no information loss occurs between channels. This architecture is used in *MobileNetV2* and *MobileNetV3* because it offers efficiency in the use of parameters and computation, while still being able to maintain performance on image classification tasks [34].

2.4.7 Fully Connected Layer (Dense Layer)

The Fully Connected Layer (FCL), or often called the *dense layer*, is an important component in the CNN architecture that is usually placed at the final stage of the network. In this layer, each neuron is connected to all neurons in the previous layer, allowing full integration of all the features that have been extracted by the convolution and pooling layers. This process is carried out through a linear operation, namely the multiplication of *weights* by the input plus a bias, which is then followed by a non-linear activation function to produce a representation that can be used in classification. With this fully connected property, the FCL functions as a combiner of information from various spatial features into a denser vector representation, which is then mapped to the target

class. This layer plays an important role in building complex relationships between features, so that the model is able to make the final decision accurately based on the patterns learned during training [35].

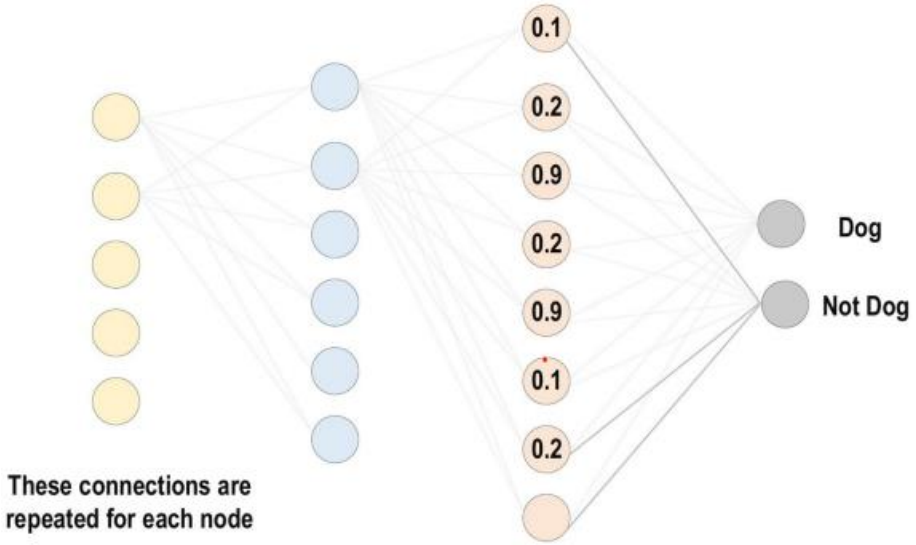


Figure 2. 9 Fully Connected Layer

In Figure 2.9, each neuron in one layer is connected to all neurons in the next layer. The main purpose of this process is to integrate the features that have been extracted from the convolution and pooling layers into a vector representation that can be used for the final classification. Each node in the FCL receives *weights* from all nodes in the previous layer, then a linear operation is performed in the form of the multiplication of weights by the input plus a bias, followed by a non-linear activation function to produce the output. The output value of the neuron in the *hidden layer* is then passed to all neurons in the next layer until the final decision is produced, for example the classification of an image into the "Dog" or "Not Dog" category [35].

2.4.8 Bottleneck

In the context of neural networks, the *bottleneck* refers to a layer or component that limits the flow of information. Usually, this layer is used to reduce the dimensionality of the data in order to improve computational efficiency and prevent *overfitting*. The *Information Bottleneck (IB)* approach is utilized in the design of the bottleneck layer to create a data representation that is compressive but still retains information relevant to the prediction task. The bottleneck layer usually has fewer units than the previous layer, forcing the model to learn the most essential features. This improves generalization ability and reduces computational complexity. The main

challenge in its application is determining the optimal bottleneck size so that important information is not lost [36].

2.5 Transfer Learning

Transfer learning is a technique in *deep learning* that focuses on leveraging the feature representations that have been learned by a model from a large-scale *dataset*, to be applied to solving a new task that has a limited amount of data. Training a *Convolutional Neural Network* (CNN) architecture *from scratch* generally requires massive computational resources and millions of training image data so that the model avoids *overfitting*. Through the *transfer learning* approach, the neural network architecture does not need to start its learning from random weights. The model can use *pre-trained weights* that have previously been optimized using a giant public *dataset* such as ImageNet. Basic knowledge in the form of the ability to recognize universal visual patterns—such as the detection of lines, color gradations, and curved shapes—is then "transferred" to the new model. This allows the model to adapt and classify specific images, such as infection textures on plant leaves, with a far better level of convergence and accuracy efficiently [37]. A fundamental comparison of the computational flow between the traditional training process *from scratch* and this knowledge-transfer-based approach is illustrated in more detail in Figure 2.10.

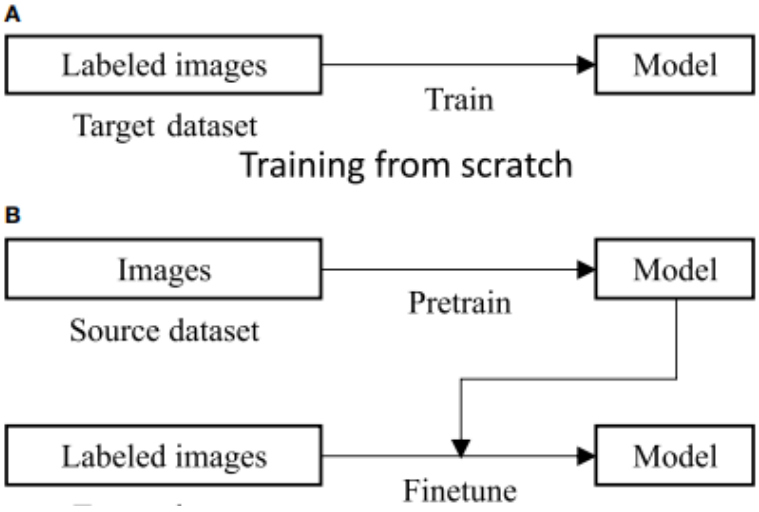


Figure 2. 10 Comparison of Training Schemes

Based on the scheme in Figure 2.10, the operational difference between the two approaches can be identified. In scheme (A), the model is trained independently using the *target dataset* entirely from scratch, which demands the availability of massive data and a longer *training* duration. Conversely, in scheme (B), the *pre-training* phase leverages a large-scale *source dataset*

to lock in general visual representations, which are then transferred into the new model architecture before finally being adjusted (*fine-tuning*) using labeled images from the *target dataset*. This scheme (B) approach is what is fundamentally implemented in this research in order to classify potato leaf diseases with high efficiency [38].

To optimize the application of *transfer learning* so that it matches the target *dataset*, an architecture engineering strategy known as *fine-tuning* is required. This training process is generally divided into two main phases. In the first phase (*feature extraction*), the feature- forming *base layers* of the pre-trained model are frozen (*frozen*), and learning is only focused on adjusting the final classification layer (*classifier head*). After the performance of the top layer is stable, the *fine-tuning* phase is activated by "unfreezing" (*unfreeze*) some of the last convolution layers of the pre-trained model architecture. These unfrozen layers are then retrained simultaneously using a very low learning rate (*learning rate*). This two-stage adjustment mechanism is designed so that the model is able to recognize specific, finely graded features on the target dataset (such as the spot patterns of *Early Blight* and *Late Blight*) without causing the model to lose or damage the memory of the basic visual knowledge that was previously built solidly (the *catastrophic forgetting* phenomenon) [37].

2.6 Optimization Algorithms

In training deep artificial neural network architectures, the optimization algorithm plays a very important role in updating the model's parameters or weights iteratively in order to minimize the value of the loss function (*loss function*). The selection of the right optimization algorithm greatly determines the speed of convergence and the model's ability to reach the global minimum point. In adaptive training scenarios such as two-phase *transfer learning*, the use of a combination of different optimization algorithms in each phase has been proven to provide more optimal stability and accuracy results [39].

In the first phase (*feature extraction*), the *Adaptive Moment Estimation* (Adam) optimization algorithm is highly recommended. Adam is an algorithm that computes the learning rate (*learning rate*) adaptively for each parameter by combining the concepts of first moment estimation (the average of past gradients) and second moment estimation (the average of the squares of past gradients). Mathematically, the parameter update in Adam optimization can be formulated as follows:

$$\boldsymbol{\theta_t + 1 = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t} \qquad (2.5)$$

Where $\boldsymbol{\theta}$ is the model weight parameter, $\boldsymbol{\eta}$ is the learning rate, $\boldsymbol{\hat{m}_t}$ is the bias-corrected estimate of the first moment, $\boldsymbol{\hat{v}_t}$ is the estimate of the second moment, and $\boldsymbol{\epsilon}$ is a small constant to prevent division by zero. Adam's adaptive capability makes it very fast and efficient in bringing the model toward the initial convergence point. This computational characteristic is very ideal for use when most of the feature-forming architecture layers are still frozen (*frozen*), so that the classification of the top layer can adjust quickly to the new *dataset* [40].

Entering the second phase (*fine-tuning*), in which some of the convolution layers near the *output* layer begin to be unfrozen (*unfrozen*) to be retrained, the optimization algorithm is changed to use *Stochastic Gradient Descent* (SGD) combined with *Nesterov Accelerated Gradient* (NAG). Unlike standard SGD, Nesterov momentum has an anticipation or "look-ahead" mechanism (*look-ahead*) toward the position of the parameter before computing the gradient direction. The parameter update formula for SGD Nesterov can be defined as:

$$\boldsymbol{v_t + 1 = \gamma v_t + \eta \nabla_\theta J(\theta_t - \gamma v_t)}$$

$$\boldsymbol{\theta_t = \theta_t - v_t + 1}$$

Where $\mathbf{v}$ represents the update velocity (*velocity*), $\boldsymbol{\gamma}$ is the momentum coefficient, and the expression $\boldsymbol{\nabla_\theta J(\theta_t - \gamma v_t)}$ represents the gradient of the objective function computed at the estimate of the future parameter position. The implementation of SGD with Nesterov momentum produces weight-update movements that are far more stable, precise, and directed. This property is very essential in the final stage of *fine-tuning* which uses a very small learning rate, because it functions as a safeguard to prevent drastic weight changes that risk damaging the well-established basic feature representation [39].

2.7 *Loss Function: Categorical Crossentropy*

In *deep learning* architectures, the loss function (*loss function*) plays a crucial role as an evaluation metric that measures how far the deviation is between the model's prediction result and the actual truth label (*ground truth*). For image classification case studies involving more than two class categories (multi-class classification), *Categorical Crossentropy* (CCE) is the most standard and optimal loss function to use. In its implementation, CCE is always paired with the *Softmax*

activation function in the last classification layer, in which the prediction probability of each object is only allowed to enter one mutually exclusive class category [41].

Mathematically, the calculation of *Categorical Crossentropy* is formulated through the following equation:

$$\mathbf{L} = \sum_{\mathbf{i=1}}^{\mathbf{C}} \mathbf{y_i} \mathbf{log}(\hat{\mathbf{y}}_\mathbf{i}) \qquad (2.7)$$

Where $\boldsymbol{L}$ is the total loss value (*loss*), $\boldsymbol{C}$ represents the total number of classification classes (such as the healthy leaf, *early blight*, and *late blight* classes), $\mathbf{y_i}$ is the actual target value represented in *one-hot encoding* format (valued 1 for the correct target class and 0 for the other classes), and $\hat{\mathbf{y}}_\mathbf{i}$ is the prediction probability value produced by the model for the $\boldsymbol{i}$-th class.

The logarithmic algorithm mechanism in this equation provides a very firm evaluation characteristic. If the model gives a prediction probability value $\hat{\mathbf{y}}_\mathbf{i}$ that is very close to its actual label $\mathbf{y_i}$, then the logarithm value will approach zero, which means the overall *loss* value will become very small. Conversely, if the model confidently gives a high probability to the wrong disease class, this function will immediately impose an exponentially increasing loss penalty. This asymmetric and firm penalty characteristic is what forces the optimization algorithm to update the parameter weights (*weights*) in a more directed manner during the *fine-tuning* phase, so that the model's convergence in recognizing leaf spot patterns becomes far more accurate and stable [42].

2.8 Callback Functions: *Early Stopping* and *ReduceLROnPlateau*

In the training process of *deep learning* architectures, determining the most optimal number of iterations (*epoch*) is often a major challenge. Training that is too short will produce an undertrained model (*underfitting*), whereas training that exceeds the limit will trigger *overfitting*, namely a condition in which the model memorizes the training data excessively so that it loses its ability to recognize new image data. To overcome these problems, the callback function (*callback*) is implemented as a mechanism for automatic monitoring and intervention throughout the course of the training process. One of the main *callback* functions used is *Early Stopping*. This regularization function operates by periodically monitoring the movement of the evaluation metric on the validation data, particularly the *validation loss*, at the end of each iteration. If the metric is detected to no longer show a trend of improvement or reduction within a certain range of iterations set as the patience threshold (*patience*), then *Early Stopping* will stop the training execution automatically. This mechanism of cutting off iterations not only saves computation time

significantly, but also guarantees that the model restores the best parameter configuration (*restore best weights*) before indications of *overfitting* begin to occur [43].

Besides preventing *overfitting*, optimizing the convergence rate toward the global minimum point is also controlled through the implementation of the second *callback*, namely *ReduceLROnPlateau*. This adaptive function is tasked with dynamically reducing the learning rate value (*learning rate*) when the model's training performance begins to get stuck in a stagnant condition (*plateau*). In its mechanism, when the *validation loss* value stops decreasing, *ReduceLROnPlateau* will reduce the current *learning rate* by multiplying it using a certain shrinkage ratio or reduction factor. This reduction in the learning-rate step size allows the optimization algorithm to update the feature weights with far smaller, tighter, and more careful movements. This dynamic adjustment intervention has been proven to be very crucial, especially in the final stage of *fine-tuning* of architectures such as MobileNet, because it helps the model break through the local stagnation limit and move closer to the optimal point without causing aggressive fluctuations of decreasing accuracy [44].

2.9 Performance Evaluation Metrics

After the *deep learning* architecture has finished training, the next fundamental step is to quantify the model's performance using performance evaluation metrics. In the case of multi-class image classification, the evaluation cannot rely on only a single parameter. A comprehensive assessment is needed to measure the extent to which the model is able to generalize its knowledge to the test data (*test set*) that has never been seen before. The basis for calculating these evaluation metrics starts from the formation of the *Confusion Matrix* (Confusion Matrix) [45]. The following is the display of the *Confusion Matrix* table, which consists of the combination of predicted values and actual values, which can be seen in Table 2.1.

Table 2. 1 Confusion Matrix Combination

	Actual Positive	*Actual Negative*
Predicted Positive	*True Positive (TP)*	*False Positive (FP)*
Predicted Negative	*False Negative (FN)*	*True Negative (TN)*

The combination of each predicted and actual value produces elements such as True Positive (TP), True Negative (TN), False Positive (FP), False Negative (FN). A more detailed explanation for each element is as follows:

1. ***True Positive (TP)***

   Indicates the amount of data predicted positive and truly positive according to the actual label. It can be concluded that the model predicts the positive case correctly.

2. ***True Negative (TN)***

   Indicates the amount of data predicted negative and matching the actual label. It can be concluded that the model predicts the negative case correctly.

3. ***False Positive (FP)***

   Is the amount of data predicted as positive, but in reality labeled negative. It can be concluded that the model fails to predict the negative case that is labeled positive. This error is usually called a Type I Error.

4. ***False Negative (FN)***

   Indicates the amount of data predicted negative even though it is actually positive. It can be concluded that the model fails to predict the positive case that is labeled negative. This error is usually called a Type II Error.

The evaluation results on the plant leaf disease recognition system are presented through the *confusion matrix*, which reflects the distribution of correct and incorrect predictions between classes. Based on this matrix, performance metrics such as *accuracy* show the proportion of correct predictions compared to the total cases, whereas *precision*, *recall*, and *F1-score* provide a more detailed assessment per class [46]. Further discussion is as follows:

1. *Accuracy*

   Accuracy is a metric that shows the proportion of correct predictions out of the total number of samples in the dataset. The calculation equation is:

$$\textbf{Accuracy} = \frac{\textbf{TP+TN}}{\textbf{TP+TN+FP+FN}} \qquad (2.8)$$

   This metric provides an overall picture of how well the model performs classification across all classes [47].

2. *Precision*

   Precision measures the proportion of correct positive predictions compared to all predictions classified as positive. The calculation equation is:

$$\textbf{Precision} = \frac{\textbf{TP}}{\textbf{TP+FP}} \qquad (2.9)$$

   In other words, *precision* shows how many positive-class predictions turn out to be truly positive according to the label [47].

3. *Recall*

   Recall, or sensitivity, describes the proportion of positive samples that the model successfully identifies compared to all samples that are actually positive. The calculation equation is:

$$\textbf{Recall} = \frac{\textbf{TP}}{\textbf{TP+FN}} \qquad (2.10)$$

   The higher the *recall* value, the fewer positive cases are missed by the model (*false negative*) [47].

4. *F1-Score*

   F1-score is the harmonic mean of *precision* and *recall*, used to balance the two, especially when there is an imbalance in the number of positive and negative samples. The calculation equation is:

$$\textbf{F1} - \textbf{Score} = \textbf{2 x} \frac{\textbf{Precision x Recall}}{\textbf{Precision+ Recall}} \qquad (2.11)$$

   This metric provides an overall picture of the model's ability to make positive predictions accurately and completely [47].

2.10 Data Augmentation

Data augmentation is a technique used to enrich the number and diversity of training images, with the aim of overcoming the problem of an imbalanced dataset while also preventing *overfitting*. One approach that can be applied is *image-to-image translation*, namely transforming the original image into a new image with a different appearance, but still representing the same object. This process is often combined with the *attention* mechanism to emphasize important parts of the image, and with the *background preservation* technique so that the background part remains consistent. The results of such augmentation are usually in the form of synthetic images with more varied textures, so that the classification model has the opportunity to learn from conditions more representative of reality in the field [48].

The *data augmentation* technique can be applied through simple yet very effective geometric transformation methods to enrich the variation of training images. Based on the physical characteristics of potato leaves in the real environment, the relevant augmentation methods to use include: rotation (*rotation range*), to rotate the image at a certain angle in order to simulate the variation of leaf orientation; horizontal mirroring (*horizontal flip*), to multiply the variation of object layout without changing the morphological nature of the disease spot; and scale variation (*zoom range*), which enlarges or reduces the image so that the model becomes more adaptive to

differences in the distance of the camera lens from the object. The selective application of these basic techniques has been proven able to improve the generalization ability (*generalization*) of the classification model by providing a more diverse distribution of training examples, while also preventing the model from the tendency to memorize certain spatial patterns in the original *dataset*.

2.10.1 Rotation Range

The horizontal flip augmentation technique (*horizontal flip*) is one of the simple yet very effective geometric approaches for increasing the variation of training data in plant disease classification. By mirroring the image against the vertical axis (Y axis), the model obtains a new visual representation of the same object without changing the original morphological structure of the plant or the color characteristics of the disease symptoms on the leaf. This approach is very important because in real conditions, the location of the appearance of infection spots on the leaf blade is random and the angle at which photos are taken by field devices can produce a visual orientation that is reversed from the left or right direction. The application of horizontal mirroring has been proven able to significantly expand the spatial diversity of the *dataset*, while also helping to reduce the risk of *overfitting*, because the network does not experience spatial bias and does not only remember the location of disease features on one particular coordinate side [49].

With the application of *horizontal flip*, the model not only learns from image patterns in the same direction, but also from the variation of its reversed perspective that may appear in real conditions in the field, for example when plant leaves are photographed from an opposing viewing angle in an agricultural land area. Mathematically, the horizontal flip transformation on the pixel coordinates of a digital image can be expressed using a matrix operation as follows:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} W-1 \\ 0 \end{bmatrix} \qquad (2.12)$$

Description:

- (x,y) = original pixel coordinates before transformation
- (x′,y′) = new pixel coordinates after horizontal flip
- W = total width (*width*) of the digital image in pixel units

2.10.2 Horizontal Flip

The *horizontal flip* technique is a form of *data augmentation* that modifies the image through horizontal mirroring. This process provides a variation of object orientation on the

horizontal axis, so that the model does not only learn to recognize disease features at their original location, but also represents real conditions in which the position of disease spots is random in the field. This approach is very useful and safe to use because the mirroring process does not change the original morphology of the leaf disease lesion. By adding this mirroring variation, the model becomes more robust and does not merely memorize the location of spots on the left or right side of the image only. Therefore, *horizontal flip* is able to help the model generalize visual patterns better, so that the accuracy of plant disease classification can increase [49]. The mathematical equation for the vertical transformation is written as follows:

$$\mathbf{x}' = W - x, \quad \mathbf{y}' = \mathbf{y} \qquad (2.13)$$

Description:

- W = maximum width of the image (*width*)
- x, y = initial pixel coordinate point
- x', y' = pixel coordinate point after mirroring

2.10.3 Zoom Range

The *zoom range* technique is an image augmentation method that enlarges or reduces a certain part of the image so as to produce variation in the scale of the object within the image. This approach helps the classification model so that it not only recognizes leaf disease patterns at the standard size, but is also able to adapt when the object appears closer to or farther from the camera. With this scale variation, the model becomes more flexible in extracting important features even though the size of the leaf in the image varies. The application of *zoom range* has been proven effective in improving the model's generalization ability toward images taken from diverse field conditions, so that it can reduce the risk of overfitting and support more stable classification performance [49]. This technique helps the model recognize objects even when they are taken from different distances. The mathematical transformation for zoom is expressed by:

$$\mathbf{x}' = \mathbf{s} \cdot \mathbf{x}, \quad \mathbf{y}' = \mathbf{s} \cdot \mathbf{y} \qquad (2.14)$$

Description:

- s = zoom factor (greater than 1 to enlarge, between 0 and 1 to reduce)

2.11 Data Normalization and Preprocessing

Image preprocessing (*image preprocessing*) is an initial stage that is very crucial in a *deep learning*-based process pipeline (*pipeline*). Raw images obtained directly from the field generally have highly diverse visual characteristics, such as variation in spatial dimensions, differences in lighting intensity, and a high level of background *noise*. If these raw images are directly fed into the artificial neural network without going through a standardization process, the model will experience difficulty in extracting essential features uniformly. This condition can slow the convergence rate and trigger a significant decline in accuracy performance due to numerical instability during the backpropagation phase (*backpropagation*) [50].

The first preprocessing process implemented is image resizing (*image resizing*). The MobileNet architecture is by default designed to accept image input with uniform spatial dimensions, generally sized 224 x 224 pixels with 3 color channels (*Red, Green, Blue*). This dimension alignment is carried out by applying spatial interpolation methods, such as bilinear interpolation (*bilinear interpolation*). This method is able to reduce or expand the resolution of the original image to the target dimension smoothly without damaging the continuity of geometric information or the spatial structure of the infection spot patterns found on the leaf surface [51].

After the image dimensions are made uniform, the next mandatory stage is pixel normalization. By *default*, each pixel in a digital image is represented by an integer value in the range 0 to 255 based on the 8-bit format. This large range of values can cause the gradient values to become unstable when the weight multiplication process (*weight*) in the convolution layer is carried out. Therefore, a linear transformation is performed to map the pixel intensity values into a smaller numerical scale, such as the range [0, 1] or [-1, 1] [1]. The mathematical equation for the normalization operation to the range [0, 1] is formulated as follows:

$$X_{norm} = \frac{\mathbf{X} - \mathbf{X_{min}}}{\mathbf{X_{max}} - \mathbf{X_{min}}} = \frac{\mathbf{X}}{\mathbf{255}} \qquad (2.15)$$

where X represents the original pixel intensity value, $\mathbf{X_{min}}$ is the minimum pixel value (0), $\mathbf{X_{max}}$ is the maximum pixel value (255), and $X_{norm} = \frac{\mathbf{X}}{\mathbf{255}}$ is the result of converting the pixel into a finely scaled *float* data type.

In the MobileNet architecture that leverages ImageNet *pre-trained weights*, the normalization process is often continued with a *channel-wise normalization* scheme. This process

is carried out by subtracting the mean value (*mean*) and dividing it by the standard deviation (*standard deviation*) of each of the ImageNet RGB color channels. This reduction in the scale of the data distribution ensures that the feature distribution of the target *dataset* has a similar distribution to the original training data of the base architecture. Through this combination of precise preprocessing and normalization, the model's computational load can be reduced drastically, the risk of data outliers (*outliers*) can be minimized, and the model architecture is able to extract disease texture features more stably in order to reach the optimal accuracy limit [51].

This page is intentionally left blank

CHAPTER III
METHODOLOGY

This research aims to optimize the performance of the MobileNet-based *deep learning* architecture in classifying disease images on potato leaves consisting of three target classes: *early blight*, *late blight*, and *healthy*. The research process begins with a literature study to obtain a deep understanding of the basic theory of the *Convolutional Neural Network* (CNN), the MobileNet architecture, and relevant previous research. The next step is the collection of the *dataset*, which is then continued to the preprocessing and data splitting stage, covering spatial dimension adjustment, geometric augmentation, and normalization of the pixel value distribution so that the images are ready to be computed. After that, the model architecture is designed and implemented using the *transfer learning* approach by leveraging the *pretrained weights* from the ImageNet *dataset*. The next crucial stage is the application of a two-phase model training scenario that dynamically leverages a combination of optimization algorithms in order to obtain the most maximal model performance. To ensure a truly optimal system configuration, this research designs a series of trial scenarios carried out in stages (*sequential ablation study*), including: (1) a trial of the *dataset* split proportion, (2) a trial of the optimization algorithm configuration, and (3) a trial of the model architecture selection. Each stage isolates one variable being tested while keeping the other variables constant, so that the effect of each factor can be analyzed objectively and independently. The final performance evaluation of the model is carried out using the *accuracy*, *precision*, *recall*, and *F1-score* metrics to validate the reliability level of the system objectively and measurably. In general, the mapping of the methodological structure of this research can be seen in the block diagram of Figure 3.1.

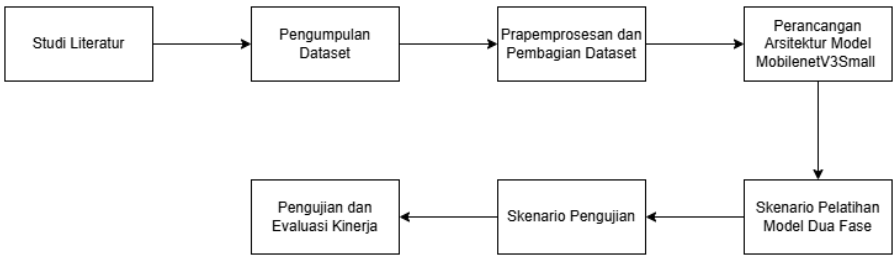


Figure 3. 1 Research Flow

Figure 3.1 maps the research methodology structure designed systematically to solve the potato leaf disease classification problem. This sequence of stages begins with a literature study to build a relevant theoretical foundation, continued with the dataset collection stage that contains visual representations of both healthy and infected leaf conditions. This set of raw data then enters the preprocessing and dataset splitting stage in order to standardize the dimensions, minimize class imbalance, and divide the data into training, validation, and testing proportions. The core computation stage is represented through the design of the MobileNetV3-Small model architecture, which is technically executed through a continuous two-phase model training scenario to reach optimal convergence. To ensure the reliability of the network design, the architecture space is evaluated through a structured testing scenario that includes the search for the best *hyperparameters*. This entire sequence of flows culminates in the testing and performance evaluation stage, in which the generalization capability and the accuracy level of the model are validated objectively using comprehensive analytical metrics.

3.1 Literature Study

The literature study is the initial stage that aims to understand the basic concept of image classification and to review previous research related to plant disease detection, particularly on the potato commodity. Potato leaf diseases such as *early blight* and *late blight* become the main focus because of their significant impact on productivity decline. In various literature, the approach based on the *Convolutional Neural Network* (CNN) has been proven able to provide very good results in recognizing visual disease patterns on leaf images. One of the CNN architectures that becomes the main focus in this research is MobileNetV3-Small. This architecture is designed with lightweight and efficient computation, so that it is very ideal for implementation on devices with resource limitations without sacrificing feature extraction capability. The literature review is carried out through scientific journals, articles, and relevant research reports to deepen the application of the *transfer learning* method, the standardization of image preprocessing, and the engineering strategy of training optimization parameters. This literature review functions as a strong theoretical foundation for designing the best computational parameters in order to reach the maximum level of accuracy and efficiency in the detection system being built.

3.2 Dataset Collection

The dataset used in this research was obtained from a combination of the Kaggle dataset named *Potato Disease Leaf Dataset (PLD)*, published by Rizwan Saeed in 2021, and *Potato Leaf*,

published by Pooja Dave in 2025. The dataset contains digital images of potato leaves grouped into three categories, namely *Healthy*, *Early blight*, and *Late blight*. The image dataset used reaches a total of 7,573 images, with variations in lighting conditions and leaf texture, so that it becomes representative training data for building a *deep learning*-based classification model. Each image is available in the standard digital *JPG* format with uniform resolution, which facilitates the preprocessing and model training process. This dataset was chosen because of its high credibility and because it has been widely used in previous research for the development of computer-vision-based plant disease detection systems.



Figure 3. 2 Healthy Potato Leaf, *Late blight* and *Early blight* Diseases

Figure 3.2 displays example samples of the potato leaf image dataset used in this research, consisting of three categories from left to right: *Healthy*, *Late blight*, and *Early blight*. Each category has different visual characteristics; for example, leaves with *Early blight* disease generally show blackish-brown spots with a concentric ring pattern, whereas leaves with *Late blight* disease are marked by greenish to grayish spots that then change into blackish brown. Healthy leaves show no spots or damage. This variation of categories is very important to support the model training process, because it allows the *deep learning* algorithm to recognize the distinctive visual patterns of each class. Thus, the classification model developed is expected to be able to identify the type of disease on potato leaves accurately based on the visual features obtained from the dataset.

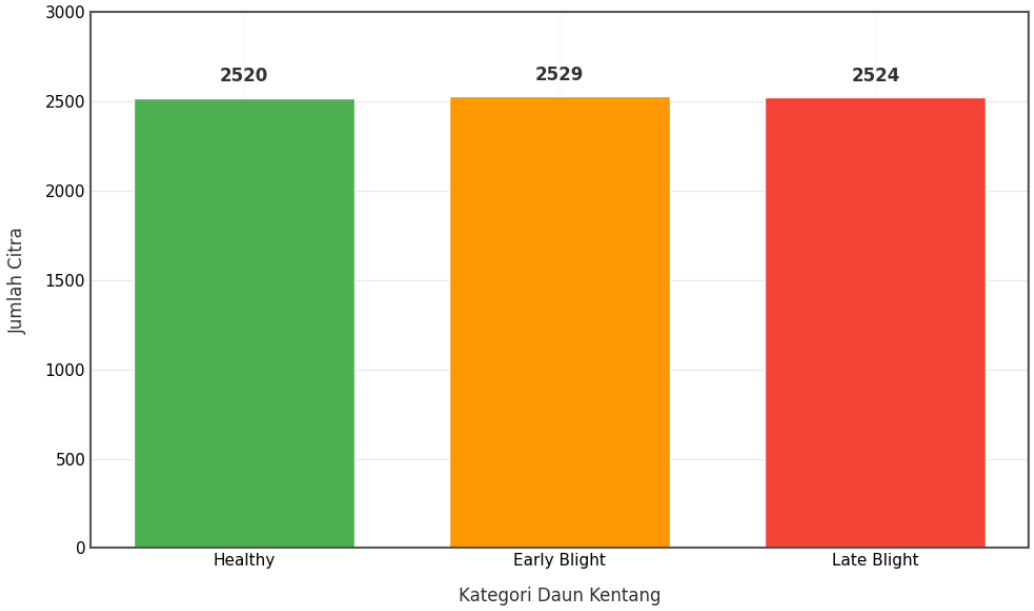


Figure 3. 3 Visualization of the Number of Images per Class

To ensure the proportion of data before entering the training stage, the distribution of the number of images in each category is visualized in the form of a bar chart in Figure 3.3. Specifically, this dataset consists of 2,520 images of the *Healthy* class, 2,524 images of the *Late Blight* class, and 2,529 images of the *Early Blight* class. This balanced composition of the number of samples between classes (*balanced dataset*) is an ideal empirical condition for the computation stage of the artificial neural network. This equality of proportion ensures that the MobileNetV3-Small architecture, supported by the *transfer learning* method, can extract and learn the visual characteristics of each leaf category objectively.

3.3 Preprocessing and Data Splitting

This section explains the series of processes carried out in the dataset *preprocessing* stage. This stage has an important role because it aims to prepare the data so that it is in optimal condition before being used in model training. In Figure 3.4, which displays the *preprocessing* flow, there are three main steps, namely: allocating the data into three subsets, augmenting the data to enrich image variation, and normalizing the data so that the pixel values are more uniform. Each step has a specific function in improving the quality of the dataset so that the model can learn image patterns more optimally and produce better classification performance. The description of each stage is presented in Figure 3.4 as follows.



Figure 3. 4 Data Preprocessing Flowchart

Figure 3.4 shows the dataset preprocessing flow that is executed sequentially. The process begins with Dataset Splitting (*Data Splitting*) into training, validation, and testing sets to prevent data leakage (*data leakage*). Then it continues with dynamic Data Augmentation to add visual variation simulating the real environment in order to prevent *overfitting*. Finally, Image Dimension Adjustment and Normalization are carried out by changing the resolution to 224 × 224 pixels and scaling the pixel values to the range [-1.0, 1.0] so that they are compatible with the MobileNetV3-Small architecture.

3.3.1 Dataset Splitting (*Data Splitting*)

Data splitting (*data splitting*) is a crucial stage in *deep learning* computational engineering that aims to guarantee the objectivity of the model learning process and to avoid the *overfitting* phenomenon. To evaluate the effect of the composition of the amount of data on the generalization ability of the MobileNetV3-Small model, this research designs a trial scenario using 3 variations of the dataset split proportion from the total population of 7,573 images, namely:

1. Proportion Scenario I (80:10:10): Allocates 80% for training data, 10% for validation data, and 10% for test data.
2. Proportion Scenario II (70:15:15): Allocates 70% for training data, 15% for validation data, and 15% for test data.
3. Proportion Scenario III (60:20:20): Allocates 60% for training data, 20% for validation data, and 20% for test data.

Methodologically, each data subset has a different operational role in the MobileNetV3-Small model training pipeline (*pipeline*):

- Training Set (*Training Set*): Used fully by the artificial neural network architecture to extract leaf visual features (edges, textures, colors, and spot patterns) and to update the weight values (*weights*) and internal biases through the *backpropagation* mechanism.
- Validation Set (*Validation Set*): Serves as an independent evaluation reference during the training process. This validation data functions to monitor the development of the loss value (*validation loss*) and accuracy, and becomes the basis for tactical decisions for

callback functions such as *Early Stopping* and *ReduceLROnPlateau* in stopping or optimizing the learning rate (*learning rate*).

- Testing Set (*Testing Set*): Is a collection of pure images that are completely locked and never seen by the model during the training or optimization phase. This data is used exclusively at the end of the research to measure the real generalization ability of the model in recognizing new objects through the accuracy, precision, *recall*, and *F1-score* metrics.

Table 3. 1 Dataset Distribution Allocation

Data Type	**Data Proportion 80 : 10 : 10**	**Data Proportion 70 : 15 : 15**	**Data Proportion 60 : 20 : 20**
Training	6,059	5,301	4,543
Validation	757	1,136	1,515
Testing	757	1,136	1,515

Based on Table 3.1, the entire dataset of 7,573 images is divided into three proportion schemes, namely 80:10:10, 70:15:15, and 60:20:20. In the 80:10:10 proportion, 6,059 images are allocated as *training* data, whereas the *validation* and *testing* data each amount to 757 images. In the 70:15:15 proportion, the allocation of *training* data is adjusted to 5,301 images, with *validation* and *testing* data each amounting to 1,136 images. Meanwhile, in the 60:20:20 proportion scheme, the *training* data consists of 4,543 images, accompanied by an increase in the portion of *validation* and *testing* data, each of which reaches 1,515 images. This variation of dataset splitting scenarios is designed systematically to analyze and identify the effect of the composition of the *training*, *validation*, and *testing* data volume on the level of convergence and the generalization ability of the model in the process of classifying potato leaf disease images.

3.3.2 Data Augmentation

After the dataset is divided into three main *subsets*, the augmentation process is applied exclusively to the training data (*training set*) in order to enrich image variation without needing to add physical data from the field. Data augmentation is a crucial stage in computer vision for expanding the representation of visual patterns, so that the model is able to recognize objects from various orientations, positions, lighting conditions, and scales. Through this technique, the generalization ability (*generalization*) of the model will improve and the risk of excessive data memorization (*overfitting*) can be significantly suppressed.

The implementation of augmentation in this research leverages the TensorFlow/Keras library through the ImageDataGenerator function that produces new images dynamically (*on-the-fly*) at each training iteration (*epoch*). This approach is very efficient in terms of computational memory allocation because the image matrix transformation is performed randomly without having to store physical copies of the new images in local storage space. Based on considerations of the morphological characteristics of the potato leaf object and the challenges of image acquisition in open fields, the augmentation techniques applied now include a combination of spatial transformation and pixel-level manipulation. Spatial transformations include rotation (*rotation range*), scale-size changes (*zoom range*), and horizontal mirroring (*horizontal flip*). Meanwhile, pixel manipulation is added through the application of *Gaussian blur* to simulate out-of-focus image distortion, *Gaussian noise* to represent optical sensor disturbances on camera devices, and brightness-level variation (*brightness variation*) to imitate the fluctuation of natural lighting intensity dynamically.



Figure 3. 5 Data Augmentation Process Flow

Figure 3.5 shows the data augmentation process flow as a whole. This process begins with preparing the original dataset in the form of raw potato leaf images before transformation is carried out. Next, augmentation is applied to add variation to the images so that a new set of data with a higher level of diversity is formed. The result of this augmentation is then used in the model training stage, with the aim of enriching the data representation and improving the model's ability to recognize patterns better.

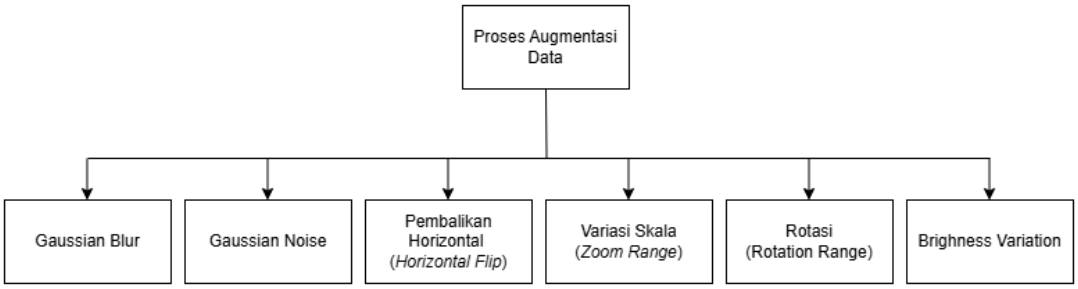


Figure 3. 6 The Augmentation Techniques Used

Figure 3.6 shows that the dynamic (online) data augmentation process cycle is carried out through the selective application of six image transformation techniques, which are divided into geometric manipulation and simulation of real environmental conditions. Geometric transformations include horizontal mirroring (horizontal flip), scale variation (zoom range), and rotation (rotation range). Meanwhile, to simulate the challenges of image acquisition in real plantation fields, photometric manipulation is added in the form of blurring (Gaussian blur), the addition of noise spots (Gaussian noise), and lighting-intensity variation (brightness variation). This entire combination of transformations is executed automatically at each epoch to enrich the dataset representation without needing to add new physical data manually. Through the design of techniques based on the natural characteristics and environmental variation of potato leaf photo capture, the MobileNetV3-Small model obtains the opportunity to learn visual patterns more broadly without damaging the essential morphology of the disease spot shape. The result of this augmentation pipeline (pipeline) is a collection of image variations with high spatial and intensity diversity, which not only helps the model extract complex patterns objectively, but also directly contributes to improving the accuracy and robustness (robustness) of the model when tested with pure data that has never been encountered before.

3.3.2.1. Horizontal Flip

This is a geometric augmentation technique used to mirror the image horizontally from left to right or vice versa. This flip operation is determined through a boolean value, in which the activation of the *horizontal flip* parameter function is set to the value *True*. Computationally, this process works by reversing the order of the column indices on the color pixel matrix of the potato leaf image without changing the arrangement of its rows. For example, a pixel that was initially at the left-edge coordinate will be moved symmetrically to the right-edge coordinate. This technique is very effective for randomly multiplying the variation of the coordinate location of disease spots (*early blight* and *late blight*) on potato leaves, so that the model does not experience spatial bias or memorize infection locations that lean toward one particular side in the original image. The following is an example of the implementation of the *horizontal flip* technique in Figure 3.7.

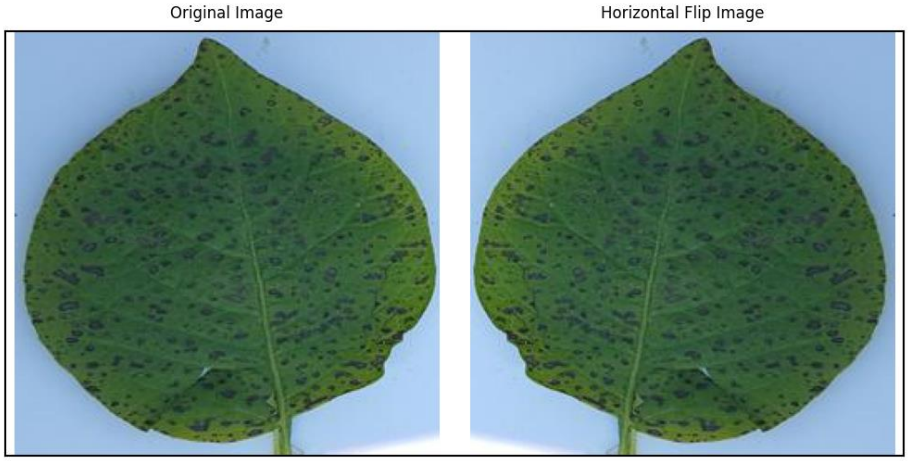


Figure 3. 7 Horizontal Flip Result

Figure 3.7 shows a comparison between the original potato leaf image (left) and the image resulting from horizontal flip augmentation (*horizontal flip*) (right). This spatial transformation technique mathematically creates a mirror effect of the original image without distorting the biological information or the morphology of the disease spots present on the leaf surface. The application of this technique aims to enrich the variation of spatial orientation in the training data set, so that the model does not memorize the specific location or position of an infection, but truly focuses on recognizing the pathological feature.

3.3.2.2. Zoom Range

This is an augmentation technique used to enlarge (*zoom in*) or reduce (*zoom out*) the image within a certain range. The *zoom range* parameter value is expressed as a proportion of the original image size. For example, when *zoom range* has a value of 0.2, the augmentation process can enlarge or reduce the image by up to ±20% of the original size. The following is an example of the implementation of the *zoom range* technique in Figure 3.8.



Figure 3. 8 Zoom Range Result

Figure 3.8 shows a comparison between the original leaf image and the image resulting from augmentation using the *zoom range* technique. This transformation performs random enlargement (*zoom-in*) or reduction (*zoom-out*) on a certain pixel area within the predetermined range scale. This technique functions to simulate the variation of the image-capture distance between the camera sensor and the leaf object in the field. Through this scale adjustment, the model is forced to recognize the morphological characteristics of the disease regardless of the visual size of the object (*scale-invariant*), thereby preventing the network from merely memorizing the size proportion of the original image.

3.3.2.3. Rotation Range

This is an augmentation technique used to rotate the image within a certain angle range randomly. The maximum rotation value is determined through the *rotation range* parameter and is expressed in degrees. For example, if *rotation range* has a value of 20, then the image can undergo random rotation in the range of −20° to +20°. The following is an example of the implementation of the *rotation range* technique in Figure 3.9.

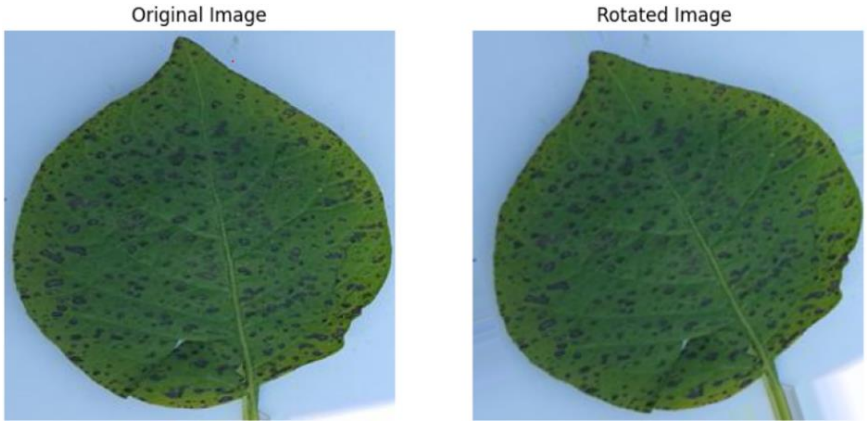


Figure 3. 9 Rotation Range Result

Figure 3.9 shows a comparison between the original leaf image (left) and the image resulting from augmentation through the *rotation range* technique (right). This transformation rotates the image position randomly on its central axis within a certain degree range, both clockwise and counterclockwise. The application of this rotation technique effectively simulates the variation of the tilt angle of image capture that is very likely to occur naturally during data acquisition in the field. Through this transformation, the model is trained to have robustness to changes in object orientation (*rotation-invariant*), so that the recognition of pathological feature patterns does not depend on the upright or tilted position of the leaf specimen within the image.

3.3.2.4. Gaussian Blur

This is a spatial-filter-based augmentation technique used to blur the image (*blurring*). Computationally, this process is carried out by convolving the image pixel matrix using a Gaussian kernel (for example sized 3x3 or 5x5 pixels). This technique effectively simulates real conditions in agricultural fields, such as out-of-focus photos (*out of focus*) due to a non-ideal shooting distance, the user's hand shaking, or gusts of wind when capturing images of potato leaves. By applying the blur effect randomly, the model is trained not only to rely on the sharpness of edges or the contrast of disease spots, but to be able to recognize color distribution patterns and shapes more generally and robustly. The following is an example of the implementation of the *Gaussian blur* technique in Figure 3.10.

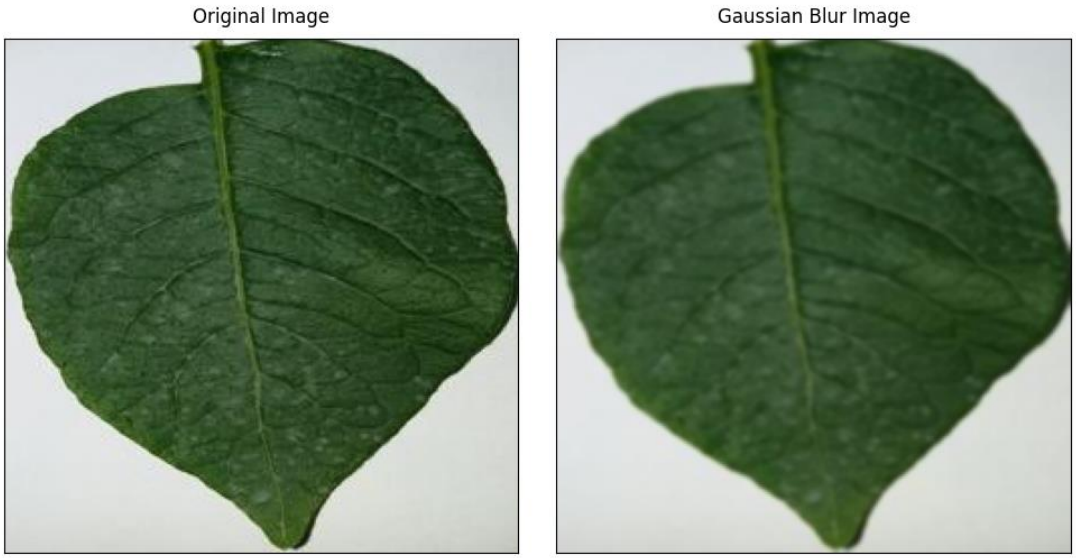


Figure 3. 10 Gaussian Blur Result

Figure 3.10 shows a comparison between the original leaf image (left) and the image resulting from augmentation using the *Gaussian blur* effect (right). This technique works by applying a spatial smoothing filter that reduces the intensity of high-frequency detail, thereby producing a blur effect on the image. The main purpose of this transformation is to simulate data acquisition conditions in the real environment, such as imperfect camera lens focus or the presence of slight shaking during image capture. By exposing the model to images that are not always sharp, the network is forced not to memorize the noise (*noise*) or microscopic detail on the pixels, but to focus on recognizing the macroscopic pattern and general shape of the disease spot.

3.3.3.5. Gaussian Noise

This is an augmentation technique used to add noise or visual artifact spots randomly into the image. This process works by generating a matrix of random values based on a normal (Gaussian) distribution that has a mean value (*mean*) of zero and a certain standard deviation, then adding it to the RGB pixel intensity matrix of the original image. The addition of this noise aims specifically to simulate the image-capture quality from the camera sensor of a low-specification mobile device or shooting conditions in a low-light environment (*low-light* conditions). The presence of *noise* forces the model to extract the essential features of the potato leaf disease without being easily deceived by pixel disturbances in the real world. The following is an example of the implementation of the *Gaussian noise* technique in Figure 3.11.

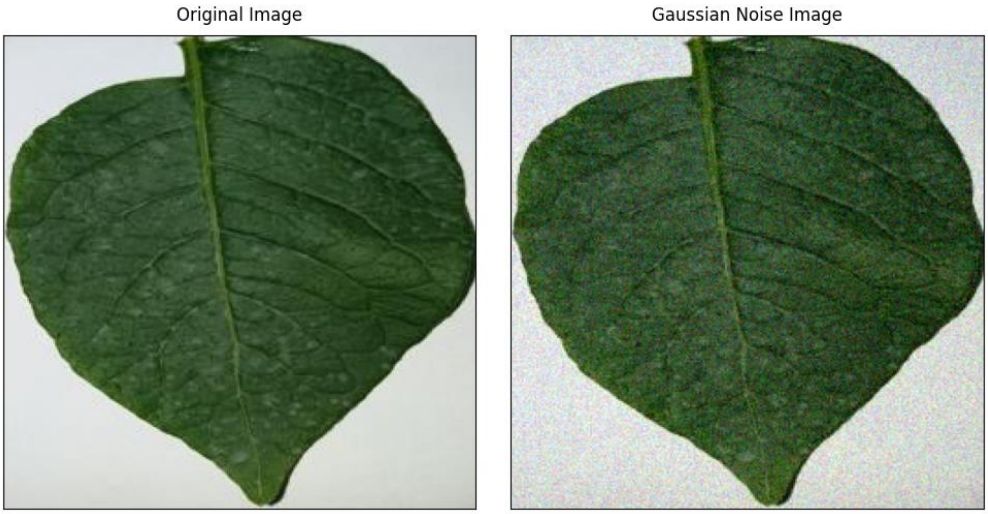


Figure 3. 11 Gaussian Noise Result

Figure 3.11 shows a comparison of the original leaf image (left) with the result of *Gaussian noise* augmentation (right). This technique injects random pixel noise to simulate data acquisition disturbances in the field, such as low camera sensor quality or image capture in low-light conditions (high ISO). The purpose is to train the robustness (*robustness*) of the model so that it remains focused on extracting the main disease features without being distracted by visual anomalies, which at the same time contributes to preventing *overfitting*.

3.3.3.6. Brightness Variation

This is an augmentation technique that manipulates the overall brightness level of the image. Mathematically, this operation is carried out by multiplying the intensity value of the image pixel matrix by a scalar multiplier (*brightness factor*). A multiplier factor with a range above 1.0

(for example 1.3) will brighten the image, whereas a range below 1.0 (for example 0.7) will dim it. This brightness variation is very crucial for representing the fluctuation of sunlight intensity in the open, such as leaf shadows, cloudy weather conditions, and very harsh midday sun exposure. Through this modification, the model architecture is prevented from the risk of lighting bias, so that disease identification does not depend on the absolute color intensity of only one environmental condition. The following is an example of the implementation of the *brightness variation* technique in Figure 3.12.

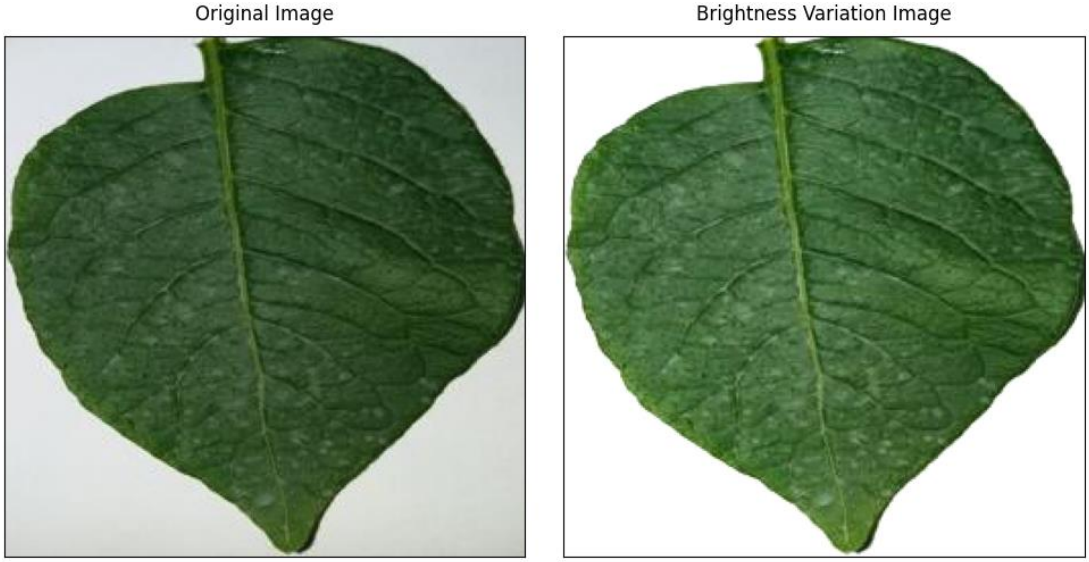


Figure 3. 12 Brightness Variation Result

Figure 3.12 shows a comparison of the original leaf image (left) with the result of *brightness variation* augmentation (right). This technique modifies the light intensity in the image randomly to simulate the fluctuation of natural lighting in agricultural fields, such as the influence of weather (cloudy or sunny) and differences in the time of image capture. The application of this transformation aims to train the model to be immune to changes in illumination (*illumination-invariant*), so that the recognition of disease pathological features remains accurate and does not depend on a certain lighting condition.

3.3.3 Image Dimension Adjustment and Normalization

Before the potato leaf image is fed into the *Convolutional Neural Network* (CNN) model, a series of preprocessing stages must be carried out to adjust the data format to the input standard (*input shape*) of the architecture used. The first stage is spatial dimension adjustment (*resizing*). All images in the *dataset*, which initially had varying resolutions, are made uniform in size to a

matrix of dimension 224 X 224 pixels with 3 color channels (RGB). This size is the standard resolution required by the base MobileNetV3-Small architecture in order to optimize the feature extraction process computationally and ensure the network structure can process the data matrix consistently.

The second stage is normalization of the image pixel intensity. This process is carried out by converting the original RGB pixel value range (which ranges from 0 to 255) into a smaller distribution scale centered on zero. This normalization is executed automatically using the *preprocess_input* function built into the TensorFlow/Keras module for the MobileNetV3 architecture, which maps the pixel matrix values into the range [-1.0, 1.0]. Mathematically, the normalization function mapping process can be formulated as follows:

$$X_{norm} = (\frac{\mathbf{X}}{\mathbf{127.5}}) - \mathbf{1.0} \qquad (3.1)$$

Description:

- X_{norm}: The image pixel intensity value after normalization (in the range [-1.0, 1.0]).
- X: The original pixel intensity value in the input image (in the range [0, 255]).
- 127.5: The constant divisor value obtained from half the maximum pixel value (255 / 2).

The transformation of the pixel value range through this normalization technique is very essential in the preprocessing phase. By reducing the scale of the input values so that they are distributed symmetrically around zero, the optimization algorithm (such as Adam) can work more efficiently. This is effective for preventing the *exploding gradient* or *vanishing gradient* phenomenon during the *backpropagation* process, thereby accelerating the model convergence time to reach the global minimum point (*global minimum*) during training.

3.4 Design of the *MobileNetV3Small* Model Architecture

In this research, a modern convolutional neural network (*Convolutional Neural Network*, CNN) architecture is used, namely MobileNetV3-Small, which is highly relevant for solving the potato leaf disease image classification task. This architecture is designed specifically to be implemented in environments of devices with computational resource limitations (*resource-constrained devices*). To achieve this efficiency, MobileNetV3-Small integrates several state-of-the-art technologies, among them the *depthwise separable convolution* operation, the *Squeeze-and-Excitation* (SE) block for feature attention focusing, and the use of the *hard-swish* activation function. The combination of this architecture structure has been proven able to reduce the

parameter load and computational complexity significantly while maintaining a competitive level of accuracy.

The selection of this single architecture is based on the research objective to design a classification system that not only focuses on achieving high accuracy metrics, but also on optimizing resource efficiency during both the training phase and the inference phase (*inference*). By focusing the performance evaluation on MobileNetV3-Small, this research is expected to be able to provide an in-depth analysis of the capability of a lightweight model (*lightweight model*) in processing agricultural images. The final result of this architecture evaluation is projected to be able to become a strong technical foundation for the application of a real-time (*real-time*) and portable plant disease detection system in the agricultural field environment.

This section presents the *MobileNetV3-Small* architecture that is leveraged as the *feature extractor* in this research. The model was developed with the aim of maintaining a balance between computational efficiency and accuracy level, so that it is very suitable for use on devices with limited computing power or memory. Figure 3.13 shows the entire design of the *MobileNetV3-Small* architecture, which is composed of a series of convolution blocks and activation layers. This structure is designed in such a way as to optimize the model's performance in image processing while still maintaining efficiency.

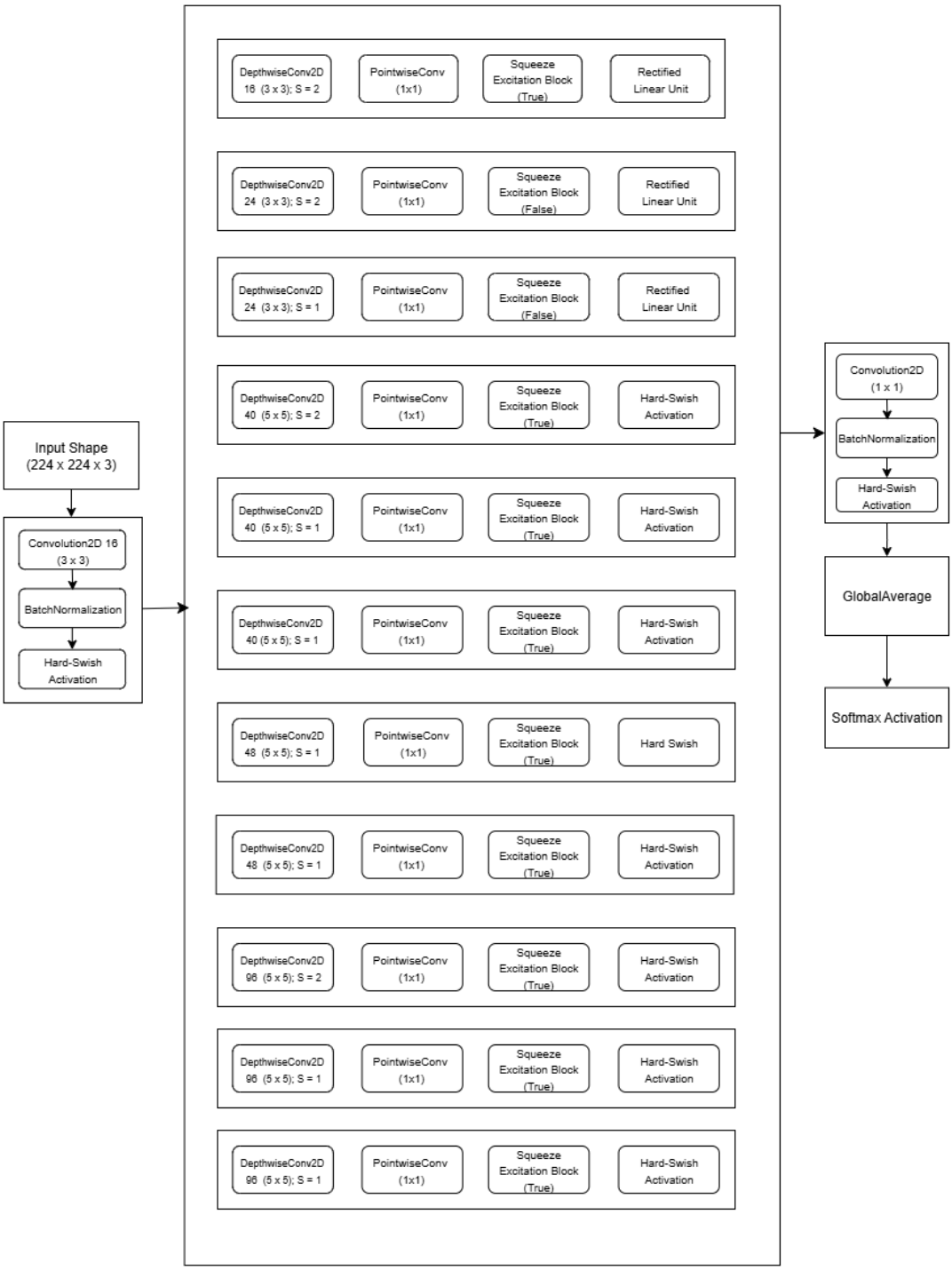


Figure 3. 13 Detail of the MobileNetV3Small Architecture

Figure 3.13 illustrates the *MobileNetV3-Small* architecture designed as a series of efficient computation blocks. Starting from an initial convolution layer, the image is then processed through a series of *bottleneck* blocks that form the backbone of the network. These blocks leverage the *inverted residual* technique and *depthwise* convolution to reduce the computational load

significantly. Some blocks are also equipped with the *Squeeze-and-Excitation* (*SE*) module that intelligently highlights the most informative feature *channels*, while the *hard-swish* activation function is used to maintain speed. Throughout the process, the spatial resolution of the image is gradually reduced, while its feature depth is enriched, so that in the final layer dense features are produced that are ready to be classified, making it ideal for devices with limited resources.

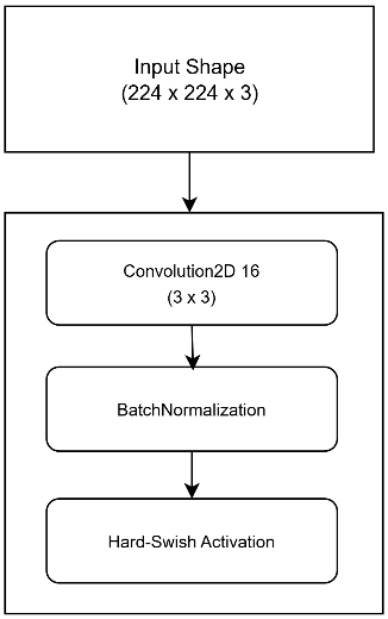


Figure 3. 14 Initial Layer of MobileNetV3Small

Figure 3.14 shows the initial process stage in the *MobileNetV3-Small* architecture, which functions as the entry gate for basic feature extraction. This block receives a color image input (224×224×3) and first passes it through a 3×3 convolution layer that produces 16 feature *channels*. Its output is then normalized by *BatchNormalization* to stabilize the learning process, before finally being processed by the efficient *Hard-Swish* activation function to introduce *non-linearity*. Overall, this initial block is tasked with preparing an optimal basic feature representation before being processed further by the more complex *bottleneck* layers in the network.

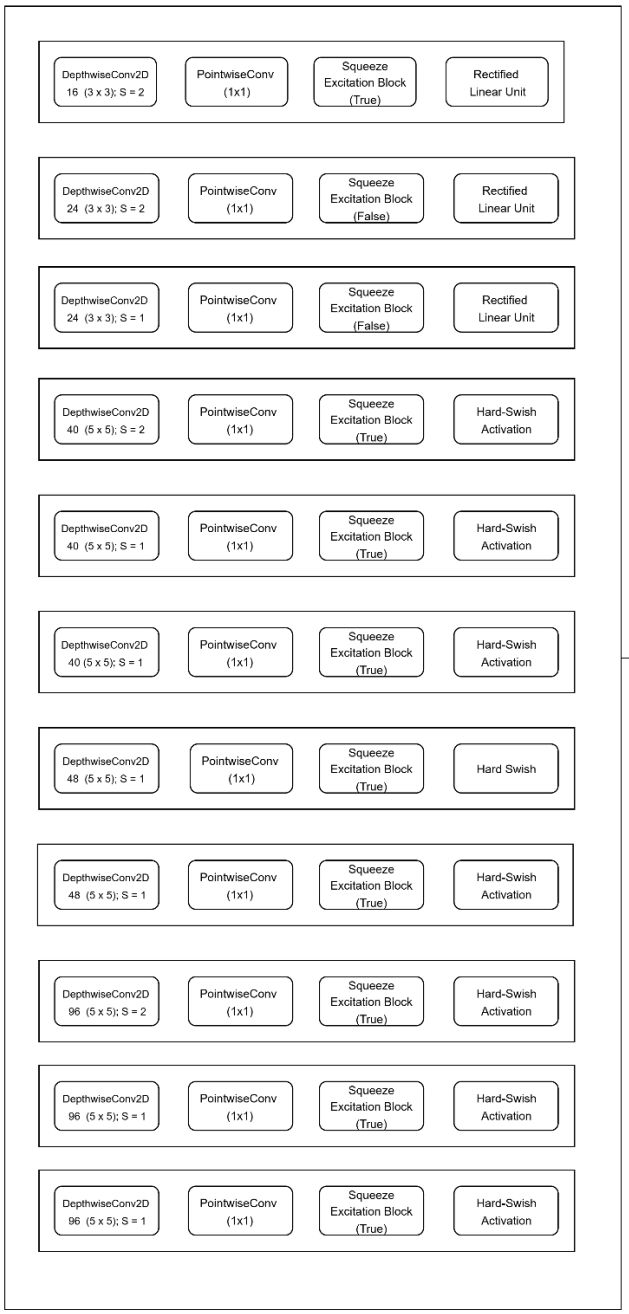


Figure 3. 15 The Bottleneck Block Process

Figure 3.15 illustrates a typical *bottleneck* block in the *MobileNetV3-Small* architecture, whose process begins with a 1×1 convolution to increase the *channel* dimension (*expansion*), followed by a 3×3 *depthwise* convolution that efficiently processes spatial features, and ends with a 1×1 convolution to lower the *channel* dimension again (*compression*). This block often integrates the *Squeeze-and-Excitation* (*SE*) module after the *depthwise* convolution to strengthen the important feature *channels*, and uses *Hard-Swish* or *ReLU6* activation on certain layers. This

lightweight and efficient *bottleneck* pattern is what is repeated and stacked with variations in the number of *channels* and *stride*, forming the entire main body of the *MobileNetV3-Small* network.

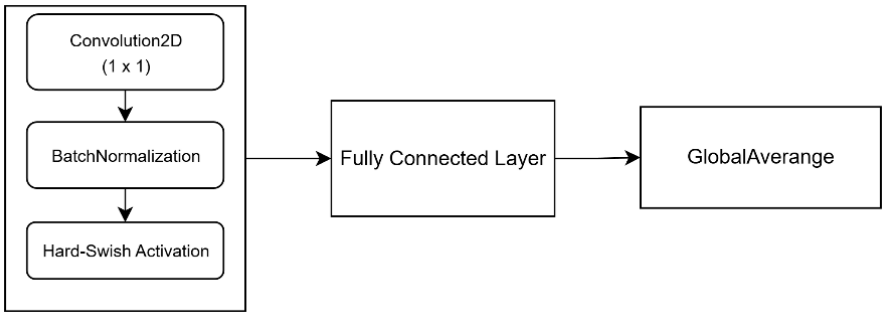


Figure 3. 16 Final Architecture of MobileNetV3Small

Figure 3.16 illustrates the final part or *head* of the *MobileNetV3-Small* architecture, which is tasked with preparing the features for classification. After passing through the entire network, the resulting features are first compressed by a 1×1 convolution that functions to consolidate the information. Its output is then normalized and activated with the *Hard-Swish* function before undergoing the *Global Average Pooling* process that reduces each feature map into an average value. This resulting one-dimensional feature vector is finally fed into the *Fully Connected Layer* that functions as the *classifier* to produce the final probability of each predicted class.

3.5Two-Phase Model Training Scenario

The training of the image classification model in this research is implemented using the *Transfer Learning* approach with a structured two-phase scenario. This scenario is designed specifically to optimize the performance of the base MobileNetV3-Small architecture in recognizing the visual characteristics of potato leaf diseases. Through the *Transfer Learning* technique, the model is not trained from scratch using random weight initialization, but leverages the parameter weights (*weights*) that were previously trained thoroughly using a massive-scale image *dataset*, namely ImageNet. The use of this built-in weight representation provides a significant computational advantage, given that the base convolution layers of the architecture already have the ability to extract fundamental visual features such as lines, object edges, and color gradations. Thus, the computation process can be focused exclusively on adapting the model so that it is able to distinguish the very specific disease spot patterns in the *Healthy*, *Early Blight*, and *Late Blight* classes, while also accelerating the overall convergence time.

3.5.1. First Phase: Feature Extraction (*Feature Extraction* / *Freeze*)

In the first phase known as feature extraction (*feature extraction*), all convolution layers of the base architecture (*base model*) MobileNetV3-Small are locked or in a frozen status (*frozen*). This freezing mechanism instructs the computation algorithm not to update the weight values (*weights*) and biases in that base layer during the backpropagation process (*backpropagation*). The main purpose of this weight holding is to preserve the spatial representation and the ability to detect general visual features (such as curve shapes, edge lines, and color composition) that the model previously learned from the massive ImageNet *dataset*.

During this phase, the learning computation process is focused exclusively only on the final classification layer (*classification head*) that has just been added at the top of the architecture. This top layer consisting of *Dense layers* is trained from scratch using random initialization to map the high-level features produced by the base model into three target- class probability labels (*Healthy, Early Blight*, and *Late Blight*). By locking millions of parameters in the base layer, this feature extraction phase runs with a very light computational load and in a relatively short time. This phase acts as a strategic initialization step that gives the new classifier layer the opportunity to adapt to the characteristics of potato leaf images, thereby preventing fatal damage to the *pre-trained weights* due to gradient updates that are too large.

To provide a comprehensive conceptual visualization of this weight-holding mechanism, the status of the model architecture in the feature extraction phase is illustrated in Figure 3.17. In this block diagram, the bottom box represents the *base model* MobileNetV3-Small with a *non-trainable* (locked) status. This status indicates that the millions of computation parameters within that block are isolated from the gradient update process. Conversely, the top box represents the new *classification head* layer with a *trainable* (open) status, which actively updates its matrix weights at each *epoch* to map the input features into the probabilities of the three target classes, namely *Healthy*, *Early Blight*, and *Late Blight*.

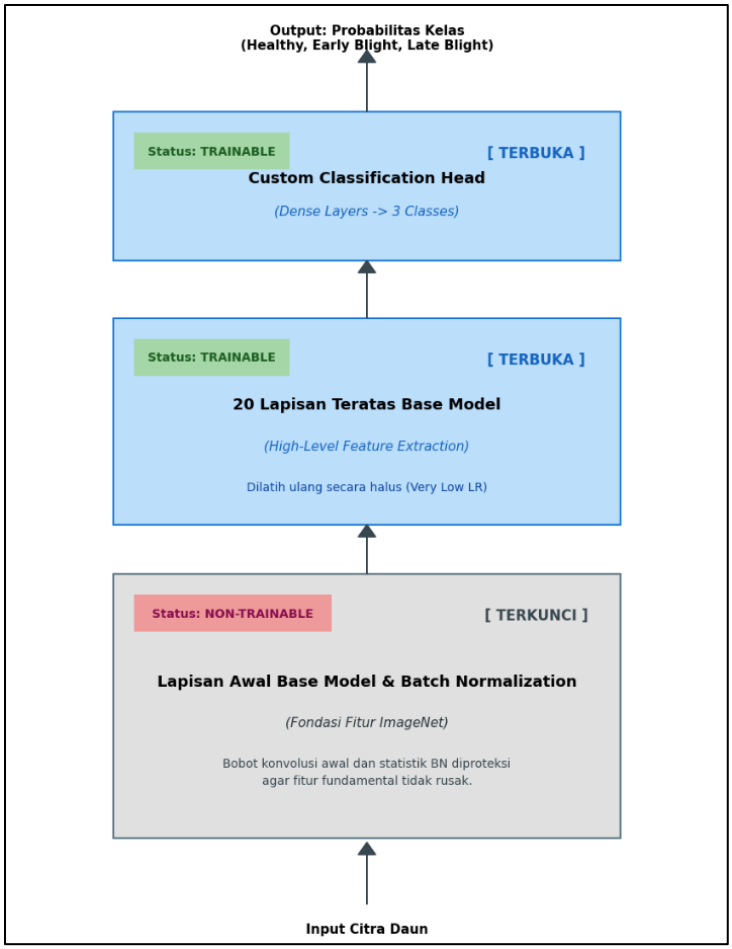


Figure 3. 17 Block Diagram of the Layer Freezing Status

Besides the visual representation, the empirical details regarding the hyperparameter configuration that guides the training process, along with evidence of the network parameter distribution in this first phase, are presented quantitatively in Table 3.2. The table shows the allocation of the use of the Adam optimization algorithm with an initial learning rate (*learning rate*) that is set specifically to trigger convergence on the new classification layer. Physical evidence of the running of the feature extraction scenario can also be observed in the ratio of the number of parameters, in which the majority of the total built-in model parameters are locked as *non-trainable parameters*, leaving a small portion of parameters in the top layer as *trainable parameters*.

Table 3. 2 Hyperparameter Configuration and Parameter Status of the First Phase

Component / Metric	First Phase Configuration
Base Model	MobileNetV3-Small
Base Model Status	*Frozen* (Frozen)

Component / Metric	First Phase Configuration
Initial *Learning Rate*	0.001 (1e-3)
Optimizer	Adam
Maximum Number of *Epochs*	30 Epochs
Total Parameters	940,851
Trainable Parameters	1,731
Non-trainable Parameters	939,120

Table 3.2 presents the hyperparameter configuration and computational parameter status for the first phase training stage (*feature extraction*). In this phase, all built-in weights of the base MobileNetV3-Small architecture are locked (*frozen*), so that of the total 940,851 parameters, only 1,731 parameters in the new classifier layer have a *trainable* status. The remaining 939,120 parameters are computationally frozen to maintain the proven ability to extract basic visual features. The training in this initial phase is executed for a maximum of 30 *epochs* using the Adam *optimizer* with an initial learning rate of 0.001. This configuration aims to focus the computation efficiently in order to map the weights of the final classification layer to the potato leaf disease dataset, without damaging the *pre-trained* feature representation of the network.

3.5.2. Second Phase: Fine-Tuning (*Fine-Tuning/ Unfreeze*)

After the new classification layer at the top of the architecture has successfully adapted and reached the initial convergence level in the first phase, the training scenario continues to the fine-tuning stage (*fine-tuning*). In this second phase, the freezing status on the convolution layers in the *base model* MobileNetV3-Small is unfrozen (*unfrozen*), so that the entire (or most of the) weight parameter matrix in that layer again becomes *trainable* and can be updated through gradient backpropagation. This strategic step is very crucial given that images of agricultural leaf specimens often have far more complex visual variation compared to a generic dataset.

The main purpose of this *fine-tuning* phase is to shift and adapt the previously general visual feature representation (built into ImageNet) to become very specific and sensitive to the detailed characteristics of the morphology of potato leaf diseases. Through this phase, the model is taught to recognize high-resolution patterns, such as the difference in the concentric ring texture in *early blight* and the irregular lesion pattern in *late blight*.

However, the weight update process in this phase must be strictly controlled in order to avoid the *catastrophic forgetting* phenomenon, namely the damage or loss of the basic feature information that was previously well formed due to gradient updates that are too aggressive.

Therefore, the adjustment of computational metrics, particularly the optimization function and the learning rate (*learning rate*), becomes the most crucial parameter. Unlike the first phase, the optimization algorithm in this second phase switches to using *Stochastic Gradient Descent* (SGD) integrated with Nesterov momentum, accompanied by a drastic reduction of the *learning rate* value (to 0.0001). This very low learning rate (*very low learning rate*) combined with this momentum ensures that the model only performs micro-scale weight adjustments, polishing the decision boundary (*decision boundary*) slowly, smoothly, and in a directed manner without damaging the well-established feature extraction foundation.

The architectural changes related to the network parameter status in this *fine-tuning* phase are illustrated in Figure 3.18. Unlike the previous feature extraction phase, this diagram shows the opening of part of the *base model* block (particularly the top 20 convolution layers), which now has a *trainable* status. Nevertheless, the initial layers forming the foundation of the architecture and all *Batch Normalization* layers are kept in a locked condition (*non-trainable*) in order to maintain the stability of the mean statistics (*running mean*) and variance (*running variance*) built into the ImageNet *dataset*.

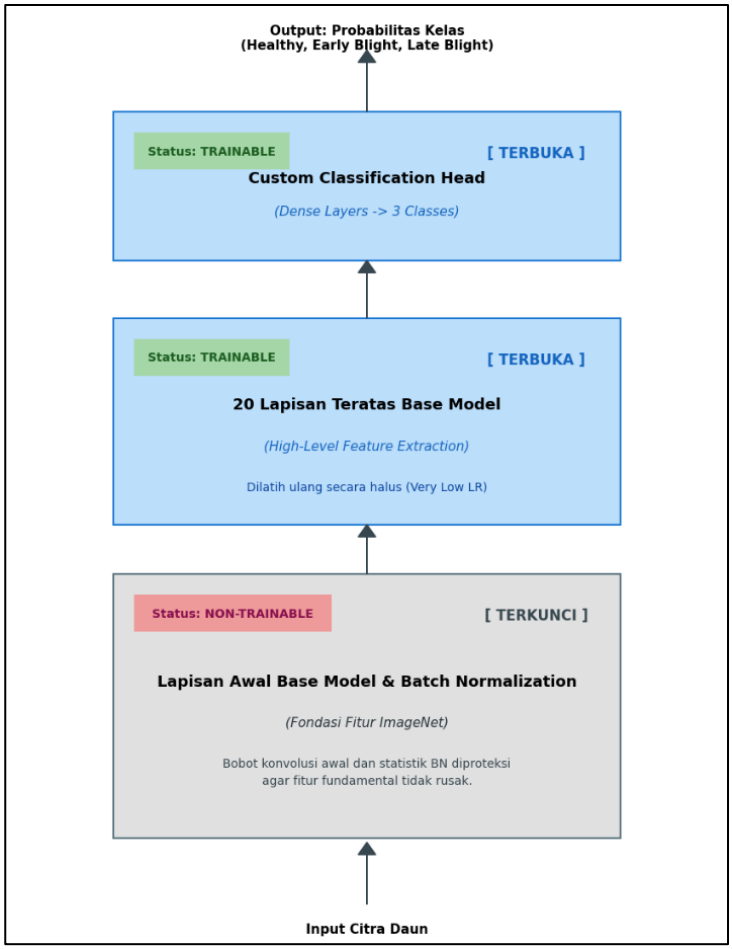


Figure 3. 18 Block Diagram of the Status of Partially Unfrozen Layers

The adjustment of metrics and hyperparameters that distinguishes this second phase from the first phase is summarized in Table 3.3. The table confirms the transition of the optimization function from Adam to *Stochastic Gradient Descent* (SGD) with the integration of *Nesterov Momentum*. The reduction of the learning rate (*learning rate*) to a very small value combined with a momentum of 0.9 is designed to provide precise, smooth, and directed weight updates. This conservative computational strategy is very crucial for preventing the loss of previously learned feature representations (*catastrophic forgetting*). In addition, the ratio of trained parameters (*trainable parameters*) increases significantly with the opening of the weight matrix in the last 20 convolution layers of the base architecture. This exposure of the upper-level layers provides room for adaptation for the network to learn the specific details of the potato leaf disease morphology, so that the separation of boundaries between classes becomes firmer and more accurate.

Table 3. 3 Hyperparameter Configuration and Parameter Status of the Second Phase

Component / Metric	Second Phase Configuration
Base Model	MobileNetV3-Small
Base Model Status	*Unfrozen*
Initial *Learning Rate*	0.0001 (1e-4)
Optimizer	SGD (with Nesterov Momentum = 0.9)
Maximum Number of *Epochs*	30 Epochs
Total Parameters	940,851
Trainable Parameters	348,627
Non-trainable Parameters	592,224

Table 3.3 summarizes the computational configuration for the second phase training stage (*fine-tuning*). In this phase, the status of the MobileNetV3-Small architecture is changed to *unfrozen* on some of its final layers, thereby exposing 348,627 *trainable* parameters to be adjusted specifically to the characteristics of the potato leaf pathology features. This weight update process is executed for a maximum of 30 *epochs* using the SGD *optimizer* with Nesterov momentum (0.9) with a very low learning rate (*learning rate*), namely 0.0001. This conservative computational configuration is designed specifically to refine the classification decision boundary without damaging the visual information representation that the network has learned in the previous phase (*catastrophic forgetting*).

3.5.3. Loss Function and Optimization Algorithm

In evaluating the level of the model's prediction error during the training stage, this research implements the *Categorical Crossentropy* loss function (as conceptually described in Equation 2.7). The selection of this loss function is based on the characteristics of the problem being solved, namely multi-class classification (*multi-class classification*) with three mutually exclusive target categories (*mutually exclusive*), covering the *Healthy*, *Early Blight*, and *Late Blight* classes. This loss function works integratively with the *softmax* activation layer on the *custom classification head* to compute the error penalty based on the difference between the probability distribution of the model's prediction result and the actual label (*ground truth*) represented in *one-hot encoding* format. The error value metric (*loss*) produced at the end of each *epoch* becomes the main reference for the optimization algorithm in updating the neural network weight matrix through the backpropagation process (*backpropagation*).

To minimize this loss function value effectively without sacrificing the stability of the architecture's performance, the weight matrix update process is executed using two types of optimization algorithms applied sequentially based on the running training phase:

- First Phase Optimization Algorithm (Feature Extraction): The training process in this initial phase is controlled by the Adam optimization algorithm (*Adaptive Moment Estimation*) by setting an initial learning rate (*learning rate*) of 0.001. The selection of Adam is based on its adaptive mathematical characteristics, in which this algorithm automatically computes and adjusts the individual learning rate for each parameter based on the estimation of the first and second gradient moments. Adam's aggressive yet measured computational nature is very ideal for handling the new classification layer (*custom classification head*) whose weight matrix is still initialized randomly, thereby allowing the model to reach the initial convergence point (*convergence*) more quickly and stably.
- Second Phase Optimization Algorithm (Fine-Tuning): After the new classification layer reaches initial stability, the optimization function is transitioned to using the *Stochastic Gradient Descent* (SGD) algorithm integrated with the Nesterov Momentum parameter of 0.9. The learning rate in this *fine-tuning* phase is reduced drastically to 0.0001. The use of SGD with a very small learning rate aims to provide a micro-scale weight update mechanism that is far more precise and controlled on the top 20 layers of the *base model* that have been unlocked. In addition, the *look-ahead gradient update* function in Nesterov Momentum acts as a mechanical safeguard that anticipates the direction of gradient movement before the actual weight update occurs. This control logic is very crucial in the second phase in order to prevent an excessive gradient update surge (*overshooting*), so that the fundamental feature representation built into ImageNet that is well-established in the base layer does not experience damage (*catastrophic forgetting*).

3.5.4. Training Control Mechanism (*Callbacks*)

To ensure the training computation process runs efficiently and avoids the *overfitting* phenomenon, this research applies an automatic control mechanism using the *callbacks* function. There are three main types of supervision implemented dynamically, adjusting to the two-phase training scenario that is running, namely:

- *Early Stopping*: Functions as an early stopping mechanism that continuously monitors the movement of the validation loss value (*validation loss*). In the feature extraction phase

(First Phase), the stagnation tolerance limit (*patience*) is set to 10 *epochs*, whereas in the fine-tuning phase (Second Phase) the supervision is tightened to 8 *epochs*. If there is no reduction of the *loss* metric beyond that tolerance limit, the training iteration process will be stopped automatically. This function is also configured to restore the model weight matrix with the best performance (*restore best weights*) before the indication of *overfitting* worsens, so that the network does not merely save the condition at the last *epoch* when it is stopped.

- *ReduceLROnPlateau*: Functions to dynamically reduce the learning rate value (*learning rate*) to half of the previous value (*factor* = 0.5) with a lower bound of 1e-6, if the model's validation performance stagnates for 3 consecutive *epochs*. This adaptive strategy is exclusively activated only in the First Phase to accompany the Adam optimization algorithm. In the Second Phase, this mechanism is deliberately deactivated because the *Stochastic Gradient Descent* (SGD) algorithm has been integrated with Nesterov Momentum which independently controls the smoothness and stability of the gradient update.
- *Model Checkpoint*: Functions as a precision saving mechanism that monitors the achievement of the highest validation accuracy metric (*validation accuracy*). This system ensures that the final result exported and saved into the architecture file format (extension .h5) is the representation of the model weights with the most optimal generalization ability during the entire course of iterations.

3.6 Testing and Hyperparameter Scenario

In this research, the testing scenario is designed using a sequential ablation approach (*sequential ablation study*) to evaluate the performance of the potato leaf disease classification model systematically and objectively. Each testing stage isolates one variable being tested while keeping the other variables constant, so that the effect of each factor on the model's performance can be analyzed independently. The testing focus covers four main stages, namely the dataset split proportion, the optimization algorithm configuration, the *MobileNet* architecture selection, and the search for the optimal *hyperparameter*. The best result from each stage is inherited as the fixed configuration in the next stage, so that the final design decision of the model is based on empirical evidence that has been validated.

The testing scenario in this research consists of four experimental stages carried out sequentially. The first stage tests three variations of the dataset split proportion, namely 80:10:10, 70:15:15, and 60:20:20, each of which represents the ratio of training data, validation data, and test data to the total of 4,072 images in the processed *Potato Disease Leaf Dataset* (PLD). The second stage tests three optimization algorithm configurations for two-phase training, namely *Adam–SGD Nesterov*, *Adam–Adam*, and *SGD Nesterov–SGD Nesterov*, using the best dataset proportion inherited from Stage 1. The third stage tests three *transfer learning* model architectures from the MobileNet family, namely *MobileNetV3Small*, *MobileNetV3Large*, and *MobileNetV2*, using the best combination of proportion and optimizer from the previous stage. To obtain a statistically representative performance estimate and minimize the bias of random weight initialization, the scenarios in Stage 1 through Stage 3 are each run three times (*run*), in which the comparison between scenarios is assessed based on the mean (*mean*) and standard deviation.

After the best architecture is determined in Stage 3, the research continues to the fourth stage, namely optimization of the *hyperparameter* specifically on the MobileNet model using the *Bayesian Optimization* method. In this stage, the model will be evaluated through 20 dynamic experimental combinations (*trials*) to search for the best configuration of four crucial parameters, namely the *Learning Rate* (Phase 1), *Dropout Rate*, *Batch Size*, and the number of layers unfrozen (*n_layers_to_unfreeze*) for the *fine-tuning* process in Phase 2. The purpose of this optimization is to find the mathematical combination that is able to produce the most maximal validation accuracy metric (*validation accuracy*). The complete details regarding the scenario and the testing search space are presented in Table 3.4 and Table 3.5.

Table 3. 4 Testing Scenario

Stage	Testing Scenario	Value
1	Data Proportion	80 10 10
		70 15 15
		60 20 20
2	Optimizer	Adam-SGD Nesterov
		Adam-Adam
		SGD Nesterov- SGD Nesterov
3	Model	MobileNetV3Small
		MobileNetV3Large
		MobileNetV2

Stage	Testing Scenario	Value
4	Hyperparameter Optimization	*Learning Rate* Phase 1
		Dropout Rate
		Batch Size
		Number of Unfreeze Layers (Phase 2)

Table 3.4 describes the four stages of the testing scenario designed to determine the most optimal and robust performing parameter configuration. These stages include the evaluation of the dataset split proportion, the comparison of *optimizer* combinations in the two-phase training scenario, the comparison of efficiency between base architecture variants (*base model*), and end with the *hyperparameter* optimization stage.

Table 3. 5 Hyperparameter Search Space (Search Space)

Parameter Being Optimized	Search Range (*Values*)	Default Value (*Baseline*)
Learning Rate Phase 1	5e-4, 1e-3, 2e-3, 5e-3	1e-3
Dropout Rate	0.1, 0.2, 0.3, 0.4, 0.5	0.2
Batch Size	8, 16, 32	16
Number of Unfreeze Layers	10, 15, 20, 30, 40	20
Trial Limit	Maximum 20 Trials	-

Table 3.5 details the search space limits (*search space*) in the *hyperparameter* optimization stage, which include the adjustment of the first phase *learning rate*, *dropout rate*, *batch size*, and the number of *unfreeze layers* in the second phase. Each parameter is explored iteratively from the default value point (*baseline*) with a maximum limit of 20 trials. This systematic limitation is applied to make the computation duration more efficient while ensuring the discovery of the best network configuration combination.

3.7 Testing and Performance Evaluation

The final stage in this computational design is the comprehensive testing and performance evaluation of the model. To prove the robustness and generalization ability of the MobileNetV3-Small architecture objectively, the testing is designed into two main phases, namely internal testing and external testing (*cross-dataset testing*).

The internal testing phase is carried out in isolation using the built-in testing subset (*testing set*) that is fed through the test_gen data generator. This data set contains representations of potato leaf images that have never been accessed by the neural network during the feature extraction or *fine-tuning* phase. This blind test scenario (*blind test*) is applied to measure the basic accuracy and

ensure that the model does not merely memorize the training data pattern (*overfitting*). Next, the external testing phase is carried out by feeding a secondary dataset downloaded from a different Kaggle profile. This cross-dataset testing is very crucial for evaluating the *Domain Shift* phenomenon, in which the model's robustness is tested when faced with images that have completely new lighting, texture, and field-background characteristics.

Programmatically, the testing process in both phases is executed through two built-in functions of the Keras library. First, the model.evaluate() function is used to calculate the accumulation of the loss value (*test loss*) and the global accuracy (*test accuracy*). Second, the model.predict() function is applied to produce the highest guess probability value (*argmax*) on each test image, which is then compared with its actual label (*ground truth*). The computation results from this inference are mapped into two main analytical instruments, namely the *Classification Report* and the *Confusion Matrix*.

3.7.1. *Classification Report*

As the first analytical evaluation instrument, this research leverages the *Classification Report* that is computed based on the comparison between the model's highest prediction value and the actual label on the test data. In evaluating the reliability of a *deep learning* architecture, the overall accuracy metric (*overall accuracy*) is often not representative enough to conclude the model's performance, especially if there is a potential imbalance in the number of image samples between disease classes (*imbalanced dataset*). Therefore, this classification report is used to dissect the model's performance more deeply on each target class (*Healthy*, *Early Blight*, and *Late Blight*) through three main evaluation metrics (as their formulas have been described in Equations 2.9 through 2.11), namely:

- ***Precision* (Precision):** This metric is used to measure the level of accuracy of the model's positive predictions. In the case of this research, *precision* shows the percentage of correctness when the model predicts an image as a certain disease (for example *Early Blight*); how likely it is that the image is indeed truly infected with *Early Blight* and not merely an ordinary spot from another class.
- ***Recall* (Sensitivity):** This metric is used to measure the model's ability to detect all existing positive samples. In the context of potato leaf pathology, a high *recall* in the *Late Blight* class indicates that of the total actual *Late Blight* images present in the test field, the model

successfully recognizes and finds most of them without many being missed (minimizing *False Negatives*).

- ***F1-Score*:** Is the harmonic mean value (*harmonic mean*) that balances the *precision* and *recall* metrics. This metric is used as the final and most crucial indicator in this research to assess model stability. The MobileNetV3-Small architecture being tested can be said to have very good generalization performance and to not be biased toward only one majority class if the *F1-Score* value produced across the three target classes is in a high and balanced range.

Therefore, the computation results of this classification report will be documented and analyzed based on two main reporting parameters, namely:

- **Class-wise Performance (*Class-wise Performance*):** The calculation of the *Precision*, *Recall*, and *F1-Score* values will be reported separately and specifically for each leaf category, namely the *Healthy*, *Early Blight*, and *Late Blight* classes. This approach aims to evaluate the model's ability to recognize the unique features of each condition without being biased by another class.
- **Macro Average (*Macro Average*):** Is the calculation of the average value of the three metrics above that is computed evenly without giving more weight to the class with the most samples (*unweighted mean*). This macro average value will be used as the final benchmark for concluding the level of balance and the overall generalization ability of the model.

3.7.2. *Confusion Matrix*

As a visual evaluation instrument that complements the classification report, this research uses the Confusion Matrix (*Confusion Matrix*). This instrument functions to directly map the cross-tabulation between the actual class label (*ground truth*) of the test data set and the predicted class label produced by the MobileNetV3-Small model. Given that this potato leaf disease classification involves three spatial categories, the implemented matrix will be of dimension 3 X 3. Structurally, the horizontal axis (X) is represented as the predicted class (*Predicted Label*), whereas the vertical axis (Y) is represented as the actual class (*Actual Label*).

Unlike the statistical metrics in the classification report, the main purpose of using the *Confusion Matrix* in this testing chapter is to trace and visualize the location of classification errors (*misclassification*) precisely. The output of this matrix is represented computationally in the form of a heat map (*heatmap*) with a color gradation scale. Through this visualization, the evaluation analysis is not only focused on the accumulated number of correct guesses (which are positioned

linearly on the main diagonal line of the matrix), but also to identify the model's bias tendency. For example, this matrix is designed to reveal whether the model often experiences "confusion" in distinguishing the visual features between the *Early Blight* and *Late Blight* classes due to the similarity of the morphology or texture of the infection spots on the leaf blade. Thus, the specific weakness of the model architecture in recognizing certain disease features can be evaluated transparently.

Table 3. 6 Confusion Matrix Framework

	Prediction: *Healthy*	Prediction: *Early Blight*	Prediction: *Late Blight*
Actual: *Healthy*	Correct (*True Positive*)	Misclassification	Misclassification
Actual: *Early Blight*	Misclassification	Correct (*True Positive*)	Misclassification
Actual: *Late Blight*	Misclassification	Misclassification	Correct (*True Positive*)

Table 3.6 represents the conceptual framework of the 3 X 3 Confusion Matrix (*Confusion Matrix*) implemented to evaluate the performance of the potato leaf disease classification model. In that matrix structure, the cells located intersecting on the main diagonal line (from the top-left corner to the bottom-right) document the number of the model's actually correct prediction computations (*True Positives*). Conversely, all cells located outside that diagonal line represent the location and number of detection errors (*misclassifications*). Through this cross-mapping framework, the performance evaluation will not only focus on the accumulation of images that were successfully guessed correctly, but is designed specifically to analyze the location of prediction anomalies. For example, this matrix can reveal the model's bias tendency if there is intense cross-classification error between the *Early Blight* and *Late Blight* classes due to the similarity of their visual features.

This page is intentionally left blank

CHAPTER IV
RESULTS AND DISCUSSION

This chapter discusses the program implementation and the test results of the *transfer learning*-based potato leaf disease classification model in stages. The discussion is arranged following the sequential experimental scenario designed in the methodology, namely: testing the dataset split proportion (80:10:10, 70:15:15, and 60:20:20), testing the optimization algorithm configuration (Adam–SGD *Nesterov*, Adam–Adam, and SGD *Nesterov*–SGD *Nesterov*), and testing the model architecture (*MobileNetV3Small, MobileNetV3Large, and MobileNetV2*). The best result at each stage is used as the fixed configuration in the next stage, *Hyperparameter* Optimization (*Bayesian Optimization*) to search for the best parameters. The description in this chapter covers the experimental environment, the implementation of two-phase training, the analysis of training dynamics, up to the final performance evaluation using the *accuracy*, *precision*, *recall*, *F1-score*, *classification report*, and *confusion matrix* metrics. With a systematic presentation, this chapter is expected to provide a clear picture of the process of deciding the best model in accordance with the research objectives.

4.1 System Development Environment

The system development environment plays a crucial role in guaranteeing the smoothness, efficiency, and validity of all the experimental stages carried out, from image preprocessing, the execution of model training, up to the final evaluation stage. Documentation regarding the specifications of this environment becomes fundamental for ensuring the aspect of research replication (*reproducibility*) and for providing a comparative picture of the fulfillment of computational needs in executing the MobileNetV3-Small *deep learning* architecture. Given that the neural network architecture in this research is trained using a two-phase scenario that requires intensive memory allocation and graphics processing power, the standardization of the combination of tools greatly determines the stability of the execution time and the convergence of the model. The details of the computational environment specifications implemented in this research are divided into two main components, namely hardware (*hardware*) and software (*software*), which are presented in the following section.

4.1.1 Hardware

Hardware (*hardware*) has a very vital role in determining the execution speed, time efficiency, and stability of the *deep learning* model training process, particularly on the MobileNetV3-Small architecture applied for potato leaf disease classification. Given that this experiment uses a two-phase training scenario, namely Feature Extraction (*Feature Extraction*) and Fine-Tuning (*Fine-Tuning*), the need for graphics processing power (*graphics processing*) and memory storage capacity becomes very intensive. To meet these computational demands, this research applies a hybrid strategy by combining the use of local hardware (*local device*) in the form of an ASUS TUF FX506LH laptop for the purposes of dataset repository management, writing program code blocks, and data analysis, with high-performance cloud-based infrastructure (*cloud computing*) through the Kaggle platform supported by a T4 GPU accelerator to execute the neural network training. The details of the hardware specifications along with their specific functions in supporting the course of this experiment are presented in Table 4.1.

Table 4. 1 Hardware Specifications

• No	**Hardware**	**Component / Specification**	**Role / Function**
1	Local Computer	ASUS TUF FX506LH Laptop : • CPU: Intel Core i5-10300H • GPU: NVIDIA GeForce GTX 1650 • RAM: 16 GB • Storage: SSD	Used for report writing, program code writing (*coding*), local dataset management, and visualization of the final evaluation results.
2	Cloud Infrastructure	Kaggle Environment : • Hardware Accelerator: NVIDIA Tesla P100 GPU	Used as the main computational environment to accelerate the two-phase neural network training process (*training*).

Table 4.1 presents the hardware specifications used during the research process, which are divided into two main working environments. The first environment is a local computer with

ASUS TUF FX506LH specifications used for the purposes of report writing, code writing (*coding*), dataset management, and visualization of the evaluation results. The second environment leverages cloud infrastructure (*cloud computing*) through the Kaggle platform equipped with an NVIDIA Tesla P100 GPU accelerator. The use of this cloud infrastructure is dedicated specifically to handling the heavy computational load, so that the two-phase neural network training process (*training*) can run faster and more efficiently.

4.1.2 Software

Software acts as the foundation of logical instructions and the computational environment that controls the entire course of the experiment. Software ensures that every stage from data processing, neural network architecture design, up to metric calculation is executed consistently and systematically. In this research, the software implementation is configured to support the execution of the *deep learning* library efficiently in the cloud environment so that the algorithm can interact directly with the graphics accelerator. The details of the software specifications along with their supporting libraries are presented in Table 4.2.

Table 4. 2 Software Specifications

• No	**Software / Library**	**Specificati on / Version**	**Main Role / Function in the System**
1	Operating System	Windows 10 (64-bit)	Provides the basic interface on the local computer for manuscript file management and web access for code writing.
2	Development Environment	Kaggle	Serves as the integrated cloud-based *notebook* platform where all Python program code blocks are written, executed, and stored.
3	Programming Language	Python 3	The main programming language used to build the entire logical flow of the leaf disease classification system, and to facilitate the integration of various computation libraries so that they run effectively.
4	TensorFlow & Keras	• TensorFlow version 2.20.0	The core library for loading the *pre-trained* MobileNetV3-Small architecture, composing the *custom head* layers (GlobalAveragePooling2D, Dense, Dropout), compiling the two-phase

• No	Software / Library	Specificati on / Version	Main Role / Function in the System
		• Keras version 3.13.2	model, and processing the image data stream via ImageDataGenerator.
5	Keras Callbacks	TensorFlow Module	Automatic supervisory | components consisting of EarlyStopping (stopping | training if stagnant), | ReduceLROnPlateau | (reducing the learning | rate dynamically), and | ModelCheckpoint (saving | the best model | weights). |
6	Scikit-Learn (Sklearn)	Evaluation Library	Used in the final stage to extract the computational performance values automatically in the form of the classification report metric (classification_report) and the confusion matrix (confusion_matrix).
7	NumPy	Mathematics Library	Manages multidimensional array operations | (*array*) to transform | pixel image data into a | numerical form | understood by the | neural network. |
8	Matplotlib	Basic Visualizati on Library	Plays a role in plotting and visualizing the line/curve graph of the movement of the accuracy (*accuracy*) and loss (*loss*) metric values during the two-phase model training process.
9	Seaborn	Advanced Visualizati on Library	Used specifically to reduce visual complexity and depict the heat map (*heatmap*) on the *Confusion Matrix* result so that the distribution of predictions between classes is easier to interpret.
1 0	OS Module	System Library	A built-in Python function for mapping, reading, and indexing the folder directory where the image *dataset* is stored in the system.

Besides the software specifications that have been described, another fundamental step in preparing the computational environment is setting the reproducibility configuration (*reproducibility*). *Deep learning* algorithms depend heavily on stochastic (random) processes, such as the initialization of the network's initial weights, the shuffling of the training data order (*shuffling*), and the *dropout* mechanism. In order to guarantee experimental consistency and ensure

that the training results can be replicated with exactly the same value if the program is run again, the computational environment is configured by setting a global *seed* value. The seed initialization code snippet can be seen in Code 4.1.

Program Code 4.1 – Reproducibility Configuration (*Seed Setup*)

```
# Set the global seed value so that the training results are consistent every
time it is executed
SEED = 42
os.environ['PYTHONHASHSEED'] = str(SEED)
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)
```

Through the setting of the constant value SEED = 42, the entire computational flow within the cloud environment will have a deterministic *pseudo-random* starting point. To ensure that experiment reproducibility runs comprehensively, the locking of the *seed* value is implemented at four levels of the system's randomization operations, namely:

1. System Library (os.environ): Disabling the built-in *hash* randomization of the Python environment so that the mapping of the basic data structure has a consistent memory order.
2. Base Library (random.seed): Locking the built-in random number generator module of the Python programming language that is often used in basic logic operations.
3. Array Computation Library (np.random.seed): Locking the random number generator in NumPy, which acts as the backbone of mathematical matrix operations and data array transformations behind the scenes.
4. *Deep Learning* Library (tf.random.set_seed): Locking the high-level internal randomization algorithm of TensorFlow. This command is directly responsible for the consistency of the initialization of the initial weight values in the classification layer (*Custom Head*), the randomization of the data distribution order per *batch* (*shuffling*), and the determination of which neurons will be deactivated by the *Dropout* parameter.

4.2 Dataset Preparation and Exploration

The preparation stage begins with loading the entire image *dataset* of potato leaf diseases, totaling 4,072 samples, into the computational environment. Given that the visual representation of the leaf morphology has been described in Figures 2.1 through 2.3 and the mapping of the data split allocation (*data splitting*) along with its distribution graph has been established in Table 3.1, the exploration at this stage is focused on the analysis of the class percentage proportion. This

proportion analysis is very important for detecting the presence of class imbalance (*class imbalance*) that can affect the objectivity of the neural network during the training process. The details of the distribution of the number of image samples along with their percentages are presented in Table 4.3.

Table 4. 3 Distribution of the Number of Potato Leaf Image Samples Per Class

No	Target Class	Leaf Condition	Number of Images	Percentage (%)
1	*Early Blight*	Alternaria solani Spot Infection	2,529	33.39
2	*Late Blight*	Phytophthora infestans Blight Infection	2,524	33.33
3	*Healthy*	Healthy / Normal Condition	2,520	33.28
Total			7,573	100

Based on the data mapping in Table 4.3, it can be analyzed that the pure dataset population (*raw data*) has shown a very balanced class composition (*balanced dataset*). The proportion of the number of images in the three categories is almost equal, in which the Early Blight class amounts to 2,529 images (33.39%), Late Blight amounts to 2,524 images (33.33%), and the Healthy class amounts to 2,520 images (33.28%). This very marginal quantity difference between categories is a very ideal empirical condition for the computation stage.

Next, all input images are programmatically processed to be resized (*resize*) from their original dimensions to the standard resolution of 224x224 pixels. This resolution standardization is an absolute requirement, given that the MobileNetV3-Small architecture, which uses *pre-trained weights* from ImageNet, is configured specifically to receive and extract spatial features on a matrix of that size before the image is distributed into the built-in *preprocessing* function.

4.3 Data Preprocessing and Augmentation (*Preprocessing*)

The preprocessing stage (*preprocessing*) and data augmentation is a transformative mechanism that bridges the raw image format with the matrix input standard that can be understood by the *deep learning* architecture. At this stage, the set of potato leaf data that has been physically divided into three main directories (training, validation, and testing) is loaded into computational memory using the *ImageDataGenerator* module. This module is tasked with reducing the load of random access memory (RAM) through the mechanism of dynamically generating small-sized data (*batching*) while aligning the image pixel distribution to match the distribution of the model's

pre-trained weights. The preprocessing and augmentation initialization code snippet on the training generator is presented in Appendix 2.

Based on the code configuration in Appendix 2, there are three main processing aspects implemented programmatically before the data is extracted by the convolution layer:

1. Dimension Standardization and Batch Distribution

All images are automatically resized to the absolute dimension *target_size=(224, 224)* pixels. This resolution is set given that the base MobileNetV3-Small architecture is designed to extract spatial features optimally at that size. In addition, the batch_size parameter is set to 16 images per iteration. The selection of this relatively small *batch* size aims to make the weight update (*weight update*) occur more frequently in one *epoch*, thereby providing richer gradient information and preventing the model from getting stuck at a local minimum point (*local minima*).

2. Pixel Value Space Adjustment (*Pixel Scaling*)

Unlike conventional models that divide the pixel value by 255 to obtain the range [0, 1], this research explicitly implements the official *preprocess_input* function built into MobileNetV3. This function transforms the original pixel intensity into the range [-1.0, 1.0]. This step is required because the *BatchNormalization* layer in the *ImageNet* pre-trained model stores the statistical distribution of the mean and variance of that value range. Failure to align this range can cause the model to fail to extract visual features optimally.

3. Data Augmentation Implementation

To overcome the indication of mild class imbalance (*mild imbalance*) and to prevent the *overfitting* phenomenon, a rational data augmentation strategy is applied *on-the-fly* to the training data set. Given that the visual representation comparing the images before and after transformation has been illustrated comprehensively, the implementation focus at this stage is emphasized on the technical parameter configuration adjusted to the physical characteristics of the object. These parameters include:

- rotation_range=20: Rotates the image randomly up to a range of 20 degrees, representing the varying angular orientation of potato leaves when photographed by farmers in the field.
- horizontal_flip=True: Flips the image horizontally, based on the principle that the mirror position of a lesion or disease spot does not change the clinical diagnosis of that disease.
- zoom_range=0.1**:** Provides a visual enlargement or reduction effect of 10% to simulate the variation of the image-capture distance between the camera lens and the potato leaf blade.

As a closing note at this stage, all the spatial augmentation methods above are applied exclusively only to the training subset (*training set*). When the validation and testing generators are initialized, that augmentation is deactivated so that the evaluation of the model's performance metrics purely reflects the recognition of the original images without any artificial distortion.

4. Implementation of Real Environmental Condition Augmentation

To overcome the limitation of visual variation in conventional data sets that are generally photographed under ideal laboratory conditions, this research designs a custom injection function (custom_augment_train) as presented in Code 4.4. This function works probabilistically (*on-the-fly*) before the pixel normalization stage is carried out, with the aim of representing the imperfections of image acquisition in real agricultural fields. The technical parameters implemented include:

- Gaussian Blur (30% Probability): If the system generates a random probability below 0.3, the image matrix will be convolved using a spatial filter with a kernel size (ksize) of 3x3 or 5x5 pixels. This computational step is designed specifically to simulate out-of-focus leaf photos (*out of focus*) due to the shaking of the mobile device camera or gusts of wind during sampling.
- Gaussian Noise (30% Probability): Injects a random matrix noise with a normal distribution (with a standard deviation of 10.0) into the image pixels, and limits it (np.clip) so that it does not exceed the value range of 255. The addition of this noise functions to train the model's robustness to visual artifacts that are often produced by low-class camera sensors or in low-light shooting conditions (*low-light*).
- Brightness Variation (40% Probability): Modifies the image lighting level through scalar multiplication with a factor range of 0.7 (darker) to 1.3 (brighter). This fluctuating value range represents the dynamics of sunlight exposure in the open, such as cloudy conditions, the shadows of other leaf canopies, and the harsh midday sun.

The application of this combination of spatial transformation (rotation, *flip*, *zoom*) and pixel-level manipulation (*blur*, *noise*, *brightness*) probabilistically ensures that the network architecture does not process exactly the same image at each *epoch*. This forces the MobileNetV3-Small architecture to extract the core morphology of the *Early Blight* and *Late Blight* spots that is very essential, without relying on resolution sharpness or absolute color intensity. This strategy

effectively minimizes the risk of *overfitting* due to the limited amount of training data, thereby directly impacting the improvement of the model's generalization ability (*robustness*).

After all the Keras generator configurations are executed, the system automatically scans the dataset directory structure to count the total image population and map it into the corresponding classes (labels). The success of this preprocessing and data set proportion splitting process is proven through the program terminal output log (*output*) presented in Output 4.1.

Output 4.1 Execution Log of the *Dataset* Directory Reading

```
Found 3251 images belonging to 3 classes.
Found 416 images belonging to 3 classes.
Found 405 images belonging to 3 classes.
──────────────────────────────────────────────────
DATASET SUMMARY
──────────────────────────────────────────────────
Classes : {'Early_Blight': 0, 'Healthy': 1, 'Late_Blight': 2}
Training Samples : 3574 images
Val Samples : 408 images
Test Samples : 406 images
Batch Size : 16 images/batch
Augmentation : rotation=20, h_flip, zoom=0.1,
GaussianBlur(p=0.3), GaussianNoise(p=0.3),
Brightness[0.7-1.3](p=0.4)
Preprocessing : MobileNetV3 preprocess_input [-1.0, 1.0]
──────────────────────────────────────────────────
```

Based on Output 4.1, the system successfully identified and allocated the total of all potato leaf images into the three target classes accurately. Based on the split parameters (*split ratio*) that were previously set, the program successfully built the training generator (*train_gen*), the validation generator (*val_gen*), and the testing generator (*test_gen*) with an ideal proportion of the number of specimens. With the integrity of this data pipeline (*data pipeline*) verified, the computation stage can be continued to the *deep learning* network architecture design phase.

4.4 MobileNetV3-Small Architecture Design

The architecture design stage is the core of the *deep learning*-based classification system. In this research, the base architecture implemented is MobileNetV3-Small. The selection of the *Small* variant is based on its very lightweight computational parameter efficiency while still maintaining high-level spatial feature extraction capability, so that it is highly relevant to precision computational vision for agricultural devices (*precision agriculture*). The approach used is *Transfer Learning*, in which the model is not built from scratch, but adopts the pre-trained

knowledge (*pre-trained weights*) from the ImageNet *dataset*. The code snippet for the architecture design and the addition of the custom head layer (*custom head*) is presented in Code 4.4.

Program Code 4.4 – Base Architecture Reconstruction and *Custom Head*

```
def build_model(num_classes, input_shape):
# 1. Load the base model (Base Model) MobileNetV3-Small
base_model = MobileNetV3Small(
include_top = False, # Remove the built-in ImageNet
classification layer
weights = 'imagenet', # Load the ImageNet pre-trained
weights
input_shape = input_shape # Determine the RGB input
image resolution (224, 224, 3) |
)
# Freeze all base architecture layers for Phase 1
training
base_model.trainable = False
# 2. Build the minimalist custom head layer stack
(Custom Head)
x = base_model.output
x = GlobalAveragePooling2D(name='global_avg_pool')(x) #
Reduce the spatial dimension of the map
x = Dropout(0.2, name='dropout')(x) # Regularization to
handle overfitting
output = Dense(num_classes, activation='softmax',
name='predictions')(x) # Output 3 classes
# 3. Combine the input and output components into one
complete custom model
model = Model(inputs=base_model.input, outputs=output,
name='MobileNetV3_PLD_v5')
return model, base_model
```

Based on the program implementation in Code 4.4, the architecture design process is divided into two main conceptual stages that are mutually integrated:

1. Base Model Modification (*Base Model*)

The call of the MobileNetV3Small function is accompanied by a very crucial argument, namely include_top=False. This argument programmatically cuts and discards the built-in

classification layer (*Fully Connected Layer*) of the architecture, which was originally designed to detect 1,000 object classes on the ImageNet dataset. Instead, this base model is purely repurposed as a feature extraction engine (*feature extractor*) that will produce a high-level feature map (*feature map*) from the potato leaf specimen images. In addition, the setting of the argument weights='imagenet' is leveraged to initialize the initial values of the convolution matrix through the *transfer learning* approach. Through this initialization, the network no longer needs to learn basic visual feature extraction computationally from scratch (such as edge detection, color gradation, and generic leaf texture), but directly leverages the comprehensive visual knowledge that has been proven. To maintain integrity and prevent damage to these pre-trained weights due to random gradient updates at the beginning of training, all layers of the *base model* are temporarily frozen (trainable = False).

2. Custom Head Layer Construction (*Custom Head*)

To map the feature map into the three specific potato leaf disease classes, a new network structure is built on top of the base model consisting of three sequential layers:

- **GlobalAveragePooling2D:** This layer acts as a very efficient spatial dimension reducer. Unlike the conventional Flatten layer that merely converts the matrix into a long-dimensional vector and often triggers an exponential surge in the number of parameters, this layer computes the aggregate average value of each feature map. This approach not only drastically reduces the computational load and minimizes the risk of *overfitting*, but also makes the network more immune to spatial position shifts (*spatial translation invariance*) of the disease lesion on the leaf surface.
- **Dropout(0.2):** This layer is an implementation of a dynamic regularization technique that operates by randomly deactivating 20% of the neurons at each iteration of the training cycle. This mathematically forces the neural network not to depend excessively on a certain set of minor features (preventing neuron co-adaptation), but to distribute the feature weighting more evenly across the entire architecture.
- **Dense with *Softmax* Activation:** Is the final classification layer containing 3 neurons (representing the *Early Blight*, *Late Blight*, and *Healthy* classes). The use of the *softmax* activation function ensures that the output of the model is a mathematical probability with a total value that sums exactly to 1 (or 100%).

The result of combining the base model and the custom head layer is validated using the model.summary() command. The output of that command confirms that the designed architecture is very efficient. In this initial phase, the majority of parameters have a *Non-trainable* status (belonging to the *base model*), and the model only needs to compute the weight update on the end layer (*trainable parameters*) whose number is very small compared to training the entire network from scratch.

Output 4.2 Summary of the Final Model Architecture Parameters

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 224, 224, 3)	0
...	...	...
(150+ lapisan base_model MobileNet disembunyikan)		
...	...	...
global_avg_pool (GlobalAveragePool…	(None, 576)	0
dropout (Dropout)	(None, 576)	0
predictions (Dense)	(None, 3)	1,731

```
Total params: 940,851 (3.59 MB)
Trainable params: 1,731 (6.76 KB)
Non-trainable params: 939,120 (3.58 MB)
```

Based on Output 4.2, the computational efficiency of the *Transfer Learning* approach can be proven quantitatively. Of the total 940,851 parameters that form the model architecture, 939,120 parameters built into ImageNet are in a frozen status (*Non-trainable params*). This indicates that in Phase 1 (*Feature Extraction*), the computation engine only needs to compute and update the gradient on the 1,731 parameters (*Trainable params*) that purely come from the Dense layer in the *Custom Head*. This very minimalist proportion of active parameters (less than 0.2% of the total parameters) guarantees that the initial training process runs very quickly and avoids the risk of damaging the well-established basic feature representation. After this head layer adapts to the visual features of potato leaves, this architecture is ready to enter the layer unlocking stage (*unfreezing*) in the fine-tuning process (*Fine-Tuning*). This gradual transition strategy becomes a crucial foundation for sharply improving classification accuracy without overburdening memory capacity.

4.5 Implementation of Two-Phase Model Training (*Transfer Learning*)

The training process (*training*) of the MobileNetV3-Small architecture in this research is not carried out in a single computation step, but is implemented through a two-phase strategy (*two-phase training*). This approach is the best practice (*best practice*) in the application of *Transfer Learning* in order to maintain the stability of the convergence of the evaluation metrics and to prevent damage to the pre-trained weights (*pre-trained weights*) that have been initialized from ImageNet.

4.5.1. Phase 1: *Feature Extraction* (Feature Extraction)

The first phase aims to train the custom head layer (*custom head*) that was previously built with random weights (*random initialization*), without changing the feature memory structure of the base model. The execution of Phase 1 is in Code 4.5.

Kode Program 4.5 – Kompilasi dan Pelatihan Model Fase 1

```
# ============================================================
# FASE 1 — FEATURE EXTRACTION (Base Model Frozen)
# ============================================================

def compile_phase1(model, lr_p1=1e-3):
    """
    Kompilasi model untuk Fase 1.
    Hanya custom head yang dilatih, base model tetap dibekukan.
    """
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=lr_p1),
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )
    print(f"✓ Phase 1 dikompilasi | Optimizer: Adam | LR = {lr_p1}")

# 1) Pastikan seluruh layer base model dibekukan
base_model.trainable = False

# 2) Kompilasi Fase 1
compile_phase1(model, lr_p1=1e-3)

# 3) Callback Fase 1 (ReduceLR + EarlyStopping + Checkpoint)
callbacks_p1 = get_callbacks(MODEL_PATH_P1, phase='P1')

# 4) Training Fase 1
history_p1 = model.fit(
    train_gen,
    epochs=EPOCHS_P1,
    validation_data=val_gen,
    callbacks=callbacks_p1,
    verbose=1
)
```

In this *Feature Extraction* phase, all convolution layers in the *base model* are in a frozen status (trainable = False). Programmatically, this limits the parameter update (*trainable parameters*) only to the *Dense* layer at the end of the architecture.

The network compilation in this initial phase explicitly uses the Adam *optimizer* with a learning rate (*learning rate*) of 1e-3. The selection of an adaptive *optimizer* such as Adam is very ideal in this phase because the architecture requires fast and aggressive convergence to map the randomly weighted matrix into a rational loss function (*loss function*), before being continued to the finer adjustment stage.

4.5.2. Phase 2: *Fine-Tuning* (Fine Adjustment)

After the classification layer (*custom head*) reaches a stable basic accuracy level in Phase 1, the computation continues to the *Fine-Tuning* stage. This phase aims to adjust the high-level feature representation of MobileNetV3-Small so that it is more sensitive to the unique microscopic visual characteristics of the potato leaf object, such as distinguishing the concentric ring pattern of *Early Blight* and the necrotic lesion of *Late Blight*. The transition code snippet toward Phase 2 is presented in Appendix 3.

In Appendix 3, there is a transition toward Phase 2 that begins with the unlocking (*unfreezing*) of the top 20 convolution layers of the *base model*. The decision to limit the *fine-tuning* process only to the last 20 layers is based on the principle of efficiency and the *deep learning* feature hierarchy; the initial layers of the neural network detect generic features (such as lines and edges) that are already optimal from ImageNet, whereas the final layers detect specific patterns (disease textures) that need to be adjusted to the research case. In addition, all BatchNormalization layers are kept in a frozen status to keep the mean and variance values (*running statistics*) from ImageNet from being distorted during the weight update iterations.

The most fundamental change in this phase lies in the replacement of the *optimizer*. The adaptive process of Adam is deactivated and replaced by recompilation using *Stochastic Gradient Descent* (SGD). This *optimizer* is configured with a much smaller learning rate (1e-4) and combined with the Nesterov momentum function (0.9). This replacement of the *optimizer* is a mitigation strategy to prevent the *Catastrophic Forgetting* phenomenon, namely the condition in which the network "forgets" the essential features of the pre-trained weights due to gradient updates that are too large. Through the momentum boost of SGD, the movement of the weight update becomes far smoother and more consistent. Nesterov momentum specifically works by

evaluating the gradient at the future predicted position (*look-ahead*), thereby providing a braking effect as the algorithm approaches the minimum point. This proactive mechanism allows the model to reach the final convergence optimality with a high level of precision, without damaging the basic spatial representation that has been formed.

4.6 Testing Scenario and Hyperparameter Optimization

This subsection presents the testing scenario used to determine the configuration of the potato leaf disease classification model empirically and in a structured manner. The testing is designed using a sequential approach (*sequential ablation*), so that each stage only varies one main factor, while the other factors are kept constant so that the effect of each variable can be analyzed objectively. This approach is very crucial given that *deep learning* architectures often operate with very complex inter-parameter interactions. The three testing stages carried out include: variation of the dataset split proportion, variation of the optimization algorithm configuration, and variation of the MobileNet model architecture. Each scenario is run three times with the same *seed* value to neutralize the bias of stochastic weight initialization and maintain the consistency of the computation execution, so that the statistical validity of the experimental results can be accounted for. The best result at each stage is then used as the base configuration in the next stage until the final model with the most optimal performance is obtained.

4.6.1 Data Proportion

At this stage, the testing is focused on the effect of the dataset split proportion on the performance of the potato leaf disease classification model. The three data split scenarios compared are 80:10:10, 70:15:15, and 60:20:20 for the training, validation, and test data. This testing aims to identify the most balanced data composition between the model's ability to learn patterns (*fitting*) and the ability to generalize on new data (*unseen data*). So that the comparison is fair, the model architecture configuration, the two-phase training scheme, and the optimization parameters are kept constant across all scenarios, so that the differences in the results obtained represent the impact of the data proportion directly. The following are the results of the 3 scenarios tested:

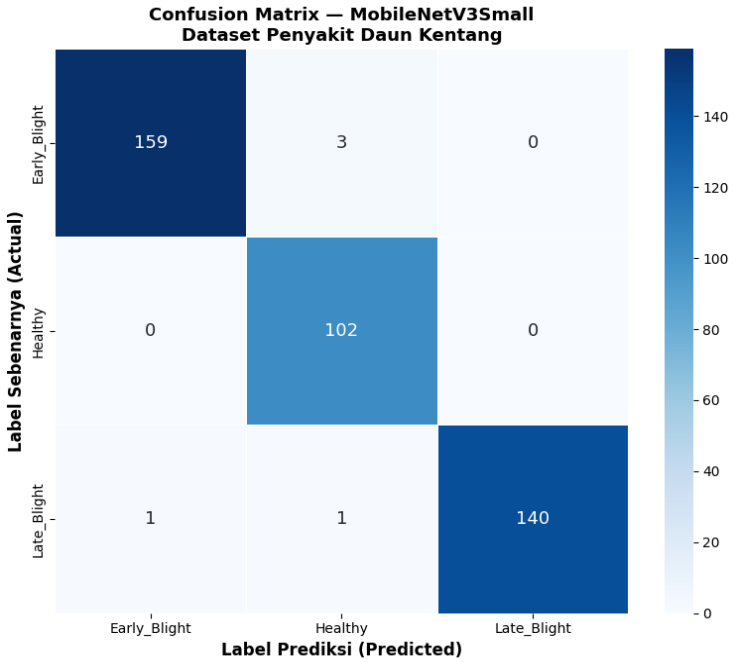


Figure 4. 1 Confusion Matrix 80:10:10

In Figure 4.1, the Confusion matrix shows 159 correct predictions for Early_Blight, 102 correct predictions for Healthy, and 140 correct predictions for Late_Blight. The classification errors are relatively very small, occurring only in a few samples: 3 Early_Blight images predicted as Healthy, and 2 Late_Blight images spread to the Early_Blight and Healthy classes. No errors were found in the Healthy class (0 misclassifications), so this class is detected as the most stable.

CLASSIFICATION REPORT

	precision	recall	f1-score	support
Early_Blight	0.9938	0.9815	0.9876	162
Healthy	0.9623	1.0000	0.9808	102
Late_Blight	1.0000	0.9859	0.9929	142
accuracy			0.9877	406
macro avg	0.9853	0.9891	0.9871	406
weighted avg	0.9880	0.9877	0.9877	406

Figure 4. 2 Classification 80:10:10

In Figure 4.2, the model shows very good classification performance with an overall accuracy of 0.9877 on 406 test data. The precision, recall, and f1-score values across all classes

are above 0.96, indicating that the model is able to identify the three categories consistently. The Late_Blight class has the highest precision (1.0000), which means all predictions in this class are correct, whereas the Healthy class has perfect recall (1.0000), showing that all healthy samples were successfully recognized without being missed. In addition, the macro average value (f1-score 0.9871) and the weighted average value (f1-score 0.9877) that are almost the same indicate a balanced model performance between classes and no bias toward a certain class. This level of reliability of the comprehensive statistical metrics is directly proportional to the network's ability to specifically suppress the number of diagnostic errors on each image sample.

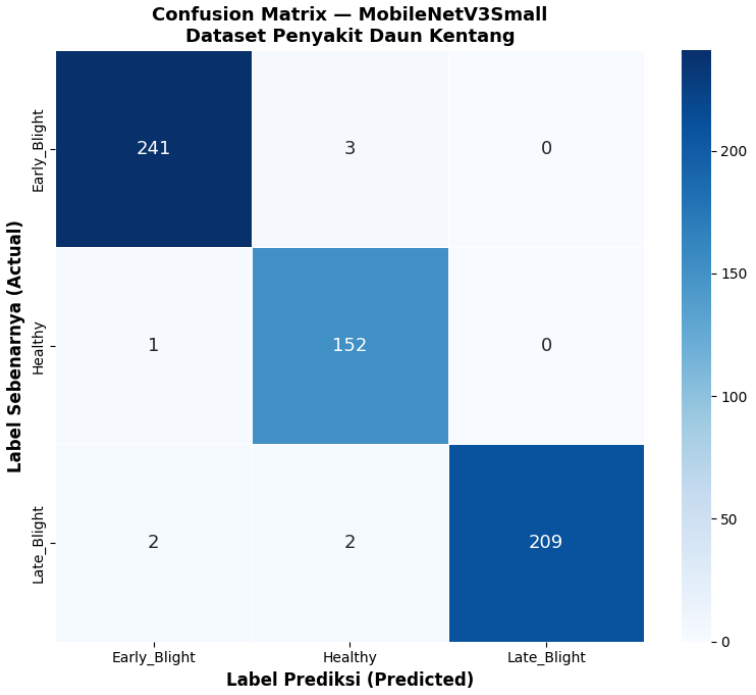


Figure 4. 3 Confusion Matrix 70:15:15

In Figure 4.3, the *confusion matrix* shows that the majority of predictions — 241 Early_Blight images, 152 Healthy images, and 209 Late_Blight images — were classified correctly. The number of classification errors is relatively low: 3 Early_Blight images predicted as Healthy, 1 Healthy image predicted as Early_Blight, and 4 Late_Blight images mistaken for other classes (2 to Early_Blight and 2 to Healthy). There is no Late_Blight predicted in a misdirected way toward a certain dominant class, so the distribution of errors remains small and does not show extreme bias.

CLASSIFICATION REPORT

	precision	recall	f1-score	support
Early_Blight	0.9877	0.9877	0.9877	244
Healthy	0.9682	0.9935	0.9806	153
Late_Blight	1.0000	0.9812	0.9905	213
accuracy			0.9869	610
macro avg	0.9853	0.9875	0.9863	610
weighted avg	0.9871	0.9869	0.9869	610

Figure 4. 4 Classification Report 70:15:15

Based on Figure 4.4, the model obtained an overall accuracy of 0.9869 on a total of 610 test data, which shows very good classification performance. The precision, recall, and f1-score values across the three classes are in a high range (around 0.9682–1.0000), indicating that the model is able to recognize disease patterns consistently. The Late_Blight class has perfect precision (1.0000), so all predictions in that class produce no false positives. The Healthy class shows the highest recall (0.9935), which means almost all healthy leaf images were successfully detected correctly. In addition, the macro average and weighted average values that are close to each other (f1-score 0.9863 and 0.9869) indicate that the model's performance is relatively balanced between classes.

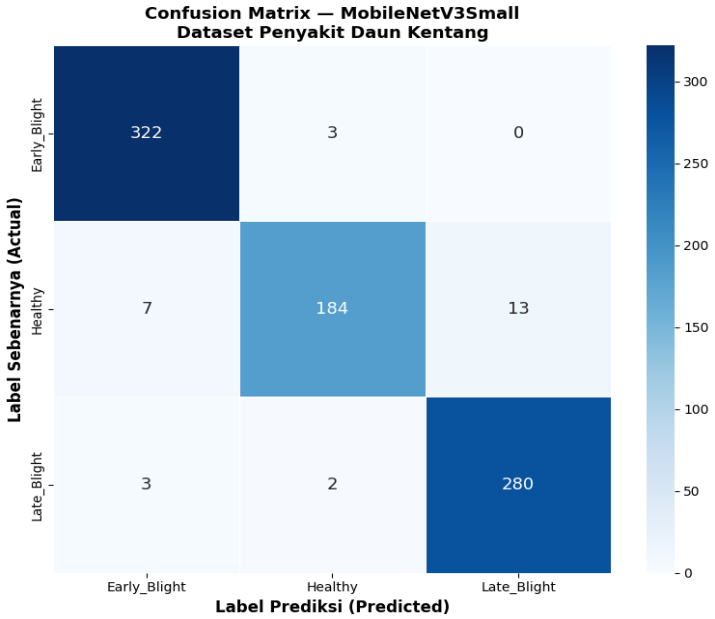


Figure 4. 5 Confusion Matrix 60:20:20

In Figure 4.5, the confusion matrix shows 322 for Early_Blight, 184 for Healthy, and 280 for Late_Blight. Nevertheless, the most prominent classification error occurs in the Healthy class, which is quite often predicted as Late_Blight (13 images) and a small portion as Early_Blight (7 images). Conversely, the errors in the Early_Blight and Late_Blight classes are relatively low.

CLASSIFICATION REPORT

	precision	recall	f1-score	support
Early_Blight	0.9699	0.9908	0.9802	325
Healthy	0.9735	0.9020	0.9364	204
Late_Blight	0.9556	0.9825	0.9689	285
accuracy			0.9656	814
macro avg	0.9664	0.9584	0.9618	814
weighted avg	0.9658	0.9656	0.9653	814

Figure 4. 6 Classification Report 60:20:20

In Figure 4.6, the model obtained an overall accuracy of 0.9656 on 814 test data, which shows that the classification performance is already good but not as high as the best previous scenario. The *f1-score* values for the *Early_Blight* (0.9802) and *Late_Blight* (0.9689) classes are still high, whereas the *Healthy* class has the lowest *f1-score* (0.9364). This indicates that the model has relatively more difficulty distinguishing healthy leaf images compared to the two disease classes. In aggregate, the *macro average f1-score* value of 0.9618 shows a fairly balanced performance between classes, but there is still a decline in the sensitivity of one of the classes.

Table 4. 4 Data Proportion Test Results

Model	Scenario	Testing Accuracy (%)
MobileNetV3Small	**80:10:10**	**98.77**
	70:15:15	98.69
	60:20:20	96.56

In Table 4.4, the testing of the data split proportion scenario shows that the MobileNetV3Small model reaches the best performance in the 80:10:10 scenario with a testing accuracy of 98.77%. The 70:15:15 scenario produces a very close accuracy, namely 98.69%, with a difference of only 0.08 percentage points from the best scenario. Meanwhile, the 60:20:20

scenario gives the lowest accuracy of 96.56%, which indicates a decline in performance when the portion of training data becomes smaller. This finding shows that increasing the proportion of training data contributes positively to the model's ability to learn the visual patterns of potato leaf diseases, so that the 80:10:10 scenario was chosen as the most optimal data proportion configuration for the next testing stage.

4.6.2 Optimizer

In this subsection, the testing is focused on comparing the optimization algorithm configuration in the two-phase training scheme to determine the most effective combination for improving classification performance. After the best data proportion was obtained in the previous stage, the variable that is specifically changed is the optimizer in Phase 1 and Phase 2, whereas other components such as the model architecture, image size, augmentation strategy, and *fine-tuning* configuration are kept constant so that the comparison is fair. The three scenarios tested include Adam–SGD *Nesterov* (default), Adam–Adam, and SGD *Nesterov*–SGD *Nesterov*. Through this testing, the research aims to identify the optimization configuration that is able to produce stable convergence, suppress the risk of *catastrophic forgetting*, and provide the highest testing accuracy to be used in the next model architecture selection stage. The following are the results of the 3 scenarios tested.

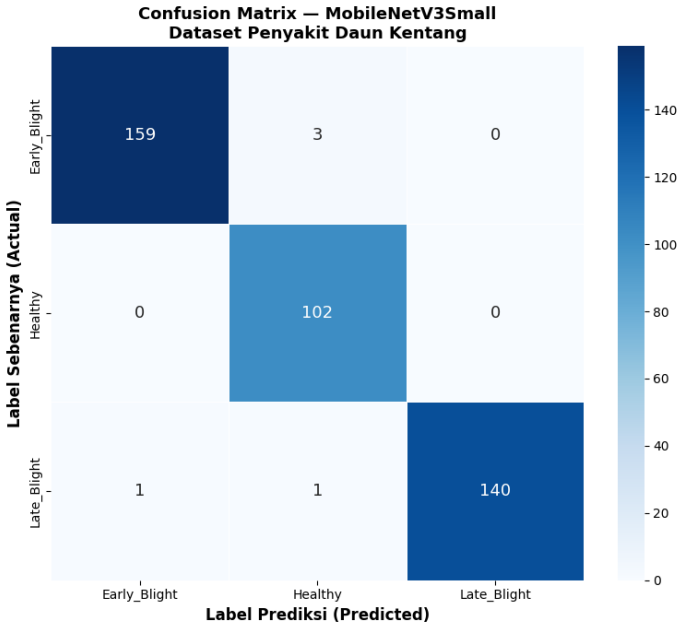


Figure 4. 7 Confusion Matrix Adam-SGD

In Figure 4.7, the *confusion matrix* shows correct predictions on 159 Early_Blight images, 102 Healthy images, and 140 Late_Blight images. The misclassification rate is very minimal, namely only 3 Early_Blight samples (detected as Healthy) and 4 Late_Blight samples (2 to Early_Blight, 2 to Healthy). The absence of prediction errors (0 misclassifications) on healthy leaf specimens affirms the very high stability of the model in recognizing the Healthy class.

CLASSIFICATION REPORT

	precision	recall	f1-score	support
Early_Blight	0.9938	0.9815	0.9876	162
Healthy	0.9623	1.0000	0.9808	102
Late_Blight	1.0000	0.9859	0.9929	142
accuracy			0.9877	406
macro avg	0.9853	0.9891	0.9871	406
weighted avg	0.9880	0.9877	0.9877	406

Figure 4. 8 Classification Report Adam-SGD

In Figure 4.8, the model reaches an overall accuracy of 0.9877 on 406 test data. The *precision*, *recall*, and *f1-score* values are consistently above 0.96 for all classes. The Late_Blight class records perfect *precision* (1.0000), while the Healthy class achieves perfect *recall* (1.0000), which means no healthy samples were missed. The closeness of the *macro average* (0.9871) and *weighted average* (0.9877) values on the *f1-score* also confirms that the model's performance is very balanced between classes and free from dominant prediction bias.

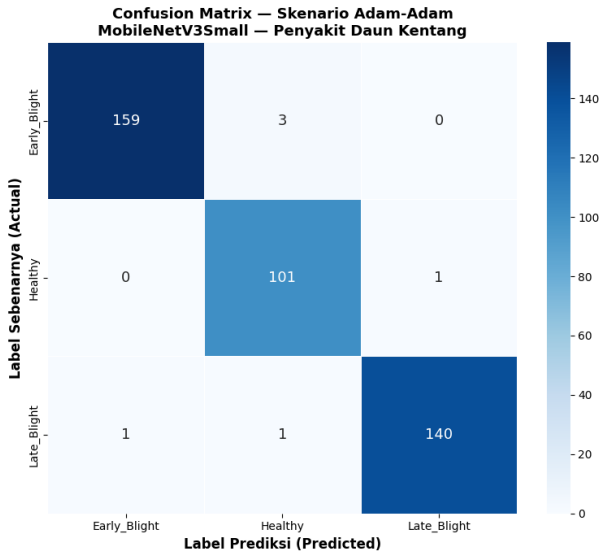


Figure 4. 9 Confusion Matrix Adam-Adam

In Figure 4.9, the Confusion matrix shows correct predictions on 159 Early_Blight images, 101 Healthy images, and 140 Late_Blight images that were classified accurately. The number of classification errors is relatively small, respectively 3 Early_Blight images predicted as Healthy, 1 Healthy image predicted as Late_Blight, and 2 Late_Blight images spread to other classes. This pattern shows that the model has good class-separation ability, with errors generally occurring on samples that have visual similarity between classes. This distribution of correct and incorrect guesses mathematically becomes the main foundation in the calculation of the model's overall performance. The network's ability to suppress the number of false positives (*false positive*) and false negatives (*false negative*) on these mutually resembling leaf specimens proves the high reliability of the model's feature extraction, whose quantitative representation of that success is further evaluated through the description of statistical metrics.

CLASSIFICATION REPORT

	precision	recall	f1-score	support
Early_Blight	0.9938	0.9815	0.9876	162
Healthy	0.9619	0.9902	0.9758	102
Late_Blight	0.9929	0.9859	0.9894	142
accuracy			0.9852	406
macro avg	0.9829	0.9859	0.9843	406
weighted avg	0.9855	0.9852	0.9853	406

Figure 4. 10 Classification Report Adam-Adam

In Figure 4.10, the *classification report* result shows that the model reaches an overall accuracy of 98.52% on 406 test data. Per class, the model's performance is very good, with *f1-score* values of 0.9876 for *Early_Blight*, 0.9758 for *Healthy*, and 0.9894 for *Late_Blight*. The *Early_Blight* and *Late_Blight* classes have a high and relatively balanced *precision–recall* combination, whereas the *Healthy* class has high *recall* (0.9902) but slightly lower *precision* (0.9619), which indicates that there are still a small number of erroneous healthy positive predictions. The *macro average* (0.9843) and *weighted average* (0.9853) values also confirm that the model is able to maintain strong performance across all classes.

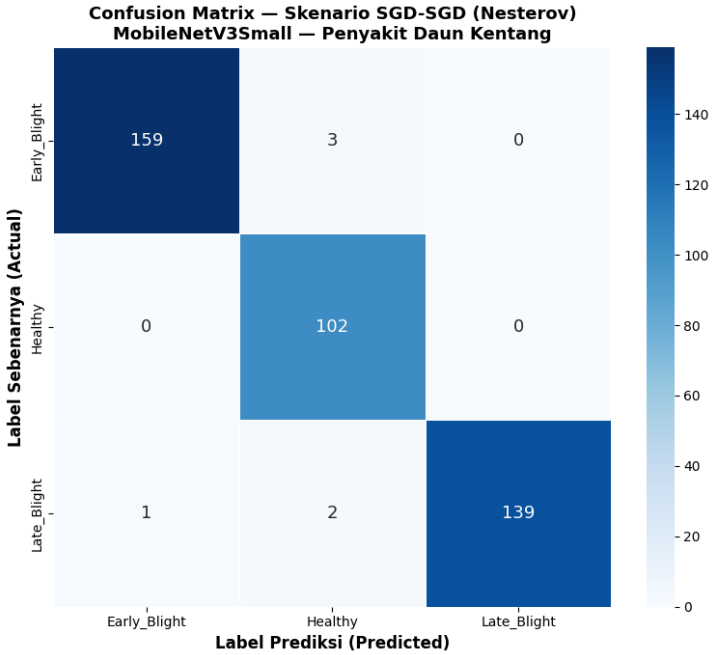


Figure 4. 11 Confusion Matrix SGD Nesterov-SGD Nesterov

In Figure 4.11, the *confusion matrix* shows the dominance of correct values on the main diagonal, namely 159 correct predictions for Early_Blight, 102 correct predictions for Healthy, and 139 correct predictions for Late_Blight. This concentration of distribution on the diagonal area empirically proves that the network has succeeded in formulating a firm decision boundary (*decision boundary*) in separating the visual characteristics between categories. The number of classification errors is relatively small: 3 Early_Blight images predicted as Healthy, and 3 Late_Blight images mispredicted to other classes (1 to Early_Blight and 2 to Healthy). This very minimal error distribution is generally triggered by the complexity of the image sample itself, such as early-stage infection symptoms whose spots are still very faint so that they resemble normal specimens.

CLASSIFICATION REPORT

	precision	recall	f1-score	support
Early_Blight	0.9938	0.9815	0.9876	162
Healthy	0.9533	1.0000	0.9761	102
Late_Blight	1.0000	0.9789	0.9893	142
accuracy			0.9852	406
macro avg	0.9823	0.9868	0.9843	406
weighted avg	0.9858	0.9852	0.9853	406

Figure 4. 12 Classification Report SGD Nesterov-SGD Nesterov

Figure 4.12, the classification report result shows that the model reaches a testing accuracy of 98.52% on a total of 406 test images, which indicates very good classification performance. At the class level, the model provides a high f1-score for Early_Blight (0.9876) and Late_Blight (0.9893), while the Healthy class obtains an f1-score of 0.9761. The recall value of the Healthy class reaches 1.0000, meaning all healthy leaf samples were successfully recognized, although its precision (0.9533) shows that there are still some healthy predictions that come from other classes. In general, the macro average (0.9843) and weighted average (0.9853) values show consistent and balanced model performance across all classes.

Table 4. 5 Optimizer Test Results

Model	Scenario	Testing Accuracy (%)	Validation Accuracy (%)
MobileNetV3Small	**Adam - SGD Nesterov**	**98.77**	**98.78**
	Adam - Adam	98.52	99.27
	SGD – SGD (Nesterov)	98.52	98.03

Based on Table 4.5, the comparison of the three optimizer scenarios on the MobileNetV3Small model shows that the Adam–SGD Nesterov configuration produces the best performance with a testing accuracy of 98.77% and a validation accuracy of 98.78%. The Adam–Adam and SGD–SGD (Nesterov) scenarios both produce a testing accuracy of 98.52%, but their validation accuracy values differ, namely 99.27% for Adam–Adam and 98.03% for SGD–SGD. Although Adam–Adam has the highest validation accuracy, the decision to select the best configuration still refers to the testing accuracy as the main metric of generalization on data that is

truly unseen during training. Therefore, the Adam–SGD Nesterov scenario was chosen as the best optimizer configuration to be used in the next model architecture testing stage.

4.6.3 Model

In this subsection, the testing is focused on comparing the base architectures (*backbone*) — namely MobileNetV3-Small, MobileNetV3-Large, and MobileNetV2 — to find the most optimal classification model. By locking all parameters using the best configuration from the previous stage, this objective evaluation aims to identify the architecture that offers the best balance between the level of accuracy, performance stability, and computational efficiency. The following are the test results of the three scenarios:

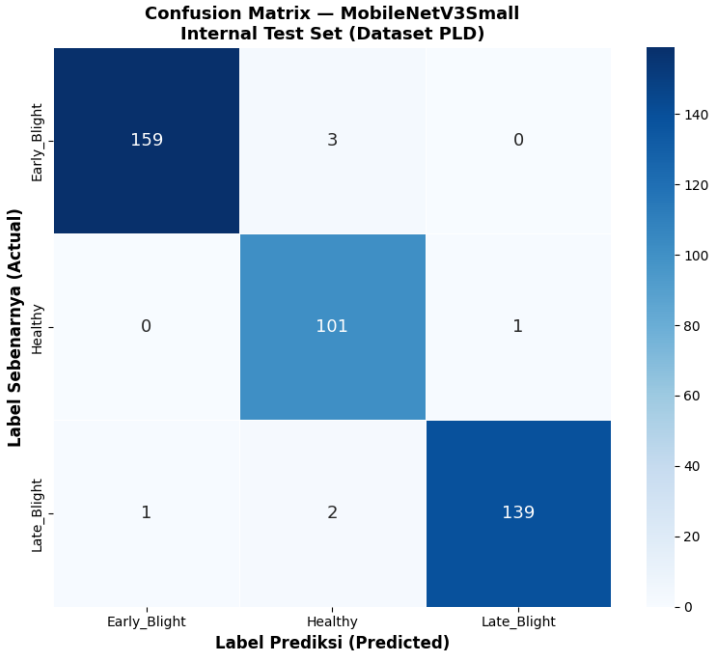


Figure 4. 13 Confusion Matrix MobileNetV3Small

In Figure 4.13, the main diagonal values dominate, namely 159 correct predictions for Early_Blight, 101 correct predictions for Healthy, and 139 correct predictions for Late_Blight. The classification errors are very minimal, with only 3 Early_Blight samples predicted as Healthy, 1 Healthy sample predicted as Late_Blight, and 3 Late_Blight samples mispredicted to other classes (1 to Early_Blight and 2 to Healthy).

CLASSIFICATION REPORT — INTERNAL

	precision	recall	f1-score	support
Early_Blight	0.9938	0.9815	0.9876	162
Healthy	0.9528	0.9902	0.9712	102
Late_Blight	0.9929	0.9789	0.9858	142
accuracy			0.9828	406
macro avg	0.9798	0.9835	0.9815	406
weighted avg	0.9832	0.9828	0.9828	406

Figure 4. 14 Classification Report MobileNetV3Small

In Figure 4.14, the model produces a testing accuracy of 98.28% on 406 test data. The *precision*, *recall*, and *f1-score* values for the three classes are in a very high range, namely above 0.95. The *Early Blight* class has the highest *precision* (0.9938), whereas the *Healthy* class has the highest *recall* (0.9902), which shows the model's very good ability to recognize healthy leaves with a very minimal rate of missed samples. The *macro average* (*f1-score* 0.9815) and *weighted average* (*f1-score* 0.9828) values that are close to each other indicate consistent model performance between classes and show no significant bias toward a certain class.

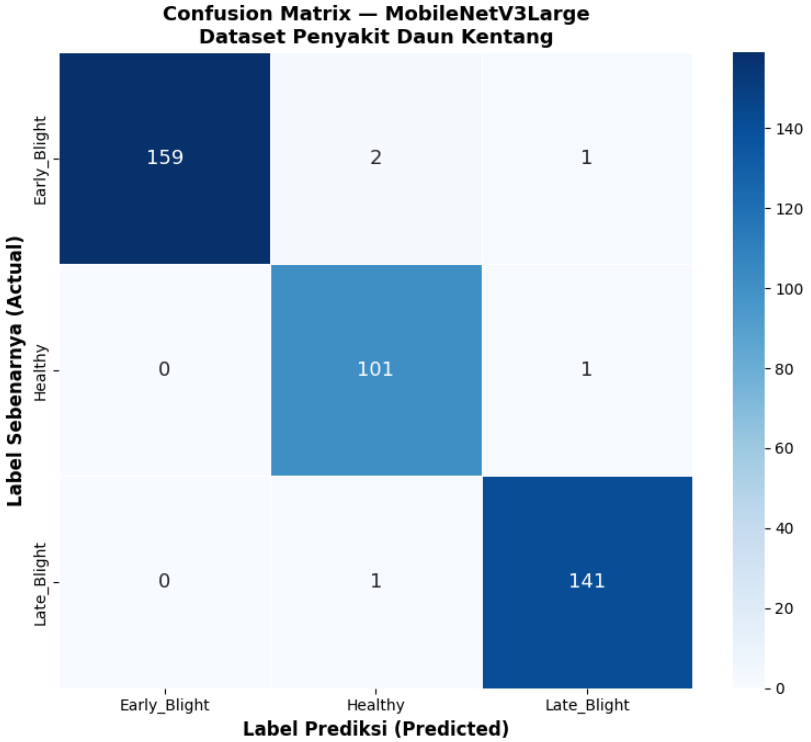


Figure 4. 15 Confusion Matrix MobileNetV3Large

In Figure 4.15, the MobileNetV3Large model is able to classify the majority of samples accurately across the three classes. In the Early_Blight class, 159 of 162 data were successfully predicted correctly, with small errors to the Healthy class (2 data) and Late_Blight (1 data). In the Healthy class, the model recognizes 101 of 102 data correctly, with only 1 data mixed up into Late_Blight. Meanwhile, the Late_Blight class has 141 correct predictions out of 142 data, with 1 data wrong to Healthy. This pattern affirms that the model's errors are very low and mainly occur between classes that are visually similar, whereas the class separation in general is already very good.

CLASSIFICATION REPORT

	precision	recall	f1-score	support
Early_Blight	1.0000	0.9815	0.9907	162
Healthy	0.9712	0.9902	0.9806	102
Late_Blight	0.9860	0.9930	0.9895	142
accuracy			0.9877	406
macro avg	0.9857	0.9882	0.9869	406
weighted avg	0.9879	0.9877	0.9877	406

Figure 4. 16 Classification Report MobileNetV3Large

Figure 4.16, with a very high overall performance (accuracy 98.28%). The precision, recall, and f1-score values across the three classes are in the range of 0.97 to 1.00, indicating that the model is not only accurate but also consistent in suppressing false positives and false negatives. The Early_Blight class has the highest f1-score (0.9907), the Late_Blight class is also very strong (0.9895), whereas the Healthy class is slightly lower (0.9806) but still very good. The macro average f1-score value of 0.9869 shows a balanced performance between classes.

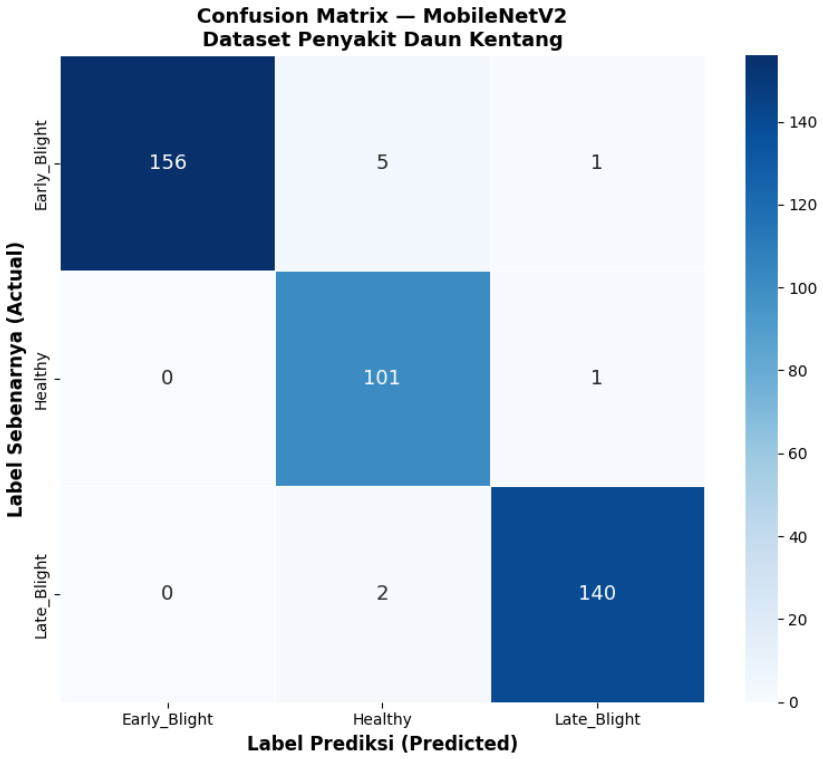


Figure 4. 17 Confusion Matrix MobileNetV2

Figure 4.17, the test results of the MobileNetV2 model on the potato leaf disease dataset. Of the total test data, most of the samples were successfully classified correctly, as seen from the dominant main diagonal values, namely 156 Early_Blight samples, 101 Healthy samples, and 140 Late_Blight samples. Prediction errors still exist but are relatively small, especially in the Early_Blight class which was several times predicted as Healthy, and in the Late_Blight class which was in a small part predicted as Healthy. This pattern shows that the model is already able to distinguish the three classes fairly well, although the Healthy class is still the main source of confusion because it is located visually closest to the two other disease classes. The network's tendency to be slightly biased toward the Healthy class when experiencing misclassification directly impacts the calculation of the false positive ratio (*false positive*).

CLASSIFICATION REPORT

	precision	recall	f1-score	support
Early_Blight	1.0000	0.9630	0.9811	162
Healthy	0.9352	0.9902	0.9619	102
Late_Blight	0.9859	0.9859	0.9859	142
accuracy			0.9778	406
macro avg	0.9737	0.9797	0.9763	406
weighted avg	0.9788	0.9778	0.9780	406

Figure 4. 18 Classification Report MobileNetV2

Figure 4.18, the overall accuracy value reaches 97.78%, which indicates that the model's performance is very good. The Early_Blight class has perfect precision, but slightly lower recall, which means all Early_Blight predictions are correct, although there are still some Early_Blight data that were not successfully recognized. The Healthy class has very high recall, but lower precision compared to other classes, so there are quite a lot of samples from other classes that were predicted as Healthy. Meanwhile, the Late_Blight class shows the most balanced performance with equally high precision, recall, and f1-score. Overall, this classification report shows that the MobileNetV2 model is fairly stable in classifying all classes, although its performance is still slightly below the model with the best accuracy in the previous experiment.

Table 4. 6 Model Test Results

Scenario	**MobileNetV3Small**	MobileNetV3Large	MobileNetV2
Testing accuracy (%)	**98.28**	98.28	97.78
Validation Accuracy (%)	**98.29**	98.15	98.28
Test Loss	**0.0567**	0.0514	0.0933
Macro F1-Score	**98.15**	98.15	97.63
Training Computation Time (s)	**2265**	2017	2281
Total Parameters	**940,851**	2,999,235	2,261,827
Model Size (MB)	**3.59**	11.44	8.63

Table 4.6 displays a comparison of the performance and computational efficiency of the three architectures, namely MobileNetV3Small, MobileNetV3Large, and MobileNetV2. MobileNetV3Small and MobileNetV3Large both reach the highest *testing accuracy* of 98.28%,

whereas MobileNetV2 is at 97.78%. In terms of *test loss*, MobileNetV3Large records the lowest value (0.0514), followed by MobileNetV3Small (0.0567) and MobileNetV2 (0.0933). However, MobileNetV3Small excels in validation accuracy (98.29%) and *macro F1-score* (98.15%), which shows a very good balance of performance.

In terms of efficiency, the *training* time of MobileNetV3Large is indeed the lowest (2,013 seconds), but this is influenced by *early stopping* at the 24th *epoch*, not because the model is lighter. In terms of model complexity, MobileNetV3Small is the most efficient with 940,851 parameters and a size of 3.59 MB, far smaller than MobileNetV3Large (2,999,235 parameters; 11.44 MB) and MobileNetV2 (2,261,827 parameters; 8.63 MB). Based on all the metrics, MobileNetV3Small was chosen because it provides an accuracy equal to the highest model with the best computational efficiency. This architectural advantage in cutting the number of parameters without sacrificing feature extraction power makes it very ideal if it is later implemented on mobile computing devices (*mobile*) that have memory limitations.

4.6.4. Hyperparameter Optimization (Bayesian Optimization)

After determining the dataset split proportion and the best *optimizer* configuration in the previous ablation stage, the process continues by searching for the most optimal *hyperparameter* combination on the MobileNetV3-Small architecture. This optimization process is carried out programmatically leveraging the *Bayesian Optimization* method through the Keras Tuner library by setting the validation accuracy (*validation accuracy*) as the main target metric (*objective function*).

The search is carried out dynamically through 20 experimental combinations (*trials*) with the overall computation duration taking 3 hours 21 minutes 31 seconds in the GPU environment. The search space limits (*search space*) are focused on four crucial parameters, namely the Phase 1 *learning rate* (Adam), *dropout rate*, *batch size*, and the number of convolution layers unfrozen for Phase 2 *fine-tuning* (*n_layers_to_unfreeze*). The details of the summary of the top-three (*Top-3*) ranking of the best trials resulting from the *Bayesian Optimization* computation are presented in Table 4.7.

Table 4. 7 Summary of the Top-3 Bayesian Optimization Results

No	Test Acc	Test Loss	Dropout	LR P1	Batch	Unfreeze
1	97.89%	0.0575	0.1	0.005	16	10
2	97.89%	0.0599	0.1	0.005	16	10
3	98.15%	0.0458	0.1	0.005	16	40

No	Test Acc	Test Loss	Dropout	LR P1	Batch	Unfreeze
Baseline	-	-	0.2	0.001	16	20

Based on the quantitative data in Table 4.7, the final computation evaluation successfully identified the empirically best configuration in the third position (Top-3) with the highest test accuracy (*test accuracy*) of 98.15% and the lowest *loss* value of 0.0458. These results provide several very crucial analytical findings related to the convergence behavior of the MobileNetV3-Small model in classifying potato leaf disease images:

1. **Learning Rate Dynamics (*Learning Rate*):** The top three rankings consistently chose a Phase 1 *learning rate* value of **0.005**. This value is larger and more aggressive compared to the initial *baseline* value of the research (0.001). This proves that with the support of the massive data augmentation implementation in the preprocessing stage, the model requires a slightly larger gradient jump at the beginning of training to escape from the local minimum point (*local minima*) and accelerate the convergence of the weight matrix.
2. **Regularization Balance (*Dropout Rate*):** The optimal configuration sets a *dropout rate* value of 0.1, which is a decrease from the *baseline* value (0.2). This decrease indicates that with a dataset composition condition that is already very balanced naturally, the architecture does not require deactivating neurons too strictly. Maintaining 90% of the neural nodes so that they remain active in the *Fully Connected* layer has been proven far more effective in preserving the integrity of the visual feature information of the disease lesion without triggering the risk of *overfitting*.
3. ***Fine-Tuning* Depth (*Layers to Unfreeze*):** The computational decision to unfreeze the last 40 layers of the MobileNetV3-Small architecture to be finely adjusted provides the most superior metric representation (98.15%). This deeper *unfreeze* proportion than the *baseline* (20 layers) provides computational flexibility for the upper convolution layers (*higher-level convolutional layers*). Through this step, the ImageNet built-in pre-trained weights can be re-engineered to be more adaptive to the unique spatial patterns of *Alternaria solani* spots (*Early Blight*) and *Phytophthora infestans* (*Late Blight*), without triggering the *catastrophic forgetting* phenomenon.
4. ***Batch Size* Stability:** All the top configurations agreed to maintain a *batch size* of 16. This affirms that a compact *batch* size is very ideal for the MobileNetV3-Small

architecture because it is able to maintain constant gradient estimation variability, so that the *backpropagation* process runs with high precision at every *epoch*.

Through the mathematical mapping carried out by the probabilistic *Gaussian* function via the mapping of objective performance metrics on this test data, the parameter configuration in the third position is proven to mathematically outperform the other settings. This parameter combination was then absolutely determined as the final model architecture to be executed and tested for its robustness thoroughly. This decision also affirms that a reliable model performance evaluation does not only rely on the height of the validation accuracy alone, but on its ability to adapt to data that has never been learned before (unseen data).

4.7 Analysis of Training Stability and the Effect of *Callbacks*

The evaluation of the model's learning stability is not only measured through the final result, but also through the analysis of the movement of the accuracy (*accuracy*) and loss (*loss*) metrics at each *epoch* during the course of the computation. To guarantee stable convergence and prevent the occurrence of *overfitting*, this research implements an automatic training control mechanism (*callbacks*). The initialization code snippet of the *callbacks* function used in both training phases is presented in Code 4.7.

Kode Program 4.7 – Inisialisasi Mekanisme Kontrol Pelatihan Otomatis (*Callbacks*)

```
def get_callbacks(model_save_path, phase='P1'):
    # 1. ReduceLROnPlateau: Menurunkan learning rate secara dinamis jika
evaluasi stagnan
    reduce_lr = ReduceLROnPlateau(
        monitor  = 'val_loss',
        factor   = 0.5,        # Memotong nilai learning rate menjadi
setengahnya (50%)
        patience = 3,          # Batas toleransi stagnasi selama 3 epoch
        min_lr   = 1e-6,       # Batas bawah learning rate terkecil
        verbose  = 1
    )

    # 2. EarlyStopping: Menghentikan komputasi jika model terindikasi
overfitting
    # Batas toleransi (patience) disesuaikan: 10 epoch untuk Fase 1, 8 epoch
untuk Fase 2
    patience_es = 10 if phase == 'P1' else 8
    early_stop = EarlyStopping(
        monitor              = 'val_loss',
        patience             = patience_es,
        restore_best_weights = True,     # KRUSIAL: Mengembalikan bobot pada
epoch terbaik
        verbose              = 1
    )

    # 3. ModelCheckpoint: Menyimpan bobot jaringan secara fisik ke dalam
penyimpanan
    checkpoint = ModelCheckpoint(
        filepath       = model_save_path,
        monitor        = 'val_accuracy',
        save_best_only = True,           # Hanya menyimpan jika akurasi
validasi meningkat
        verbose        = 1
    )

    # Fase 1 (Adam) mengembalikan ketiga parameter kontrol
    # Fase 2 (SGD Momentum) membuang ReduceLROnPlateau agar pembaruan stabil
    if phase == 'P1':
        return [reduce_lr, early_stop, checkpoint]
    else:
        return [early_stop, checkpoint]
```

Based on the implementation in Code 4.7, there are three intervention functions that work silently in the background during the computation process:

1. **ReduceLROnPlateau:** Functions in Phase 1 by monitoring the improvement of the validation metric. Through this mechanism, the built-in *learning rate* of the Adam *optimizer* is reduced gradually when convergence begins to slow down (*plateau*). In the results of this experiment, the intervention of reducing the *learning rate* was recorded dynamically at the 21st *epoch* (to 0.00125), the 24th *epoch* (to 0.000625), and the 27th

epoch (to 0.0003125). This adaptive mechanism proved essential to help keep the optimization process stable and avoid weight-update jumps that are too aggressive approaching the optimal point.

2. **EarlyStopping:** Acts as an *overtraining* prevention mechanism with the option restore_best_weights = True. In the Phase 1 experiment, this mechanism actively cut the iteration process and stopped training early at the 28th *epoch* because there was no longer any significant metric improvement. The system then automatically restored the best recorded weight representation from the 18th *epoch*. Meanwhile, in Phase 2, EarlyStopping was not triggered, and the training continued smoothly up to the maximum limit of 30 *epochs*. This became empirical evidence that in the fine-tuning phase, the model still continued to learn and refine features without experiencing continuous deterioration of the validation metric.
3. **ModelCheckpoint:** Plays a role in monitoring the val_accuracy value and automatically saving the model only when there is a performance improvement (save_best_only = True). In Phase 1, the best recorded validation accuracy reached 98.28%. Entering Phase 2, the checkpoint (*checkpoint*) continued to update the model store until it reached its peak at the 29th *epoch* with a validation accuracy level of 99.02%. Thus, the exported final model is the best mathematical representation based on peak performance, not merely the random weight condition at the last *epoch*.

The results of this strict control by the three *callbacks* are represented in a continuous computation history visualization (combining Phase 1 and Phase 2). This graphic visualization not only shows the trend of increasing accuracy and decreasing *loss*, but also provides an analytical representation of the stability of the learning transition as the architecture moves from the feature extraction stage to *fine-tuning*. The visualization result is presented in Figure 4.19.

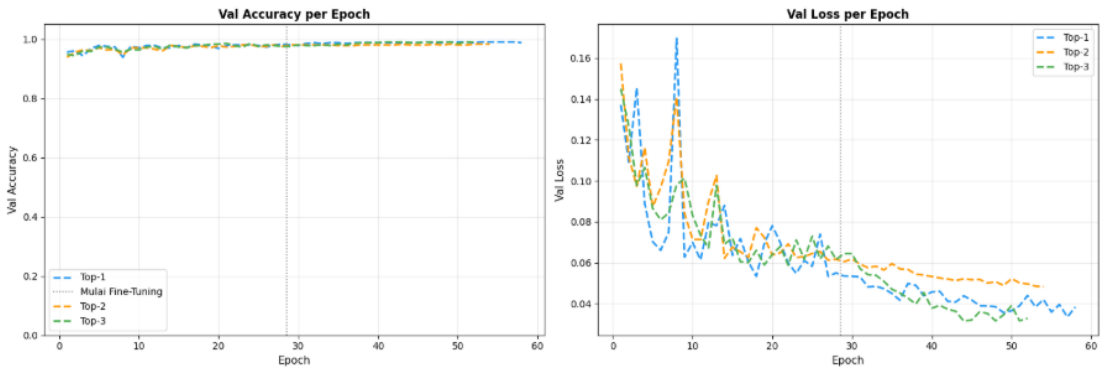


Figure 4. 19 Accuracy and Loss Movement Curve

Analysis of the curve in Figure 4.19 shows the evaluation of computation stability through the convergence pattern in both training phases. At the beginning of Phase 1 (*feature extraction*), the trajectory of the training and validation accuracy curve soared up exponentially, accompanied by a sharp decline in the loss value (*loss*). This pattern confirms that the Adam *optimizer* works effectively in mapping the initial weights on the custom classifier layer, in which the val_accuracy value successfully touched the optimal figure of 98.28%. The very responsive convergence acceleration in this initial phase also proves that the use of pre-trained weight initialization (*pre-trained weights*) is effective in making the computation time more efficient.

Entering Phase 2 (*fine-tuning*), marked by the vertical dashed dividing line on the curve movement graph, it continues with a more gradual and precise trajectory. The computational decision to unfreeze the last 40 convolution layers and optimize them using the SGD algorithm with Nesterov momentum proved not to trigger the *catastrophic forgetting* phenomenon (network memory damage). Conversely, the val_loss curve continued to be suppressed slowly until it touched the lowest point of 0.0335 at the 29th epoch, accompanied by the achievement of a maximum val_accuracy of 99.02%. The very tight margin difference between the training and validation accuracy trajectories up to the end of the iterations provides strong indication that the MobileNetV3-Small architecture not only reaches a stable global convergence point, but also has high generalization ability with a very minimal probability of *overfitting*.

4.8 Testing and Performance Evaluation of the Final Model

In this subsection, testing and evaluation are carried out on the final model that has been chosen based on the results of the previous comparison scenarios, namely MobileNetV3Small. The testing is carried out using a test data set that was not involved in the training or validation process, so that the results obtained can describe the generalization ability of the model objectively.

4.8.1 Internal Testing

The final evaluation is carried out using several metrics, namely test accuracy, test loss, the classification report which includes precision, recall, and F1-score, and the confusion matrix to map the distribution of correct predictions and classification errors in each class. The code snippet for the testing stage and the generation of evaluation metrics is presented in Code 4.9.

Program Code 4.9 – Implementation of Model Testing and Generation of the Evaluation Report

```
# 1. Evaluate the basic metrics on the test data set
test_loss, test_acc = model.evaluate(test_gen, verbose=1)
print(f"Testing Accuracy: {test_acc*100:.2f}%")
# 2. Perform mass prediction on all test images
test_gen.reset()
y_pred = model.predict(test_gen)
y_pred_classes = np.argmax(y_pred, axis=1) # Take the index of the highest
probability
y_true = test_gen.classes # Take the original labels from the directory
# 3. Generate the Classification Report
class_names = list(test_gen.class_indices.keys())
report = classification_report(
y_true,
y_pred_classes,
target_names = class_names
digits=4
)
print(report)
# 4. Generate the Confusion Matrix
cm = confusion_matrix(y_true, y_pred_classes)
```

Based on Code 4.9, the testing does not only rely on the global accuracy metric (test_acc), but is dissected more deeply per class using the *Precision*, *Recall*, and *F1-Score* metrics. In the context of an expert system or plant disease diagnostics, this class analysis is important because an imbalance of detection performance between the *Early Blight*, *Late Blight*, and *Healthy* classes can impact agronomic handling errors. The calculation results of the *Classification Report* are presented in Table 4.8.

Table 4. 8 Classification Report of the Internal Testing Results

Class / Category	Precision	Recall	F1-Score	Support
Early Blight	0.9842	0.9842	0.9842	253
Healthy	0.9765	0.9881	0.9823	253
Late Blight	0.9839	0.9722	0.9780	253
Accuracy			0.9815	757
Macro Avg	0.9816	0.9815	0.9815	757

Class / Category	Precision	Recall	F1-Score	Support
Weighted Avg	0.9816	0.9815	0.9815	757

Based on Table 4.8, the MobileNetV3-Small architecture shows very good classification performance on the internal test data. The model successfully reached an overall accuracy of 98.15% on 757 images, with a macro average (*macro average*) F1-score value of 0.9815. At the class level, very strong performance is shown by the *Early Blight* class, recording the highest precision (*precision*) and F1-score values, each at 0.9842. This high level of precision shows that the model's prediction probability toward that infection is very reliable and has minimal false positive errors (*False Positive*). The *Late Blight* class also shows balanced computational reliability with a precision value of 0.9839 and an F1-score of 0.9780. Meanwhile, the *Healthy* class obtains a very optimal sensitivity metric (*recall*) at 0.9881, indicating that the model detected almost all healthy leaf samples without any being missed.

The balance of performance between classes is also proven by the alignment of the macro average (*macro average*) and weighted average (*weighted average*) values that constantly stay in the range of 0.98. This confirms that the model does not show meaningful prediction bias toward a certain class. This empirical finding indicates that the two-phase *transfer learning* approach combined with *hyperparameter* optimization proved effective in producing a model architecture with very strong visual generalization ability in the classification of potato leaf diseases within the scope of internal testing.

To complement the statistical-metric-based evaluation, the next test results are visualized using the *confusion matrix*. This visualization is used to show the distribution of correct predictions on the main diagonal as well as the location of classification errors between classes in more detail. Thus, the analysis is not only focused on the global accuracy value, but also on the error patterns that occur in each disease category. This visualization is presented in Figure 4.20.

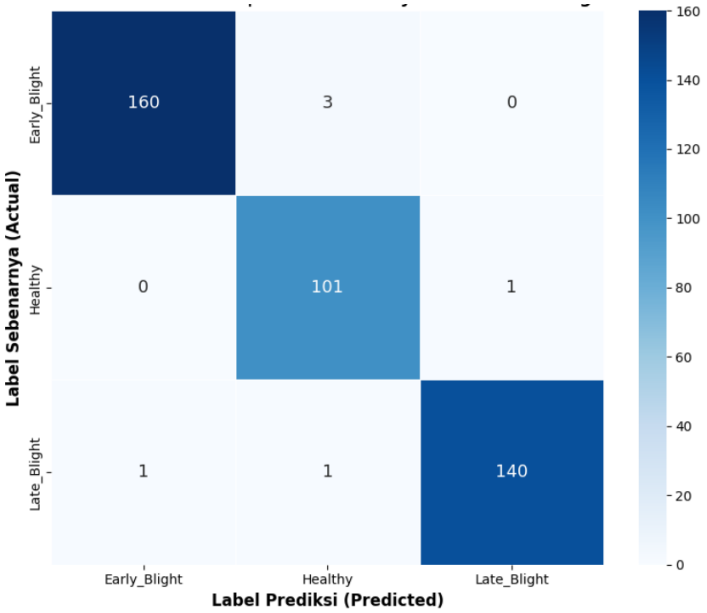


Figure 4. 20 Confusion Matrix on the Internal Testing

Based on Figure 4.20, the majority of the model's predictions are on the main diagonal, which indicates that most of the images were successfully classified correctly. In the Early_Blight class, the model predicted 160 of 163 images correctly, with 3 images wrongly predicted as Healthy. In the Healthy class, the model predicted 101 of 102 images correctly, with 1 image wrongly predicted as Late_Blight. Meanwhile, in the Late_Blight class, the model predicted 140 of 142 images correctly, with 1 image each wrong to the Early_Blight and Healthy classes. This pattern shows that the MobileNetV3Small model has good class-separation ability, with a low error rate that only appears in a small portion of samples that possibly have visual characteristic similarity between classes. To strengthen these visual findings, the next discussion is directed to Output 4.3 in the form of the basic evaluation metrics, namely test loss and test accuracy, as quantitative indicators of the model's overall performance.

4.8.2 External Testing

Although the model recorded brilliant performance in internal testing, the reliability of an artificial intelligence (*AI*) system for the real agricultural environment must be tested using a collection of images acquired outside the laboratory conditions of this research. Therefore, the second-stage testing was carried out by feeding 600 images from a secondary (external) dataset. The calculation results of the *Classification Report* are presented in Table 4.9.

Table 4. 9 Classification Report of the External Testing Results

Class / Category	**Precision**	**Recall**	**F1-Score**	**Support**
Early Blight	0.8571	0.9600	0.9057	200
Healthy	0.9920	0.9600	0.7631	200
Late Blight	0.7649	0.9600	0.8514	200
Accuracy			0.8467	600
Macro Avg	0.8714	0.8467	0.8401	600
Weighted Avg	0.8714	0.8467	0.8401	600

Based on Table 4.9, the testing using the external data set shows a decline in global accuracy to 84.67%. This performance degradation confirms the occurrence of the *domain shift* phenomenon, in which images from a completely new acquisition environment have deviations in visual characteristics such as the lighting spectrum, the texture of the field background, and the camera sensor specifications that deviate from the training data distribution. In the class-level analysis, the MobileNet architecture actually shows robustness of feature extraction in detecting infected specimens. The *Early Blight* and *Late Blight* classes both record very optimal sensitivity metrics (*recall*) at 0.9600. This proves that the model is able to identify and capture almost all leaves that have disease symptoms on a completely new data domain.

However, computational disorientation actually occurs in the feature separation of the normal class, in which the *Healthy* class experiences a drastic decline in *recall* value to 0.6200. This low sensitivity level in the *Healthy* class indicates that the model becomes over-sensitive (*over-sensitive*) to foreign visual elements in the external data (such as dirt spots or shadows on the leaves) and considers them to be pathological features. This is directly correlated with the *Late Blight* class recording a fairly low precision level (*precision*), namely 0.7649. This classification metric pattern empirically proves that on the new data domain, the model experiences difficulty in mapping a firm decision boundary (*decision boundary*) on leaves without lesions, so that it tends to wrongly predict some healthy leaves (38%) into the infected classes. Overall, this finding affirms the challenge of generalization in the implementation of artificial intelligence, in which the fluctuation of visual conditions of the real environment can blur the parameters of leaves assumed to be healthy by the network. The details of the location of these classification errors are visualized in more depth through the *confusion matrix* in Figure 4.21.

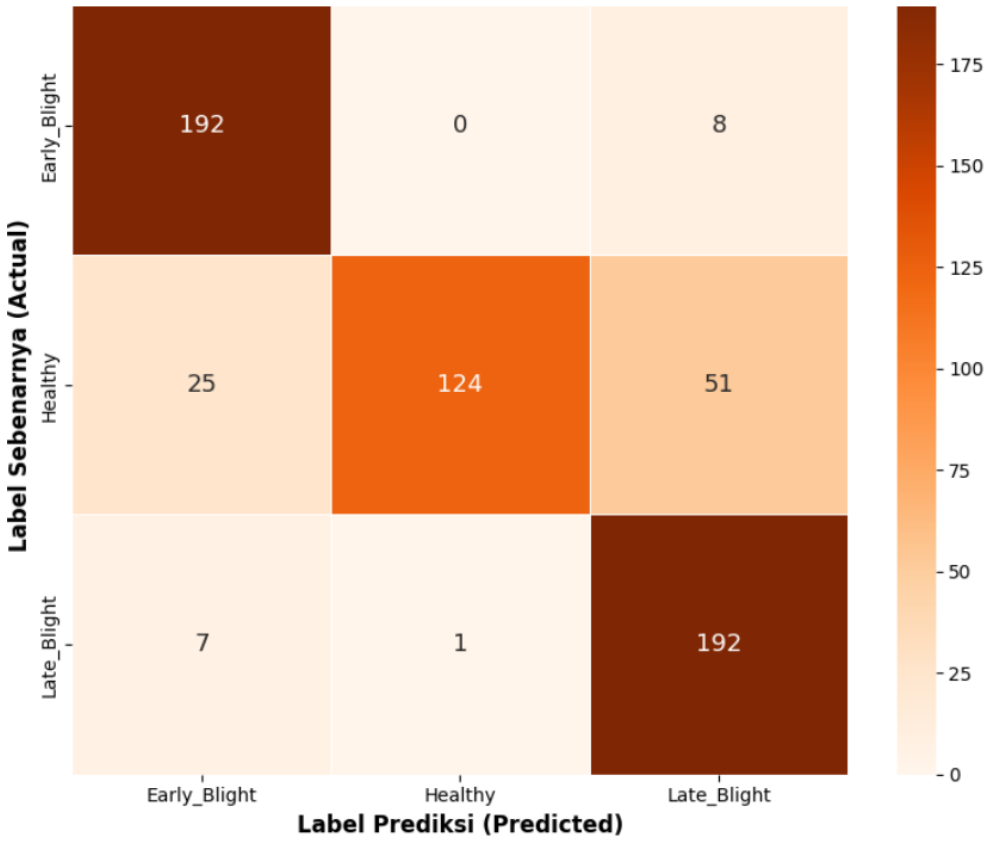


Figure 4. 21 Confusion Matrix on the External Testing

Based on the *Confusion Matrix* visualization in Figure 4.21, the distribution of the location of classification errors (*misclassification*) that occurred in the cross-dataset testing is confirmed precisely. Contrary to the dynamics in the internal testing, the model actually shows very precise feature extraction robustness on the pathological classes. Of the total 200 actual *Early Blight* samples, the model successfully predicted 192 images correctly, in which only 8 images were wrongly classified as *Late Blight* and not a single specimen (0 images) was predicted as healthy. Similar superior performance was also shown in the actual *Late Blight* class, in which the model was able to maintain a very high *True Positive* rate by recognizing 192 images correctly, with a very minimal computational error (*error*) rate (7 images predicted as *Early Blight* and 1 image as *Healthy*).

However, a fairly massive computational prediction anomaly actually occurs in the separation of the *Healthy* class. Of the total 200 actual healthy leaf samples, the model was only able to predict 124 images correctly. Most of the normal leaf specimens were wrongly diagnosed as entering the infected category, namely 51 images predicted as *Late Blight* and 25 images as *Early Blight*. This lopsided cross-error pattern from healthy leaves toward the pathological classes

provides strong visual empirical evidence of the impact of *domain shift*. This affirms that although the model is very reliable and sharp in distinguishing the variation of disease morphology characteristics, the model experiences a hypersensitivity syndrome toward foreign visual elements in the real world.

Based on the comparison between the internal and external test results, it can be concluded that the MobileNetV3-Small architecture has very superior feature representation ability (accuracy 98.15%) as long as the processed images come from the same domain distribution as the training data. However, the performance disparity that appeared in the external testing (accuracy 84.67%) empirically proves the model's vulnerability to the *domain shift* phenomenon. Although the computational robustness of the model in detecting and distinguishing pathogen infections (*Early Blight* and *Late Blight*) proved to be very robust across domains, the model still experiences a false positive bias (*False Positive*) that harms the diagnosis of normal leaves (*Healthy*). This finding provides the final conclusion that the reliability of a *deep learning* model as a universal agronomic diagnostic instrument cannot be achieved only through *hyperparameter* optimization, but depends heavily on the physical enrichment of dataset diversity that encompasses various geographic conditions, natural lighting, and specimen cleanliness levels in the real field at the training phase.

This page is intentionally left blank

CHAPTER V
CONCLUSION AND SUGGESTIONS

This chapter contains the conclusions of the entire series of research that has been carried out, as well as suggestions that are expected to be useful for the development of further studies. The conclusion section summarizes the main findings obtained during the testing process and describes how these findings answer the problem formulation that has been established. Meanwhile, the suggestion section provides constructive recommendations based on the evaluation results, both for the purposes of practical system implementation and as a foundation for future research.

5.1 Conclusion

Based on the results of the design, implementation, and testing of the potato leaf disease classification model using the MobileNetV3-Small architecture based on transfer learning with a two-phase scenario, several conclusions can be drawn as follows:

1. The implementation process of the MobileNetV3-Small architecture was executed through a series of structured computation stages. This implementation began with the preprocessing of a pure dataset that naturally already had a balanced composition (*balanced dataset*), continued with the application of dynamic augmentation (covering spatial transformation and pixel-level manipulation), and the use of ImageNet pre-trained weights (*pre-trained weights*). The built-in classifier layer was modified using a custom *Fully Connected* architecture whose final configuration (such as *dropout* 0.1 and the unfreezing of the top 40 layers) was based on an empirical *hyperparameter* search. Through this flow, the *Small* variant proved to provide compact computational efficiency in extracting visual features compared to the *Large* variant, with an internal test data evaluation performance achievement in the form of a *test accuracy* of 98.15% and a *test loss* of 0.0458.
2. The process of standardizing the pixel distribution on the input images was carried out using the official built-in preprocessing function of the architecture (preprocess_input), which transforms the pixel values to the scale [-1.0, 1.0]. This stage aligns the input image format with the input characteristics of the MobileNetV3-Small pre-trained weights so that the feature extraction process can proceed more consistently.

3. The application of the two-phase computation optimization strategy provided a significant effect on the convergence stability of the network. The use of the Adam *optimizer* (with a *learning rate* of 0.005) in the *Feature Extraction* phase proved effective in accelerating the adaptation of the network's initial weights, while the transition to the SGD Nesterov *optimizer* on the unfreezing of the last 40 layers in the *Fine-Tuning* phase succeeded in maintaining the stability of the weight update without extreme fluctuation. Although this computation strategy succeeded in creating a very robust model in the internal testing environment, the external testing results with a *test accuracy* of 84.67% confirm the occurrence of the *domain shift* phenomenon. This architecture proved very reliable and immune in detecting infected leaves across domains, but the model manifested a bias in the form of visual hypersensitivity toward healthy leaves (*Healthy*). Variations of foreign elements in the real environment such as fluctuations in the lighting spectrum or surface dust tend to be misinterpreted by the model as early features of pathology.

5.2 Suggestions

Although the classification system developed has shown very satisfactory performance, there is still a number of areas for further development. The suggestions that can be given for future research are:

1. Future research can conduct an ablation study or method comparison using the latest generation of *lightweight* architectures, such as MobileNetV4, EfficientNet-Lite, or GhostNet, so that more comprehensive information can be obtained regarding the trade-off between accuracy, inference time, and memory efficiency.

BIBLIOGRAPHY

[1] Lutfi and D. U. Hafriana, 'Analisis Kadar Glukosa pada Kentang Rebus (Solanum Tuberosum) sebagai Pengganti Nasi bagi Penderita Diabetes Melitus dengan Menggunakan Spektrofotometri', *Jurnal Media Laboran*, vol. 10, no. 1, pp. 26–31, 2020.

[2] Badan Pusat Statistik, 'Produksi Tanaman Sayuran dan Buah-buahan Semusim Menurut Provinsi dan Jenis Tanaman — 2024'. Accessed: Sep. 01, 2025. [Online]. Available: https://www.bps.go.id/assets/statistics-table/3/ZUhFd1JtZzJWVVpqWTJsV05XTllhVmhRSzFoNFFUMDkjMw==/produksi-tanaman-sayuran-dan-buah-buahan-semusim-menurut-provinsi-dan-jenis-tanaman----2024.html?year=2024

[3] L. Meno, O. Escuredo, M. S. Rodríguez-Flores, and M. C. Seijo, 'Modification of the tomcast model with aerobiological data for management of potato early blight', *Agronomy*, vol. 10, no. 12, 2020, doi: 10.3390/agronomy10121872.

[4] A. M. Lesmana, R. P. Fadhillah, and C. Rozikin, 'Identifikasi Penyakit pada Citra Daun Kentang Menggunakan Convolutional Neural Network (CNN)', *Jurnal Sains dan Informatika*, vol. 8, no. 1, pp. 21–30, 2022, doi: 10.34128/jsi.v8i1.377.

[5] P. S. A. P. Wulandari, K. T. Martono, and I. P. Windasari, 'Pengembangan Sistem Pendeteksi Gesture Angka pada Tangan secara Realtime Berbasis Android', *Edu Komputika Journal*, vol. 7, no. 1, pp. 1–9, 2020, doi: 10.15294/edukomputika.v7i1.38655.

[6] F. Agustina and Z. A. Ardiansyah, 'Identifikasi Citra Daging Ayam Kampung dan Broiler Menggunakan Metode GLCM dan Klasifikasi-NN Image Identification of Local Chicken Meat and Broiler Chicken MeatUsing GLCM Method and K-NN Classification', *25 Jurnal Infokam*, vol. XVI, no. 1, 2020.

[7] T. D. Salka, M. B. Hanafi, S. M. S. A. A. Rahman, D. B. M. Zulperi, and Z. Omar, 'Plant leaf disease detection and classification using convolution neural networks model: a review', *Artif. Intell. Rev.*, vol. 58, no. 10, Oct. 2025, doi: 10.1007/s10462-025-11234-6.

[8] J. Zhang, X. Yang, X. Fu, B. Wang, and H. Li, 'LDL-MobileNetV3S: an enhanced lightweight MobileNetV3-small model for potato leaf disease diagnosis through multi-module fusion', *Front. Plant Sci.*, vol. 16, 2025, doi: 10.3389/fpls.2025.1656731.

[9] C. Bi, S. Xu, N. Hu, S. Zhang, Z. Zhu, and H. Yu, 'Identification Method of Corn Leaf Disease Based on Improved Mobilenetv3 Model', *Agronomy*, vol. 13, no. 2, pp. 1–17, 2023, doi: 10.3390/agronomy13020300.

[10] C. Y. Chang and C. C. Lai, 'Potato Leaf Disease Detection Based on a Lightweight Deep Learning Model', *Mach. Learn. Knowl. Extr.*, vol. 6, no. 4, pp. 2321–2335, Dec. 2024, doi: 10.3390/make6040114.

[11] J. Rashid, I. Khan, G. Ali, S. H. Almotiri, M. A. Alghamdi, and K. Masood, 'Multi-level deep learning model for potato leaf disease recognition', *Electronics (Switzerland)*, vol. 10, no. 17, Sep. 2021, doi: 10.3390/electronics10172064.

[12] M. I. B. MD Ilias Bappi, D. J. R. David J. Richter, and K. Kim, 'Assessing the Effectiveness of Augmentation Techniques in Enhancing Plant Leaf Disease Classification', *Korean Institute of Smart Media*, vol. 14, no. 1, pp. 17–25, Jan. 2025, doi: 10.30693/SMJ.2025.14.1.17.

[13] S. K. Shah, M. B. M. Su'ud, A. Khan, M. M. Alam, and M. Ayaz, 'PLDC-Net: Potato Leaf Disease Classification Network Based on an Efficient Convolutional Neural Network', *Engineering Reports*, vol. 7, no. 7, Jul. 2025, doi: 10.1002/eng2.70178.

[14] F. Khan, S. Sattar, S. M. Zaid, M. Bilal, S. K. Shah, and M. Ayaz, 'Enabling Early Treatment: A Deep Learning Approach to Multi-Class Potato Leaf Disease Identification'.

[15] G. Dai and J. Fan, 'An Industrial-Grade Solution for Crop Disease Image Detection Tasks', *Front. Plant Sci.*, vol. 13, Jun. 2022, doi: 10.3389/fpls.2022.921057.

[16] J. Xu, Y. Li, L. Kaur, J. Singh, and F. Zeng, 'Functional Food Based on Potato', *Foods*, vol. 12, no. 11, 2023, doi: 10.3390/foods12112145.

[17] H. Rasheed, D. Ahmad, and J. Bao, 'Genetic Diversity and Health Properties of Polyphenols in Potato', *Antioxidants*, vol. 11, no. 4, 2022, doi: 10.3390/antiox11040603.

[18] F. Kang, J. Li, C. Wang, and F. Wang, 'A Lightweight Neural Network-Based Method for Identifying Early-Blight and Late-Blight Leaves of Potato', *Applied Sciences (Switzerland)*, vol. 13, no. 3, 2023, doi: 10.3390/app13031487.

[19] L. Meno, I. K. Abuley, O. Escuredo, and M. C. Seijo, 'Suitability of Early Blight Forecasting Systems for Detecting First Symptoms in Potato Crops of NW Spain', *Agronomy*, vol. 12, no. 7, pp. 8–12, 2022, doi: 10.3390/agronomy12071611.

[20] L. J. Stridh, G. Malm, Å. Lankinen, and E. Liljeroth, 'Field and Management Factors Can Reduce Potato Early Blight Severity: an Observational Study on Farms Combined with Field Trials in Southern Sweden', *Potato Res.*, vol. 67, no. 3, pp. 833–859, 2024, doi: 10.1007/s11540-023-09669-x.

[21] C. I. Fernández, B. Leblon, A. Haddadi, K. Wang, and J. Wang, 'Potato late blight detection at the leaf and canopy levels based in the red and red-edge spectral regions', *Remote Sens. (Basel).*, vol. 12, no. 8, pp. 1–27, 2020, doi: 10.3390/RS12081292.

[22] Z. Chi *et al.*, 'Large-Scale Monitoring of Potatoes Late Blight Using Multi-Source Time-Series Data and Google Earth Engine', *Remote Sens. (Basel).*, vol. 17, no. 6, 2025, doi: 10.3390/rs17060978.

[23] U. Khultsum and G. Taufik, 'Komparasi Kinerja DenseNet 121 dan MobileNet untuk Klasifikasi Citra Penyakit Daun Kentang', *JURIKOM (Jurnal Riset Komputer)*, vol. 10, no. 2, p. 558, Apr. 2023, doi: 10.30865/jurikom.v10i2.6047.

[24] E. Firasari, F. Lia, D. Cahyanti, and U. N. Mandiri, 'Implementasi Deep Learning pada Deteksi Penyakit Daun Kentang dengan Arsitektur InceptionResNetV2', *Journal Computer Science*, vol. 4, no. 1, 2025, [Online]. Available: https://www.kaggle.com/datasets/rizwan12345

[25] D. Aji Pangestu, O. Qomaruddin Aziz, and C. Crysdian, 'Klasifikasi Penyakit pada Tanaman Berdasarkan Citra Daun Menggunakan Metode Convolutional Neural Network', MEI, 2025.

[26] Vysakh V Mohan, Pradeep Prakash, and Resmi K R, 'A Survey on Deep Learning Concepts and Techniques', *International Journal of Advanced Research in Science, Communication and Technology*, pp. 20–27, Jun. 2022, doi: 10.48175/ijarsct-4903.

[27] B. Dhanalaxmi, B. N. Kumar, Y. Raju, and R. S. R. Channapragada, 'MobileNetV3: an efficient deep learning-based feature selection and classification technique for cardiovascular disease', *Journal of Engineering and Applied Science*, vol. 72, no. 1, pp. 1–33, 2025, doi: 10.1186/s44147-025-00654-4.

[28] A. Gao, A. J. Geng, Y. P. Song, L. L. Ren, Y. Zhang, and X. Han, 'Detection of maize leaf diseases using improved MobileNet V3-small', *International Journal of Agricultural and Biological Engineering*, vol. 16, no. 3, pp. 225–232, 2023, doi: 10.25165/j.ijabe.20231603.7799.

[29] L. Wang, J. Sun, X. Wu, J. Shen, B. Lu, and W. Tan, 'Identification of crop diseases using improved convolutional neural networks', *IET Computer Vision*, vol. 14, no. 7, pp. 538–545, 2020, doi: 10.1049/iet-cvi.2019.0136.

[30] J. Lu, L. Tan, and H. Jiang, 'Review on convolutional neural network (CNN) applied to plant leaf disease classification', *Agriculture (Switzerland)*, vol. 11, no. 8, pp. 1–18, 2021, doi: 10.3390/agriculture11080707.

[31] Z. Jiao, H. Dong, and N. Diao, 'Separable CenterNet Detection Network Based on MobileNetV3—An Optimization Approach for Small-Object and Occlusion Issues', *Mathematics*, vol. 12, no. 16, 2024, doi: 10.3390/math12162524.

[32] F. Qararyah, M. W. Azhar, M. A. Maleki, and P. Trancoso, 'Fusing Depthwise and Pointwise Convolutions for Efficient Inference on GPUs', *ACM International Conference Proceeding Series*, pp. 58–67, 2024, doi: 10.1145/3677333.3678153.

[33] A. T. Khan, S. M. Jensen, A. R. Khan, and S. Li, 'Plant disease detection model for edge computing devices', *Front. Plant Sci.*, vol. 14, no. December, pp. 1–10, 2023, doi: 10.3389/fpls.2023.1308528.

[34] D. Zhou, Q. Hou, Y. Chen, J. Feng, and S. Yan, 'Rethinking Bottleneck Structure for Efficient Mobile Network Design', *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 12348 LNCS, pp. 680–697, 2020, doi: 10.1007/978-3-030-58580-8_40.

[35] L. Alzubaidi *et al.*, *Review of deep learning: concepts, CNN architectures, challenges, applications, future directions*, vol. 8, no. 1. Springer International Publishing, 2021. doi: 10.1186/s40537-021-00444-8.

[36] B. C. Geiger and G. Kubin, 'Information bottleneck: Theory and applications in deep learning', *Entropy*, vol. 22, no. 12, pp. 1–4, 2020, doi: 10.3390/e22121408.

[37] X. Zhang and X. Xu, 'PlantClassiNet: A Dual-Modal Fine-Tuning Framework for CNN-Based Plant Disease Classification', *Applied Sciences (Switzerland)*, vol. 16, no. 1, Jan. 2026, doi: 10.3390/app16010170.

[38] M. Xu, S. Yoon, Y. Jeong, and D. S. Park, 'Transfer learning for versatile plant disease recognition with limited data', *Front. Plant Sci.*, vol. 13, no. November, pp. 1–14, 2022, doi: 10.3389/fpls.2022.1010981.

[39] Z. R. Mair, R. Heriansyah, and L. O. H. S. Sagala, 'Performance Comparison of Adam and SGD Optimizers in Transfer Learning Based CNN for Banana Leaf Disease Classification', *Jurnal Sains, Nalar, dan Aplikasi Teknologi Informasi*, vol. 5, no. 1, pp. 8–15, Jan. 2026, doi: 10.20885/snati.v5.i1.43901.

[40] Y. Sun, Y. Liu, H. Zhou, and H. Hu, 'Plant diseases identification through a discount momentum optimizer in deep learning', *Applied Sciences (Switzerland)*, vol. 11, no. 20, Oct. 2021, doi: 10.3390/app11209468.

[41] D. L. S. Dimara, S. W. Putri, R. Amelia, Z. I. Arishandy, and A. M. Rizki, 'Penerapan Convolutional Neural Network (CNN) dalam Klasifikasi Citra MRI untuk Deteksi Tumor Otak Manusia', *KERNEL: Jurnal Riset Inovasi Bidang Informatika dan Pendidikan Informatika*, vol. 4, no. 2, pp. 70–77, Dec. 2023, doi: 10.31284/j.kernel.2023.v4i2.6960.

[42] K. Arjuntama Satria, I. Wayan Supriana, J. Raya Kampus Udayana, B. Jimbaran, and K. Selatan, 'Perbandingan CNN dan SVM untuk Klasifikasi Citra Rempah Indonesia', *JNATIA*, vol. 4, no. 2, 2026.

[43] T. R. Mahesh *et al.*, 'Transformative Breast Cancer Diagnosis using CNNs with Optimized ReduceLROnPlateau and Early Stopping Enhancements', *International Journal of Computational Intelligence Systems*, vol. 17, no. 1, Dec. 2024, doi: 10.1007/s44196-023-00397-1.

[44] A. Wibowo, R. W. Sembiring, and S. Solikhun, 'Deep Learning Based MobileNet Optimization For High Accuracy Classification Of Toddler Stunting', *Jurnal Teknik Informatika (Jutif)*, vol. 7, no. 2, pp. 948–960, Apr. 2026, doi: 10.52436/1.jutif.2026.7.2.5382.

[45] D. Lizard, S. Dimara, B. Rahmat, and H. Maulana, 'IDENTIFIKASI PENYAKIT DAUN PADI DENGAN METODE TRANSFER LEARNING MOBILENET-SUPPORT VECTOR MACHINE', 2025.

[46] J. Kozak, B. Probierz, K. Kania, and P. Juszczuk, 'Preference-Driven Classification Measure', *Entropy*, vol. 24, no. 4, pp. 1–24, 2022, doi: 10.3390/e24040531.

[47] G. Fenu and F. M. Malloci, 'Evaluating Impacts between Laboratory and Field-Collected Datasets for Plant Disease Classification', *Agronomy*, vol. 12, no. 10, 2022, doi: 10.3390/agronomy12102359.

[48] B. Min, T. Kim, D. Shin, and D. Shin, 'Data Augmentation Method for Plant Leaf Disease Recognition', *Applied Sciences (Switzerland)*, vol. 13, no. 3, pp. 1–17, 2023, doi: 10.3390/app13031465.

[49] L. Bi and G. Hu, 'Improving Image-Based Plant Disease Classification With Generative Adversarial Network Under Limited Training Set', *Front. Plant Sci.*, vol. 11, no. December, pp. 1–12, 2020, doi: 10.3389/fpls.2020.583438.

[50] A. Naufal Abiyyu and M. Rahardi, ‘Comparison of Transfer learning Models MobileNetV3-Large and EfficientNet-B0 for Rice Leaf Disease Classification’, 2026. [Online]. Available: http://jurnal.polibatam.ac.id/index.php/JAIC

[51] A. CHAMOLI and A. KUMAR, ‘A descriptive analysis of plant leaf disease detection using machine learning and deep learning models: a systematic review’, *Turkish Journal of Electrical Engineering and Computer Sciences*, vol. 34, no. 3, pp. 342–362, May 2026, doi: 10.55730/1300-0632.4179.

APPENDICES

1. Program Code 4.2

Program Code 4.2 – Global Configuration (Hyperparameter & Path)

```
# Path direktori dataset
TRAIN_DIR = '/content/dataset_lokal/Training'
VAL_DIR   = '/content/dataset_lokal/Validation'
TEST_DIR  = '/content/dataset_lokal/Testing'

# Hyperparameter
IMG_HEIGHT   = 224          # Resolusi standar MobileNetV3
IMG_WIDTH    = 224          # Resolusi lebar standar
IMG_SIZE     = (IMG_HEIGHT, IMG_WIDTH)
INPUT_SHAPE  = (IMG_HEIGHT, IMG_WIDTH, 3)  # 3 = channel RGB

BATCH_SIZE   = 16           # Jumlah citra per iterasi. Ditetapkan
16 agar pembaruan
                            # bobot lebih sering dan optimal untuk
dataset berskala sedang
NUM_CLASSES  = 3            # Target klasifikasi: Early Blight, Late
Blight, & Healthy

# Batas maksimal iterasi (epoch) untuk skenario pelatihan dua fase
EPOCHS_P1    = 30           # Epoch Phase 1 — Feature Extraction
(base frozen)
EPOCHS_P2    = 30           # Epoch Phase 2 — Fine-Tuning dengan SGD

# Pengaturan Laju Pembelajaran (Learning Rate) untuk 2 Optimizer
berbeda
LR_P1        = 1e-3         # Laju pembelajaran agresif (0.001)
untuk Optimizer Adam
LR_P2        = 1e-4         # Laju pembelajaran konservatif (0.0001)
untuk Optimizer SGD
MOMENTUM_P2  = 0.9          # Nilai momentum untuk SGD guna mencegah
osilasi dan menjaga stabilitas arah gradien pada fase Fine-Tuning.

# Menentukan nama file untuk menyimpan bobot model terbaik secara
otomatis
MODEL_PATH_P1 = 'best_mobilenetv3_phase1.h5'
MODEL_PATH_P2 = 'best_mobilenetv3_finetuned.h5'

print("✓ Konfigurasi global berhasil dimuat.")
```

2. Program Code 4.3

Program Code 4.3 – Initialization of Training Data Preprocessing and Augmentation

```
def build_data_generators(train_dir, val_dir, test_dir, img_size, batch_size):
    # 1. Inisialisasi fungsi preprocessing resmi bawaan MobileNetV3
    mobilenet_preprocess = tf.keras.applications.mobilenet_v3.preprocess_input
    # 2. Konfigurasi generator data latih (dilengkapi augmentasi spasial)
    train_datagen = ImageDataGenerator(
        preprocessing_function = mobilenet_preprocess,
        rotation_range         = 20,
        horizontal_flip        = True,
        zoom_range             = 0.1,
        fill_mode              = 'nearest'
    )
    # 3. Konfigurasi generator validasi dan uji (TANPA augmentasi)
    val_test_datagen = ImageDataGenerator(
        preprocessing_function = mobilenet_preprocess
    )
    # 4. Memuat citra dari direktori ke dalam bentuk generator
    train_gen = train_datagen.flow_from_directory(
        train_dir, target_size=img_size, batch_size=batch_size,
        class_mode='categorical', shuffle=True
    )
    val_gen = val_test_datagen.flow_from_directory(
        val_dir, target_size=img_size, batch_size=batch_size,
        class_mode='categorical', shuffle=False
    )
    test_gen = val_test_datagen.flow_from_directory(
        test_dir, target_size=img_size, batch_size=batch_size,
        class_mode='categorical', shuffle=False
    )

    return train_gen, val_gen, test_gen
     # 5. Implementasi Simulasi Kondisi Lingkungan Nyata
def custom_augment_train(image):
    # Implementasi Gaussian Blur (Probabilitas 30%)
    if np.random.rand() < 0.3:
        ksize = np.random.choice([3, 5])
        image = cv2.GaussianBlur(image, (ksize, ksize), sigmaX=0)
    # Implementasi Gaussian Noise (Probabilitas 30%)
    if np.random.rand() < 0.3:
        noise = np.random.normal(0.0, 10.0, image.shape).astype(np.float32)
        image = np.clip(image + noise, 0.0, 255.0)
    # Implementasi Brightness Variation (Probabilitas 40%)
    if np.random.rand() < 0.4:
        factor = np.random.uniform(0.7, 1.3)
        image  = np.clip(image * factor, 0.0, 255.0)

    return mobilenet_preprocess(image)
```

3. Program Code 4.6

Program Code 4.6 – Layer Unlocking and Phase 2 Model Training

```
def unfreeze_top_layers(base_model, n_layers_to_unfreeze=20):
    """
    Membuka 20 layer terakhir base model untuk fine-tuning.
    Layer BatchNormalization tetap dibekukan agar statistik stabil.
    """
    base_model.trainable = True
    total_layers = len(base_model.layers)
    freeze_until = total_layers - n_layers_to_unfreeze
    unfrozen = 0
    for i, layer in enumerate(base_model.layers):
        if i < freeze_until:
            layer.trainable = False
        else:
            if isinstance(layer, tf.keras.layers.BatchNormalization):
                layer.trainable = False
            else:
                layer.trainable = True
                unfrozen += 1

    print(f"Fine-Tuning: {unfrozen} layer terakhir dibuka dari {total_layers} total layer")
    print(f"Layer di-freeze : {freeze_until} | Layer trainable : {unfrozen}")

def compile_phase2(model, lr_p2=1e-4, momentum_p2=0.9):
    """
    Kompilasi ulang model untuk Fase 2 menggunakan SGD + Nesterov Momentum.
    """
    model.compile(
        optimizer=tf.keras.optimizers.SGD(
            learning_rate=lr_p2,
            momentum=momentum_p2,
            nesterov=True
        ),
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )
    print(f"✓ Phase 2 dikompilasi | Optimizer: SGD (Nesterov) | LR = {lr_p2} | Momentum =
{momentum_p2}")

# 1) Buka 20 layer terakhir untuk fine-tuning
unfreeze_top_layers(base_model, n_layers_to_unfreeze=20)

# 2) Kompilasi ulang untuk Fase 2
compile_phase2(model, lr_p2=1e-4, momentum_p2=0.9)

# 3) Callback Fase 2 (EarlyStopping + Checkpoint)
callbacks_p2 = get_callbacks(MODEL_PATH_P2, phase='P2')

# 4) Training Fase 2
history_p2 = model.fit(
    train_gen,
    epochs=EPOCHS_P2,
    validation_data=val_gen,
    callbacks=callbacks_p2,
    verbose=1)
```

4. Program Code 4.8

Program Code 4.8 – Training History Visualization

```
# Memvisualisasikan kurva Accuracy dan Loss selama training.
def plot_training_history(history_p1, history_p2=None) -> None:
# Gabungkan history kedua phase
    acc      = history_p1.history['accuracy']
    val_acc  = history_p1.history['val_accuracy']
    loss     = history_p1.history['loss']
    val_loss = history_p1.history['val_loss']
    p1_end   = len(acc)  # Penanda akhir Phase 1
    if history_p2 is not None:
        acc      += history_p2.history['accuracy']
        val_acc  += history_p2.history['val_accuracy']
        loss     += history_p2.history['loss']
        val_loss += history_p2.history['val_loss']
    epochs_range = range(1, len(acc) + 1)
    fig, axes = plt.subplots(1, 2, figsize=(16, 6))
    fig.suptitle(
        'MobileNetV3Small — Klasifikasi Penyakit Daun Kentang\nTraining History',
        fontsize=14, fontweight='bold', y=1.02
    )
    # Plot Accuracy
    ax = axes[0]
    ax.plot(epochs_range, acc,     label='Train Accuracy', linewidth=2,
color='#2196F3')
    ax.plot(epochs_range, val_acc, label='Val Accuracy',   linewidth=2,
color='#FF9800',
            linestyle='--')
    if history_p2 is not None:
        ax.axvline(x=p1_end + 0.5, color='gray', linestyle=':', linewidth=1.5,
                   label='Mulai Fine-Tuning')
    ax.set_xlabel('Epoch', fontsize=11)
    ax.set_ylabel('Accuracy', fontsize=11)
    ax.set_title('Accuracy per Epoch', fontsize=12, fontweight='bold')
    ax.legend(fontsize=10)
    ax.grid(True, alpha=0.3)
    ax.set_ylim([0, 1.05])
    # ── Plot Loss
    ax = axes[1]
    ax.plot(epochs_range, loss,     label='Train Loss', linewidth=2,
color='#F44336')
    ax.plot(epochs_range, val_loss, label='Val Loss',   linewidth=2,
color='#9C27B0',
            linestyle='--')
    if history_p2 is not None:
        ax.axvline(x=p1_end + 0.5, color='gray', linestyle=':', linewidth=1.5,
                   label='Mulai Fine-Tuning')
    ax.set_xlabel('Epoch', fontsize=11)
    ax.set_ylabel('Loss', fontsize=11)
    ax.set_title('Loss per Epoch', fontsize=12, fontweight='bold')
    ax.legend(fontsize=10)
    ax.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig('training_history.png', dpi=150, bbox_inches='tight')
    plt.show()
    print("✓ Grafik training disimpan → training_history.png")
```

5. Certificate of Turnitin Check Results

KEMENTRIAN PENDIDIKAN TINGGI, SAINS, DAN TEKNOLOGI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN" JAWA TIMUR
FAKULTAS ILMU KOMPUTER
Jl. Raya Rungkut Madya Gunung Anyar Surabaya Telp.(031) 870 6369
Laman: https://upnjatim.ac.id, Email: humas@upnjatim.ac.id

SURAT KETERANGAN
HASIL PEMERIKSAAN TINGKAT PLAGIARISME
NOMOR : 4807/UN63/SISANTIK/FIK/VII/2026

Yang bertanda-tangan di bawah ini,

Nama	:	Dr. I GEDE SUSRAMA MAS DIYASA, S.T., M.T.
NI P3K	:	197006192021211009
Jabatan	:	Wakil Dekan Bidang Akademik - Fakultas Ilmu Komputer

dengan ini menerangkan bahwa telah dilakukan pemeriksaan tingkat kesamaan (plagiarisme) menggunakan software Turnitin terhadap dokumen dalam daftar di bawah ini :

Judul Karya Tulis	:	POTATO LEAF DISEASE CLASSIFICATION USING TRANSFER LEARNING BASED MOBILENETV3SMALL ARCHITECTURE_MUHAMMAD AHSANUR RAFI
Jenis Publikasi	:	Skripsi
Penulis	:	MUHAMMAD AHSANUR RAFI (21081010305)
Tingkat Kesamaan (%)	:	18 %
Operator Pemeriksa	:	ARIES SATRIYA YS S.Kom.

Demikian Surat Keterangan ini dibuat dengan sebenarnya, agar dapat digunakan sebagaimana mestinya.

Surabaya, 21 Juli 2026
Wakil Dekan Bidang Akademik - Fakultas Ilmu Komputer

TTE oleh :
I GEDE SUSRAMA MAS DIYASA
21 Juli 2026 16:29:32 WIB
Verifikasi melalui
https://ivo.upnjatim.ac.id

Dr. I GEDE SUSRAMA MAS DIYASA, S.T., M.T.
NI P3K 197006192021211009

Balai Sertifikasi Elektronik
Dokumen ini ditandatangani secara elektronik menggunakan Sertifikat Elektronik yang diterbitkan oleh Balai Sertifikasi Elektronik (BSrE), Badan Siber dan Sandi Negara (BSSN)